

# Chapter 3

## The Data Link Layer

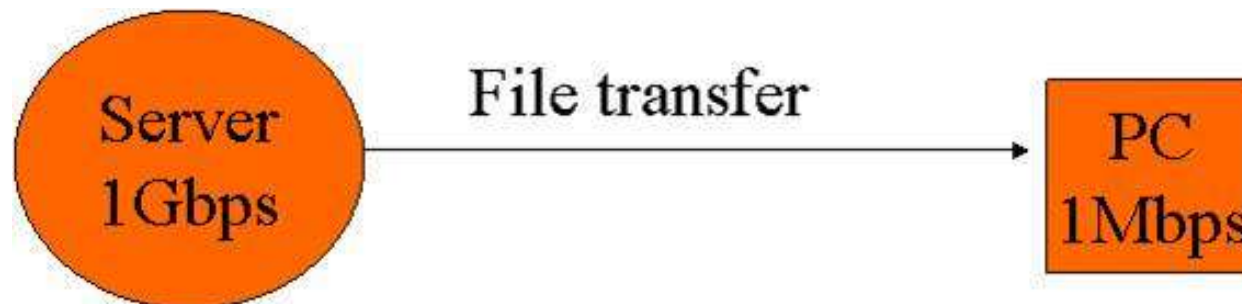
# Outline

- *Position, Functions & Services of Data Link Layer* (Textbooks 3.1.1)
- Design Issues
  - ◆ Framing
  - ◆ Error Control
  - ◆ Flow Control
- Example Protocols
  - ◆ HDLC
  - ◆ PPP

(8-9 hours' Lecture)

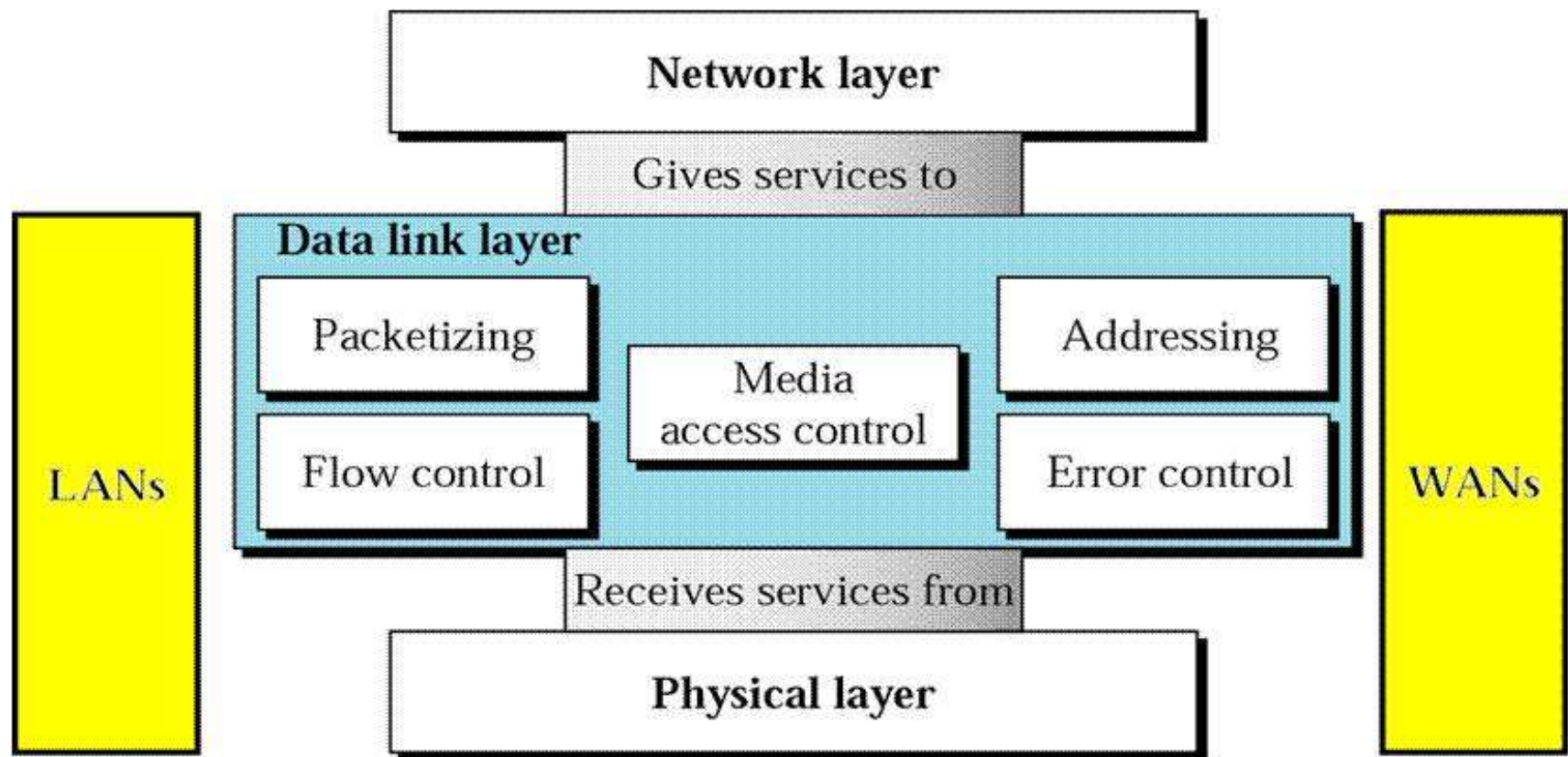
# Why do we need Data Link Control?

- Probable transmission bit errors in physical layer
- Receiver may need to prevent too much data coming in
- Synchronization & interface techniques may not be enough



# Position and Functions

- Data link layer has responsibility of transferring data (frame) from one node to adjacent node in a reliable and efficient way over a link



[Forouzan]





# Data Link Layer Design Issues

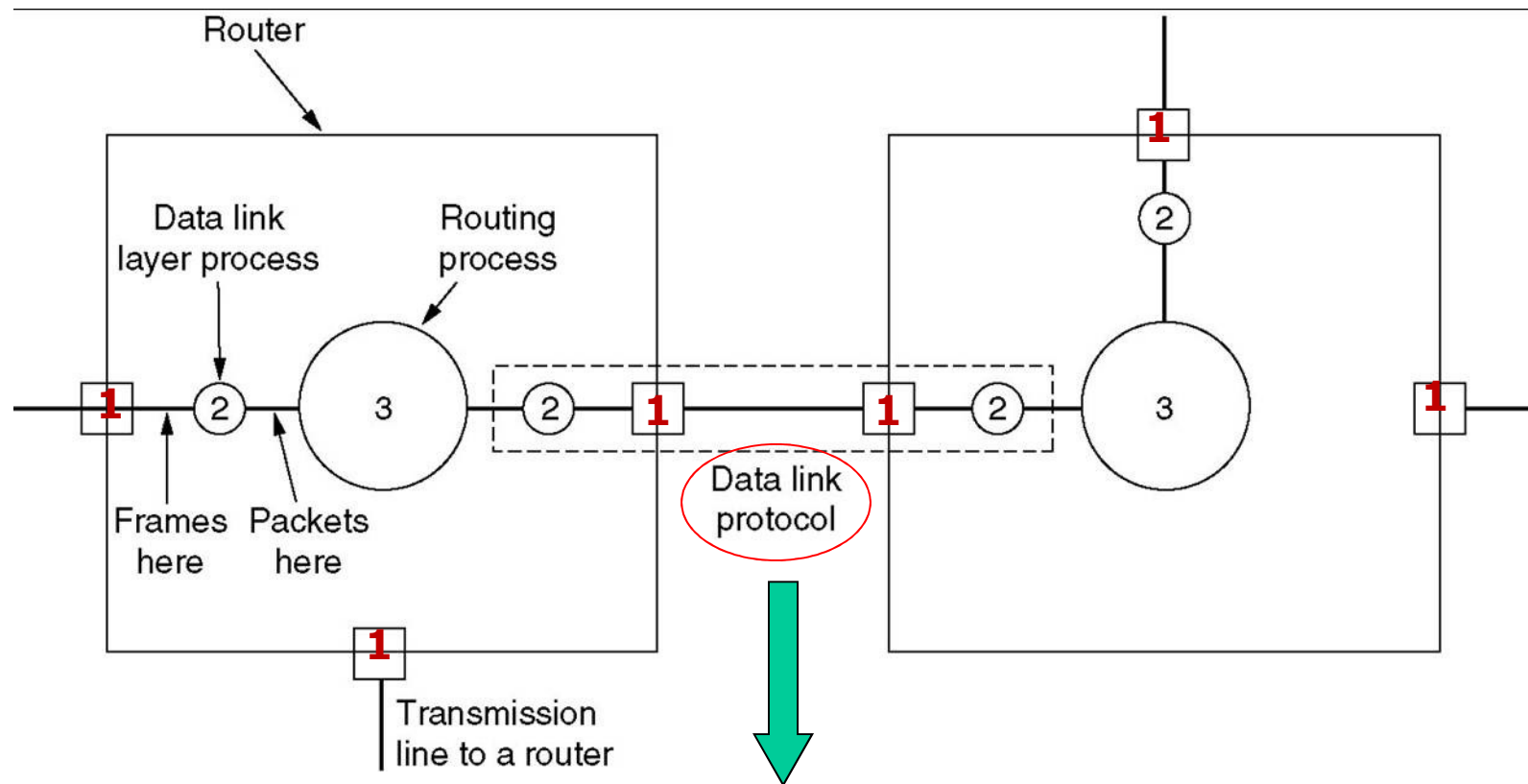
## ■ Goals

- ◆ Study the algorithms for achieving **reliable, efficient communication** between two **adjacent** machines

## ■ Design Issues

- ◆ Services Provided to the Network Layer
- ◆ Framing
  - break up the bit stream into discrete frames
- ◆ Error Control
  - Ensure data received equal to data sent
- ◆ Flow Control
  - **Slow receivers** must not be swamped by **fast** senders

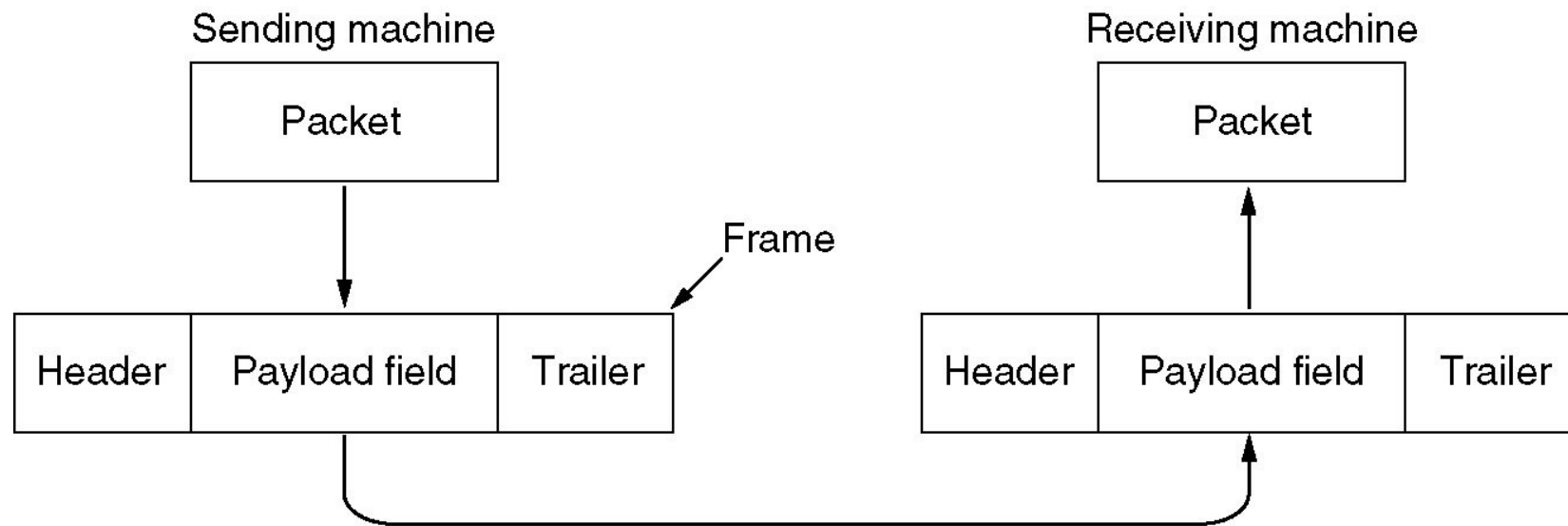
# Functions of Data Link Layer: Illustration



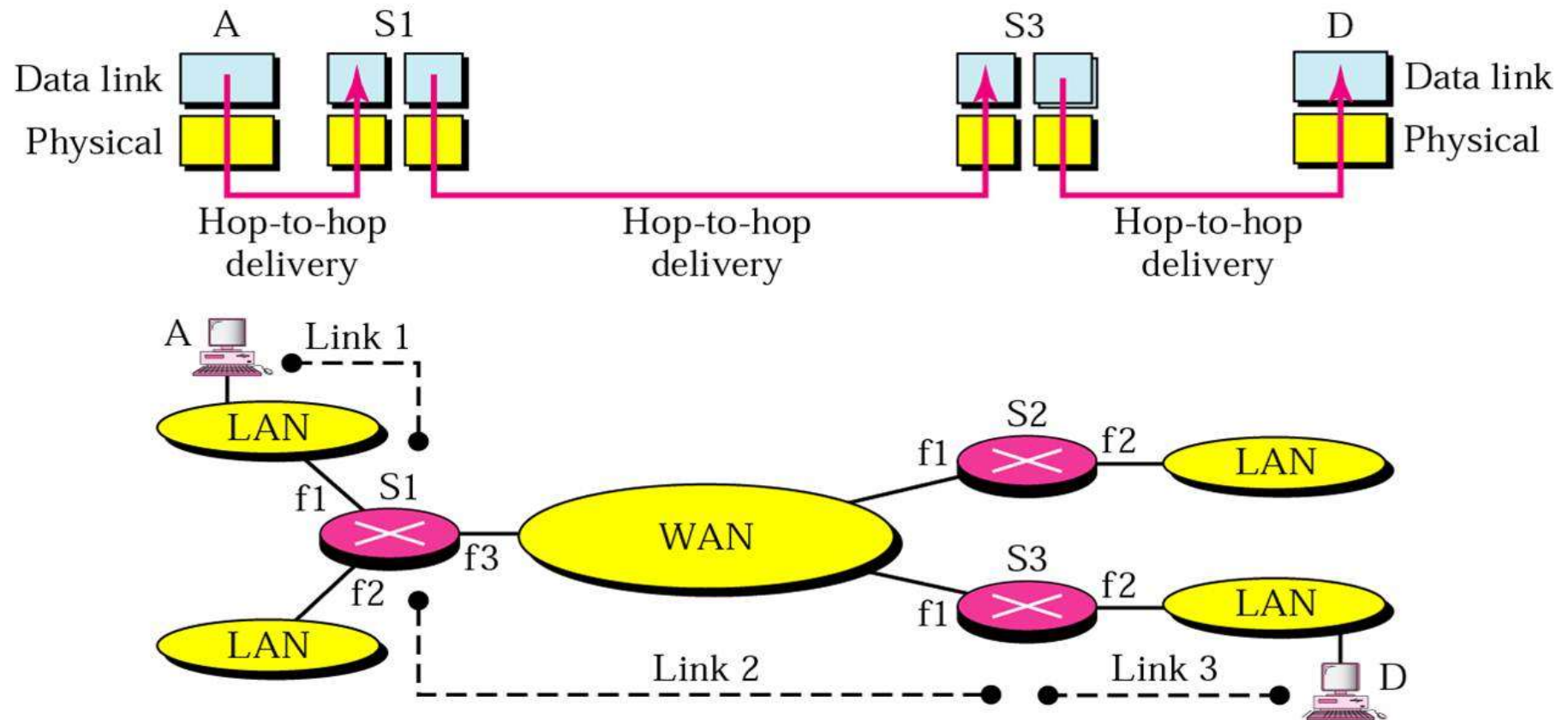
- ➡ **Flow Control:** How much data shall be sent?
- ➡ **Error Control:** How can errors be corrected?
- ➡ **Access Control:** who shall send? → chapter 4

# Packets vs. Frames

## Encapsulation(封装)

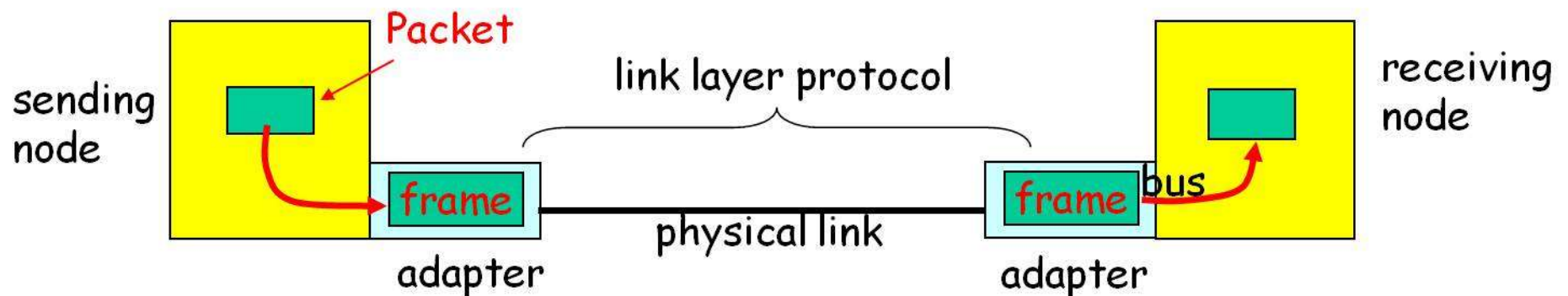


# Data Link Layer: hop-to-hop communication(逐跳通信)



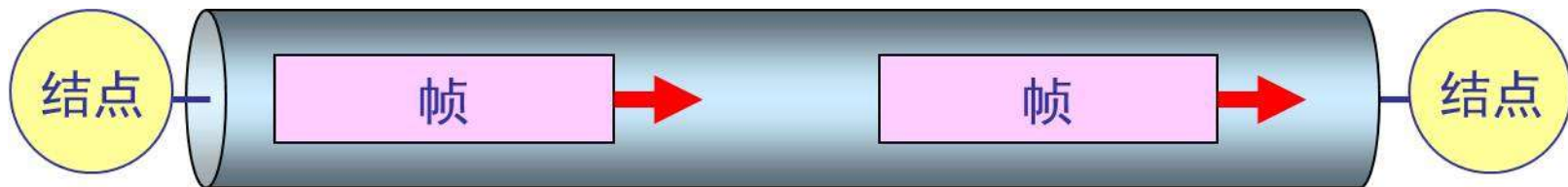
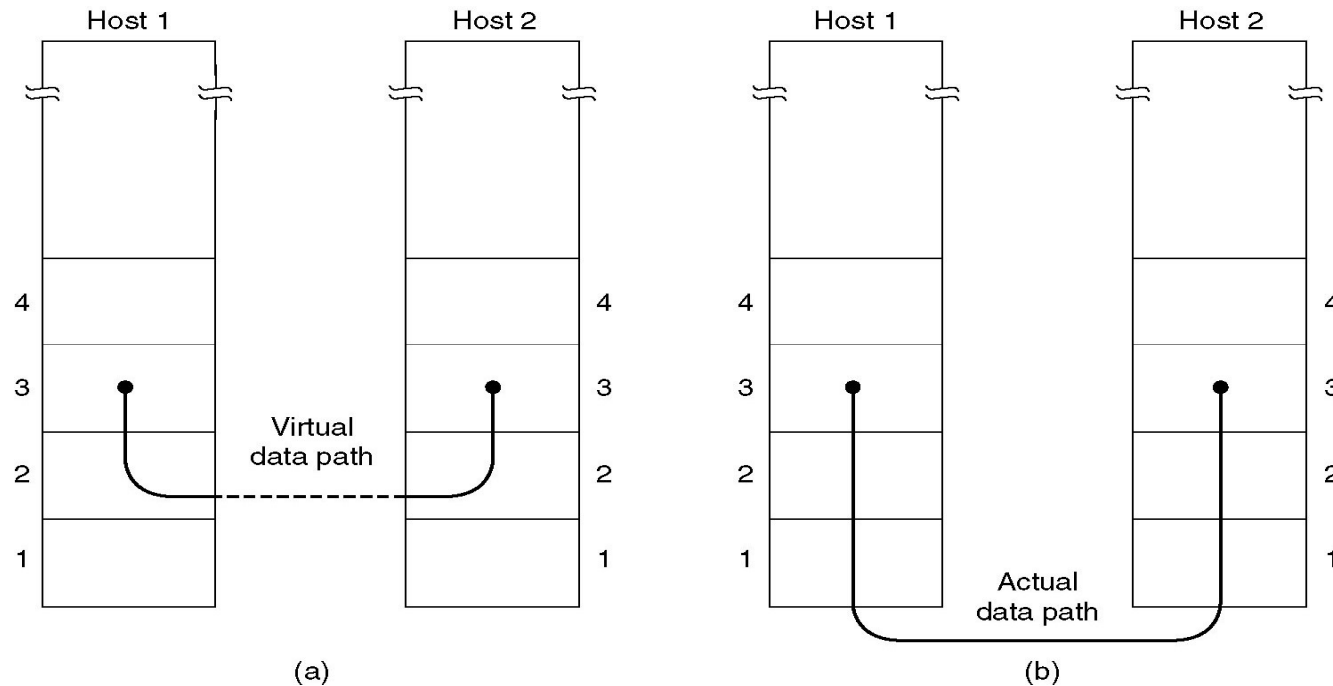
# Data Link Layer: Adapters Communicating

- Sending side:
  - ◆ Encapsulates network packet in a frame
  - ◆ Adds error checking bits, reliable transfer, flow control, etc.
- Receiving side
  - ◆ Looks for errors, reliable transfer, flow control, etc
  - ◆ Extracts packet, passes to receiving node
- Link layer implemented in "adapter" (NIC, 网卡)
  - ◆ Ethernet card, PCMCIA card, 802.11 card



# Services Provided to Network Layer (1)

## Virtual communication vs. Actual communication



数据链路层像个数字管道

# Services Provided to Network Layer(2)

## ■ Connectionless services

- ◆ Unacknowledged connectionless service
  - No acknowledgement, no logical connection
  - Most LANs use such service
- ◆ Acknowledged connectionless service
  - Each frame acknowledged, used in unreliable channel, such as wireless systems

## ■ Acknowledged connection-oriented service

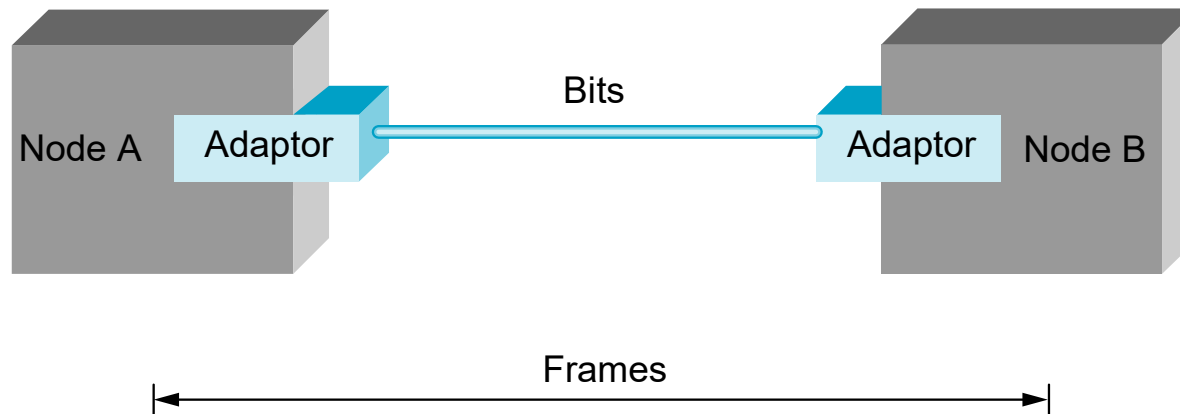
- ◆ A connection established, frames are numbered, reliable transmission guaranteed, eg. ATM

# Outline

- Position, Functions & Services of Data Link Layer
- Design Issues
  - ◆ Framing (Textbook 3.1.2)
  - ◆ Error Control
  - ◆ Flow Control
- Example Protocols
  - ◆ HDLC
  - ◆ PPP



# Framing



- A sequence of bits is sent from node A to node B over a point-to-point link. Node B must recognize exactly what **set of bits** constitutes a frame, that is, it must determine where the frame **begins** and **ends**.

# Framing(成帧)

- Requirements: Framing(拆分比特流)
- Constraints
  - ◆ 接收方容易实现——simple
  - ◆ 与传输的消息内容无关——code independence
  - ◆ 使用的信道带宽尽量少——efficient
  - ◆ 帧的长度与所传输的数据的内容无关
  - ◆ 不易出错，出错后易重新同步——robustness

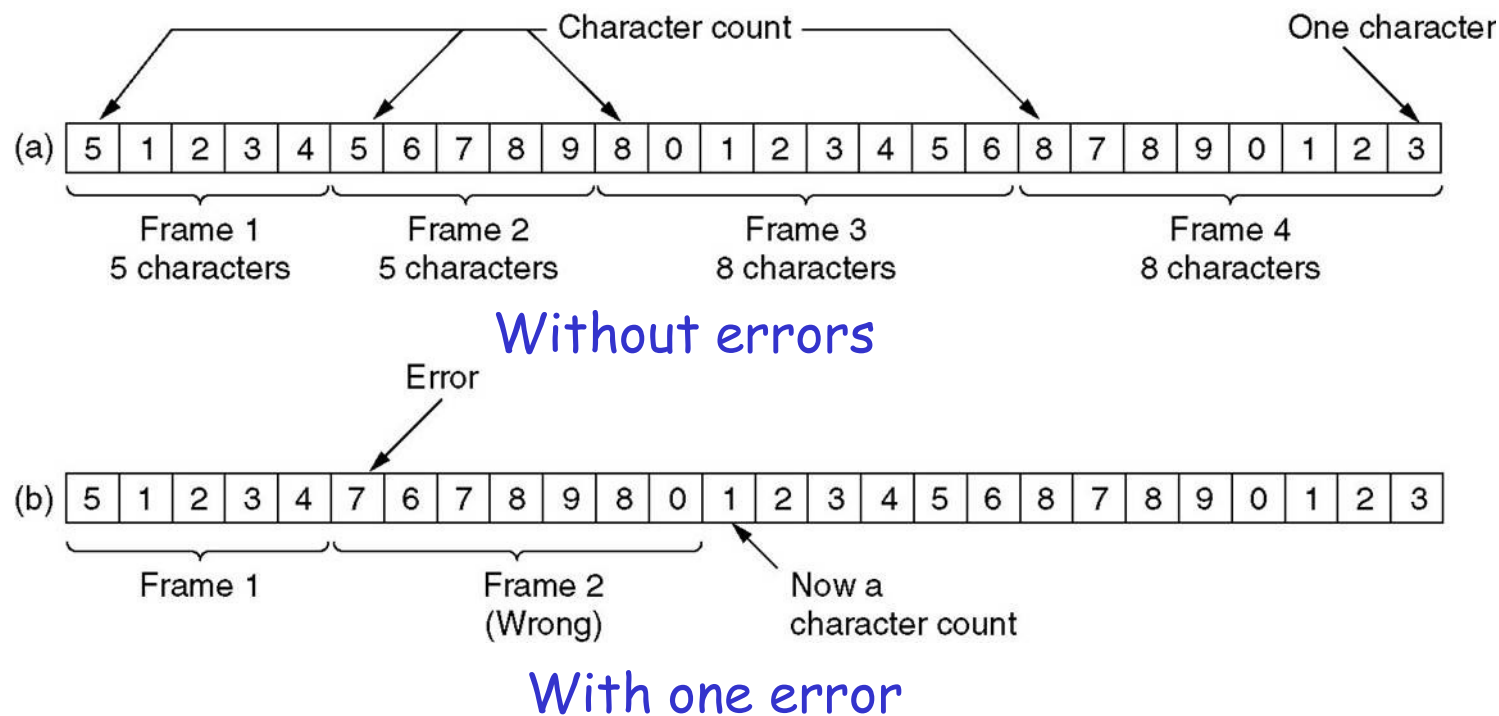
# Framing(成帧)

## ■ Methods

- ◆ Character count (字符计数法)
- ◆ Flag byte with byte stuffing (字符填充法)
- ◆ Starting and ending flags, with bit stuffing(比特填充法)
- ◆ Physical layer coding violations (物理层编码违例法)

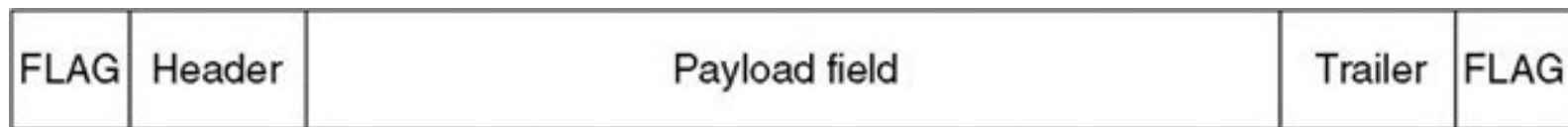
# Character Count(字符计数法)

- Using a field in header to specify number of characters in the frame
- The count may be garbled by a transmission error
- Used in DDCMP(DEC)

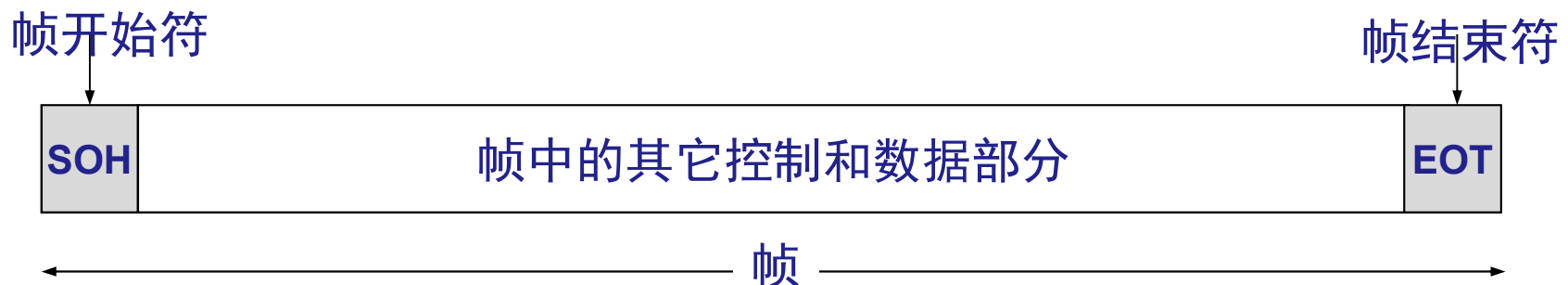


## Byte (Character) Stuffing(字节/字符填充)

- Using **flag bytes** to mark the starting and ending of a frame
- Closely tied to the use of 8-bit characters
- Used in BISYNC(IBM)



For example,



# Byte stuffing: transparent transmission (透明传输)

- What if payload includes same byte with the flag?
  - ◆ Using special stuffing byte: ESC (eg. 0x1B)
  - ◆ Inserted before "flag" in payload

Original payload

Original characters



Payload after stuffing

After stuffing

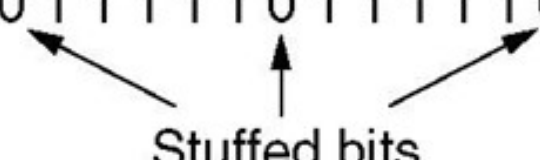


# Bit Stuffing (比特填充)

- Allowing a frame to contain arbitrary number of bits
- Starting and ending flag: 0111,1110
- Stuffing: a '0' is stuffed after five consecutive 1s
- Used in HDLC

original data (a) 0 1 1 0 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 0 1 0

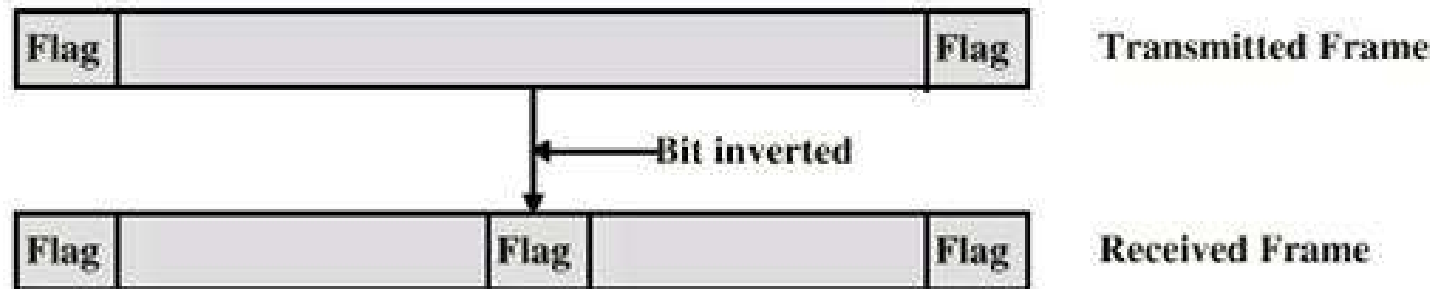
After stuffing (b) 0 1 1 0 1 1 1 1 1 0 1 1 1 1 1 0 1 1 1 1 1 0 1 0 0 1 0



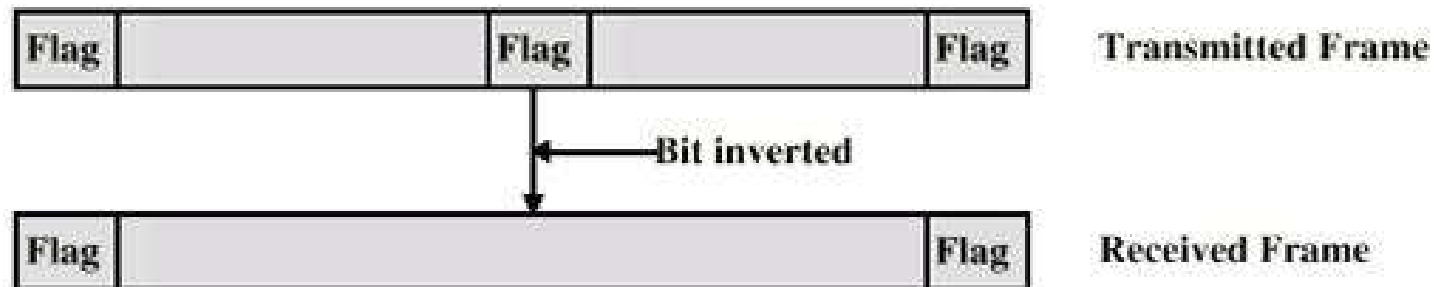
Stuffed bits

After destuffing (c) 0 1 1 0 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 0 1 0

# Possible Errors of Bit Stuffing



(b) An inverted bit splits a frame in two



(c) An inverted bit merges two frames

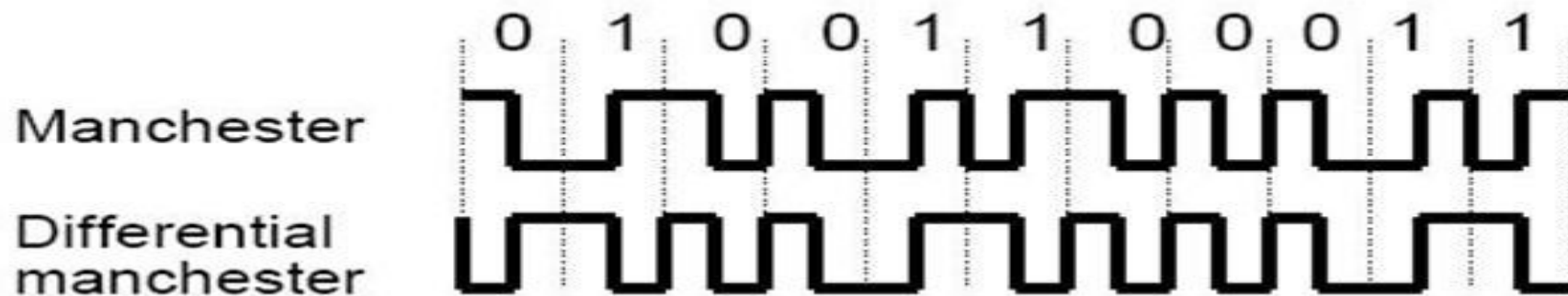


# 比特填充练习

- 从信道上收到比特序列： 1101 0111 1110  
0111 1110 1101 1011 1110 0010 1100  
0101 1111 0101 1001 1111 1001，该比特序列中包含一个完整的帧，用十六进制写出该帧的内容（不包含帧的首尾标志）。

# Physical Layer Coding Violations (物理层编码违例法)

- Applicable to encoding schemes with redundancy
  - ◆ For example, in LAN, **Manchester** encoding is used
  - ◆ Each valid bit has a transition in the middle of signal
  - ◆ Then code without transition, i.e. **high-high** and **low-low** are used for frame delimitation



# Review comes first

- 数据链路层的功能：相邻节点间可靠地传输数据帧
- 实现可靠传输的功能
  - ◆ 差错控制：检测差错、纠正差错
  - ◆ 流量控制：防止发送方速度过快而淹没接收方（接收方来不及接收而丢失数据）
  - ◆ 介质访问控制(MAC)：对于共享信道，如何防止因两个站点同时发送而导致冲突？ ---在下一章学习
- 向网络层提供的服务：取决于不同的物理网络（数据链路层+物理层）
  - ◆ Ethernet：无连接服务
  - ◆ Wifi：带确认的无连接服务
  - ◆ ATM：面向连接服务

# 数据链路层的第一个功能：成帧framing

## ■ 如何实现发送方与接收方的同步？

- ◆ 让接收方能够从来自物理层的一串二进制位流中提取出完整的一帧

## ■ 成帧方法

- ◆ 字符计数法：前面加一个帧长度字段，不实用
- ◆ 字节(字符)填充法：前面加一个帧首标志字符，后面加一个帧尾标志字符
  - 透明传输：用添加转义字符来实现
- ◆ 比特(位)填充法：用0111 1110标记帧头和帧尾
  - 透明传输：零比特填充，在帧中连续五个1之后添加一个0
- ◆ 物理层编码违例法：曼彻斯特编码，使用非法编码标记帧头和帧尾；支持透明传输，无需填充

# Outline

- Position, Functions & Services of Data Link Layer
- Design Issues
  - ◆ Framing
  - ◆ Error Control (Textbook 3.2)
    - Error Correcting Code(ECC)
    - Error detecting code(EDC)
  - ◆ Flow Control
- Example Protocols
  - ◆ HDLC
  - ◆ PPP

# Error Control(差错控制)

## ■ Type of Errors

- ◆ Lost frames: a frame failed to arrive to the other side
  - Owing to noise burst or dropped from a queue
- ◆ **Damaged frames**: some bits are in error →

## ■ Error detection

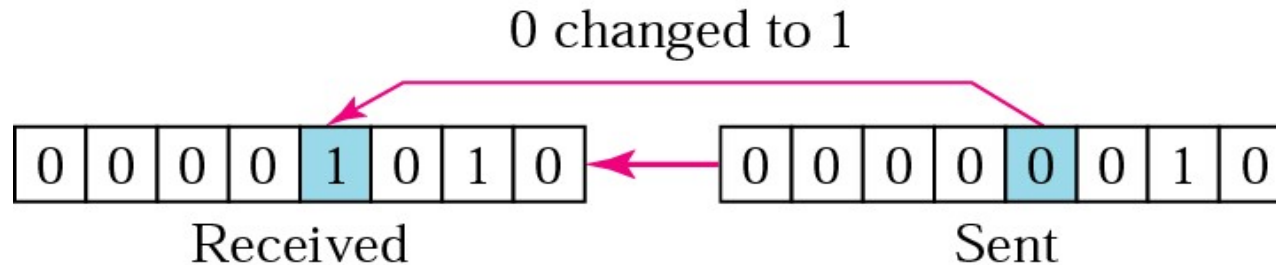
- ◆ Parity check: able to detect single bit errors →
- ◆ **Cyclic Redundancy Check (CRC)**: detecting some burst errors
- ◆ Checksum, to be studied in Network layer

## ■ Error correction

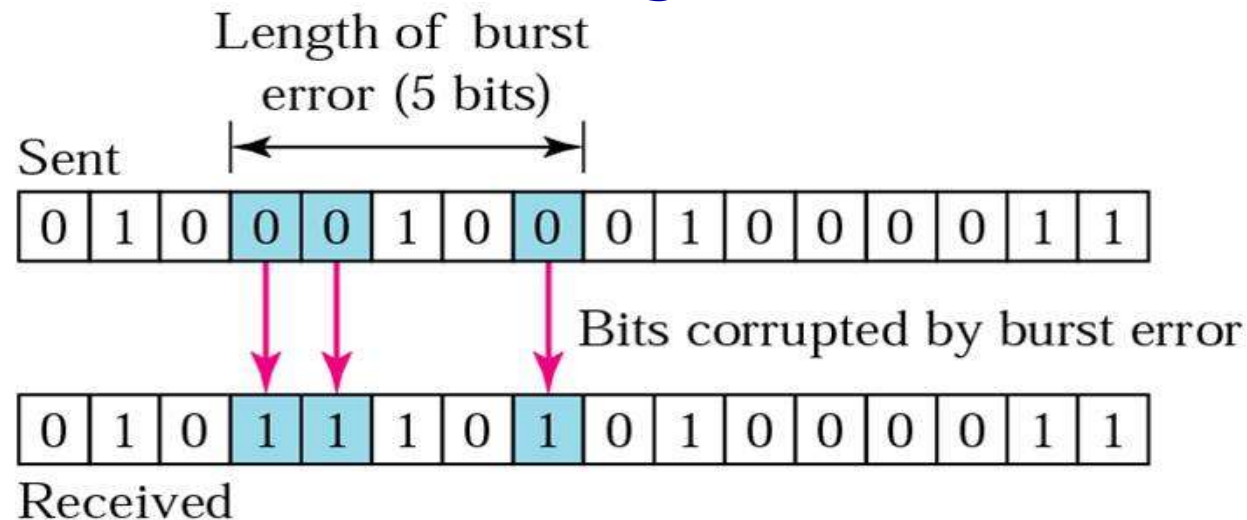
- ◆ Forward Error Correction (FEC, 前向纠错) →
- ◆ Retransmission: Automatic Repeat reQuest (**ARQ**)

# Single-bit Error & Burst Error

- Single-bit error, only one bit has changed



- A burst error(突发差错) means that 2 or more bits have changed.



# Odd & Even Parity (奇偶校验)

## ■ Odd & Even parity

|   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|
| 1 | 0 | 1 | 1 | 1 | 0 | 1 | 0 | ? |
|---|---|---|---|---|---|---|---|---|

## ■ Odd Parity

|   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|
| 1 | 0 | 1 | 1 | 1 | 0 | 1 | 0 | 0 |
|---|---|---|---|---|---|---|---|---|

1-bit error

|   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 1 | 1 | 1 | 0 | 1 | 0 | 0 |
|---|---|---|---|---|---|---|---|---|

3-bit error

|   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 1 | 0 | 0 | 1 | 0 | 0 |
|---|---|---|---|---|---|---|---|---|

2-bit error

|   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 1 | 1 | 1 | 1 | 1 | 0 | 0 |
|---|---|---|---|---|---|---|---|---|

## ■ Even Parity

|   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|
| 1 | 0 | 1 | 1 | 1 | 0 | 1 | 0 | 1 |
|---|---|---|---|---|---|---|---|---|

## 单个奇偶校验位性能

- 可检出奇数个位翻转错误
- 其它情况漏检率50%

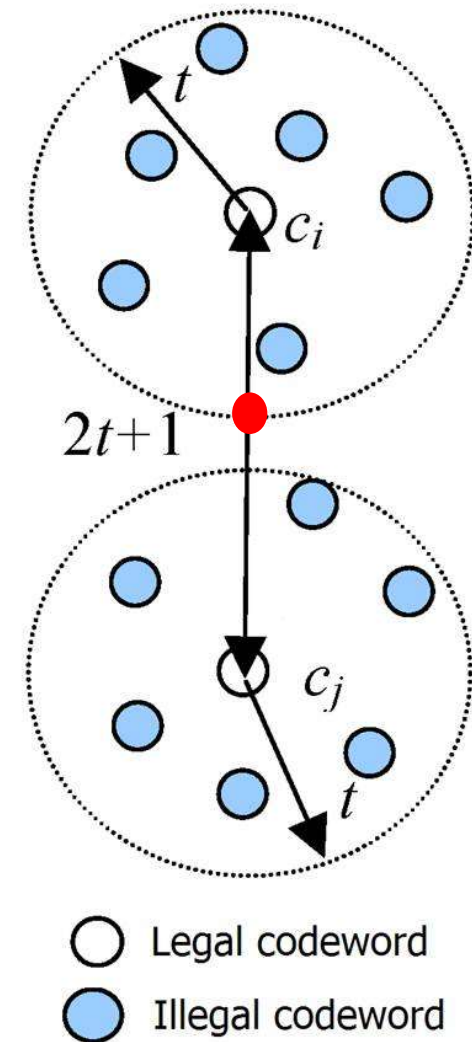


[Forouzan]



# Error Correcting Code (纠错码) : Hamming Distance(汉明/海明距离)

- Hamming distance: The number of bit positions in which two codewords differ
  - ◆ eg. 1000 1001 and 1011 0001, Hamming distance = 3
- To detect  **$t$ -bit** errors, need a distance  **$t+1$**  code
- To correct  **$t$ -bit** errors, need a distance  **$2t+1$**  code



# Example of an ECC

- ◆ Consider a code with only four valid codewords:  
00000000000, 00000 11111  
11111 00000, 11111 11111
- ◆ This code has a distance 5, which means that it can correct 2-bit errors.
  - If the codeword 00000 00111 arrives, the receiver knows that the original must have been 00000 11111
  - If a triple error changes 00000 00000 into 00000 00111, the error will not be corrected properly

# The Number of Check Bits

- Find out the number of check bits needed to correct **single bit error**
- Coding
  - ◆  $m$ : message bits
  - ◆  $r$ : check bits
  - ◆  $n$ : codeword length ( $n=m+r$ )
- $2^m$  legal messages
- Each legal message has  $n$  illegal codewords at a distance 1 from it. Altogether there are  $(n+1)2^m$  codewords

$$(n+1)2^m \leq 2^n$$

$$\longrightarrow (m+r+1) \leq 2^r.$$

Example:  $m=7, r \geq 4$

# Hamming Code: correction of single bit error

The bits of the codeword are numbered, start with 1 at left end, bit 2 to its immediate right, and so on.

## ■ Check bits

- ◆ The bits that are powers of 2 (1, 2, 4, 8, 16...) are check bits
- ◆ Each check bit forces the parity of some collection of bits, including itself, to be even (or odd).

## ■ Data bits

- ◆ The rest (3,5,6,...) are filled up with the  $m$  data bits

# Example of Hamming Code(1)

- Message: 'A' (7-bit ASCII 100,0001)  
Data Bits: 3,5,6,7,9,10,11
- 4 Check bits, Bit 1,2,4,8 (even parity, 偶校验)

Bit Number: 1 2 3 4 5 6 7 8 9 10 11

Codeword: 0 0 1 0 0 0 0 1 0 0 1

发送前编码:

$$3=1+2, \quad 5=1+4, \quad 6=2+4, \quad 7=1+2+4$$

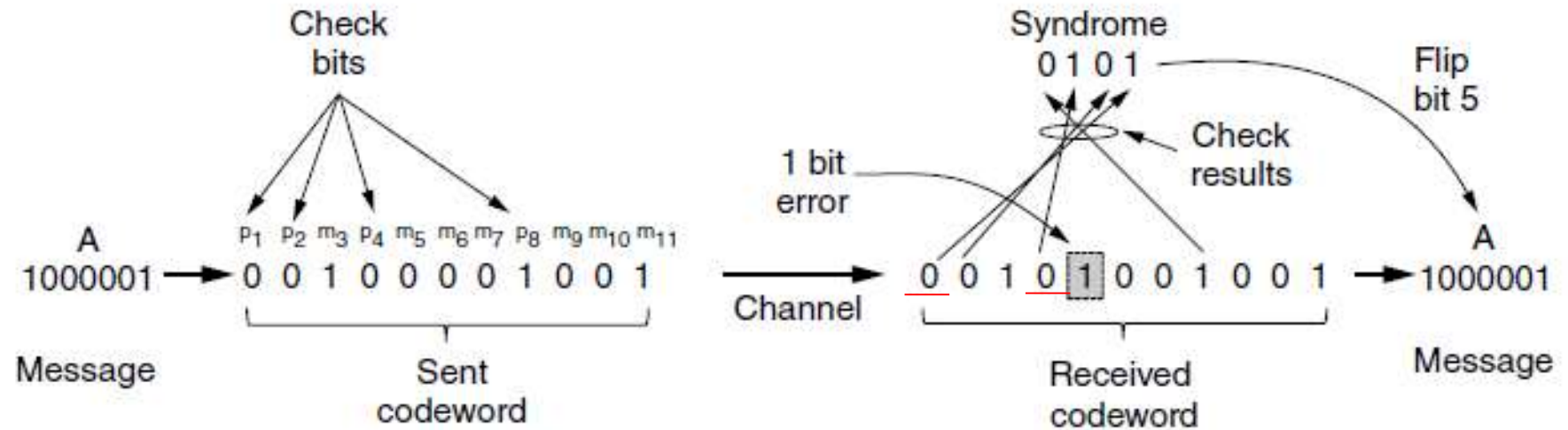
$$9=1+8, \quad 10=2+8, \quad 11=1+2+8$$

接收者纠错:

|                          |                                     |                           |       |
|--------------------------|-------------------------------------|---------------------------|-------|
| $b1 \oplus b3 \oplus b5$ | $\oplus b7 \oplus b9$               | $\oplus b11=0$            | (k=1) |
| $b2 \oplus b3$           | $\oplus b6 \oplus b7$               | $\oplus b10 \oplus b11=0$ | (k=2) |
| $b4$                     | $\oplus b5 \oplus b6 \oplus b7$     | $=0$                      | (k=4) |
| $b8$                     | $\oplus b9 \oplus b10 \oplus b11=0$ |                           | (k=8) |

# Example of Hamming Code(2)

采用偶校验来计算校验位



- Hamming distance of Hamming Code: 3
- Able to detect 2-bit errors and correct 1-bit errors

## 偶校验

# 海明编码解码—简便法

例：数据 = 1011, 

|    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|
| b1 | b2 | b3 | b4 | b5 | b6 | b7 |
| P1 | P2 | 1  | P3 | 0  | 1  | 1  |

编码简便法：将码字中为1的各位码字位号表示为二进制码，再按模2求和，所得结果就是校验码。

$$\begin{array}{r} b3 = 011 \\ b6 = 110 \\ \oplus \quad b7 = 111 \\ \hline P3P2P1 = 010 \end{array}$$

发送的码字：→ 

|    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|
| b1 | b2 | b3 | b4 | b5 | b6 | b7 |
| 0  | 1  | 1  | 0  | 0  | 1  | 1  |

收到的码字：→ 

|    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|
| b1 | b2 | b3 | b4 | b5 | b6 | b7 |
| 0  | 1  | 1  | 0  | 0  | 1  | 0  |

差错位

解码简便法：将码字中为1的各位码字位号表示为二进制码，再按模2求和，若和为0，则无差错。若和不为0，则指明差错的位号。

$$\begin{array}{r} b2 = 010 \\ b3 = 011 \\ \oplus \quad b6 = 110 \\ \hline S4S2S1 = 111 \end{array}$$

# Hamming code examples

| Char. | ASCII   | Check bits  |
|-------|---------|-------------|
| H     | 1001000 | 00110010000 |
| a     | 1100001 | 10111001001 |
| m     | 1101101 | 11101010101 |
| m     | 1101101 | 11101010101 |
| i     | 1101001 | 01101011001 |
| n     | 1101110 | 01101010110 |
| g     | 1100111 | 01111001111 |
|       | 0100000 | 10011000000 |
| c     | 1100011 | 11111000011 |
| o     | 1101111 | 10101011111 |
| d     | 1100100 | 11111001100 |
| e     | 1100101 | 00111000101 |

Order of bit transmission



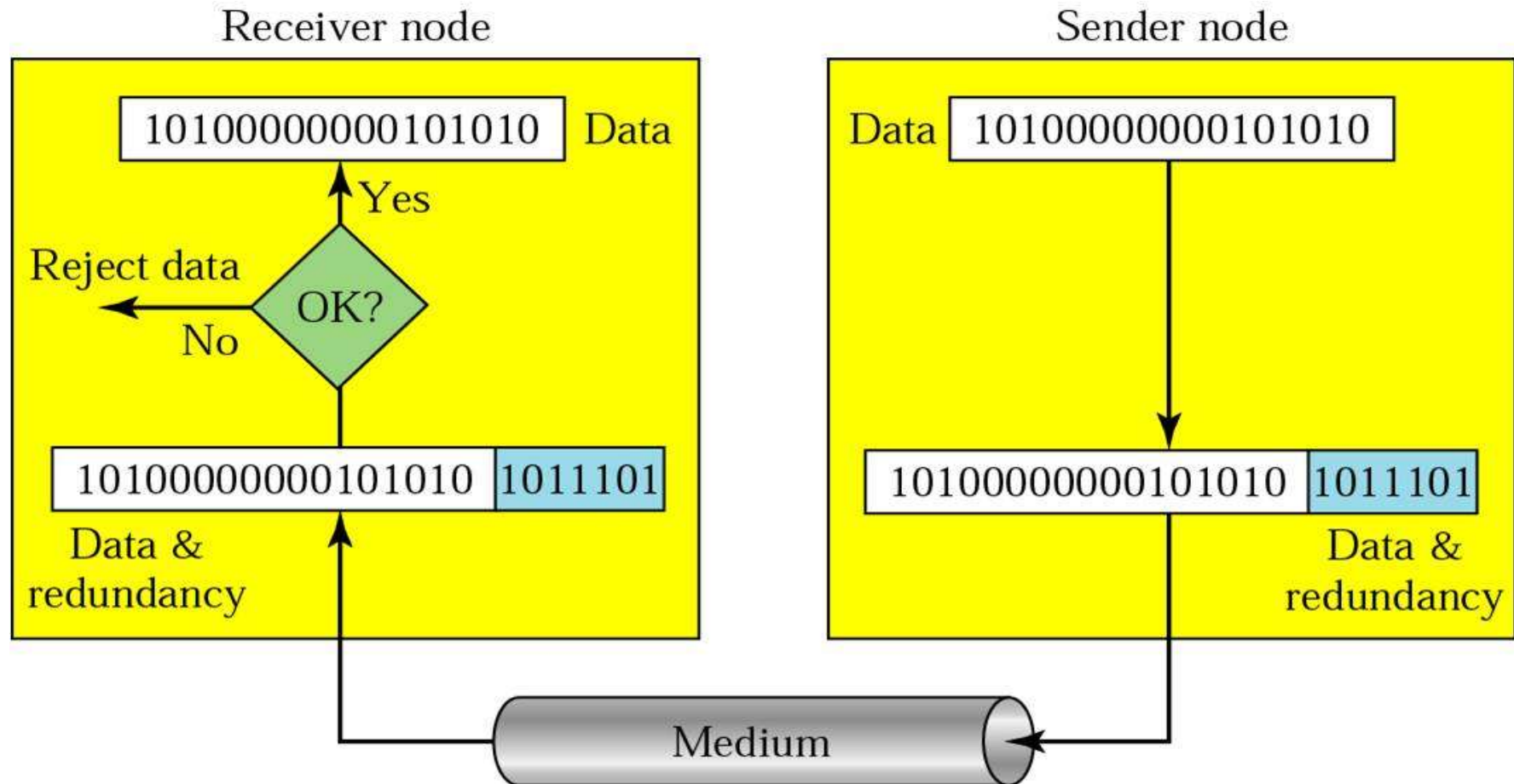
# Outline

- Position, Functions & Services of Data Link Layer
- Design Issues
  - ◆ Framing
  - ◆ Error Control
    - Error Correcting Code(ECC)
    - Error Detecting Code(EDC)
  - ◆ Flow Control
- Example Protocols
  - ◆ HDLC
  - ◆ PPP

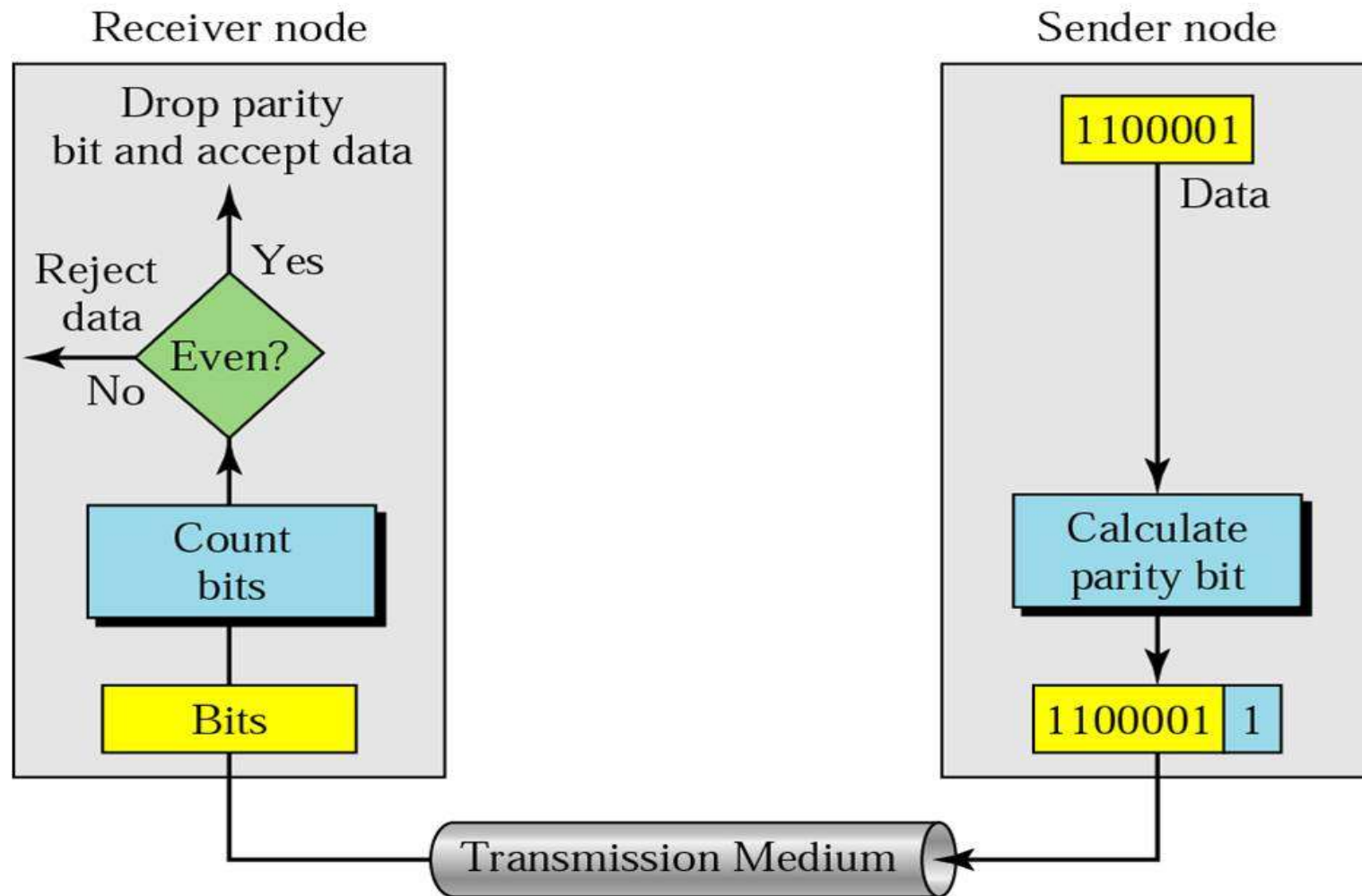
# Error Correcting & Detecting

- On channels that are **reliable**, such as fiber, it is cheaper to use an error detecting code and just **retransmit** the occasional block found to be faulty
- On channels such as **wireless** links that make many errors, it is better to **add** enough **redundancy** to each block for the receiver to be able to figure out what the original block was, i.e. using error correcting code
- Example:
  - ◆ Bit Error Rate(BER)= $10^{-6}$ , 1block=1000bits, Data=1Mbits
  - ◆ The overhead for 1000 blocks
    - Error detection(parity check) + retransmission: 2001 bits
    - Hamming code : 10,000 bits, with 10 check bits for each block

# Error Detecting



# Even-parity Checking (偶校验)

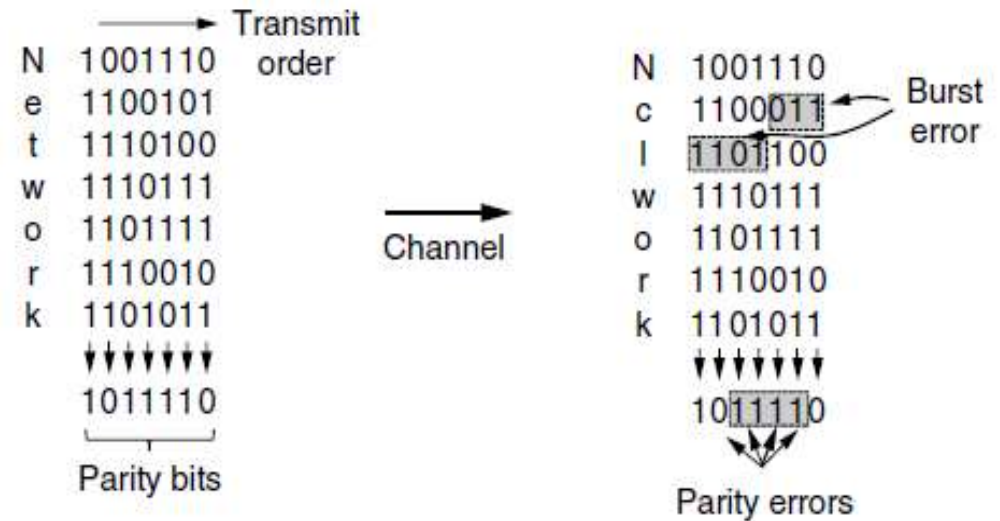


[Forouzan]

# Parity Check for Blocks (Interleaving, 交错校验)

## ■ Coding ( $n$ check bits)

- ◆ Each block is a  $k \times n$  matrix
- ◆ The matrix is transmitted one row at a time
- ◆ A parity bit is computed separately for each column



## ■ Performance

- ◆ Can detect a single burst of length  $n$
- ◆ A burst of length  $n+1$  will pass undetected if the first bit is inverted, the last bit is inverted, and all the other bits are correct
- ◆ If block is garbled by a long burst, the probability of being accepted is  $2^{-n}$

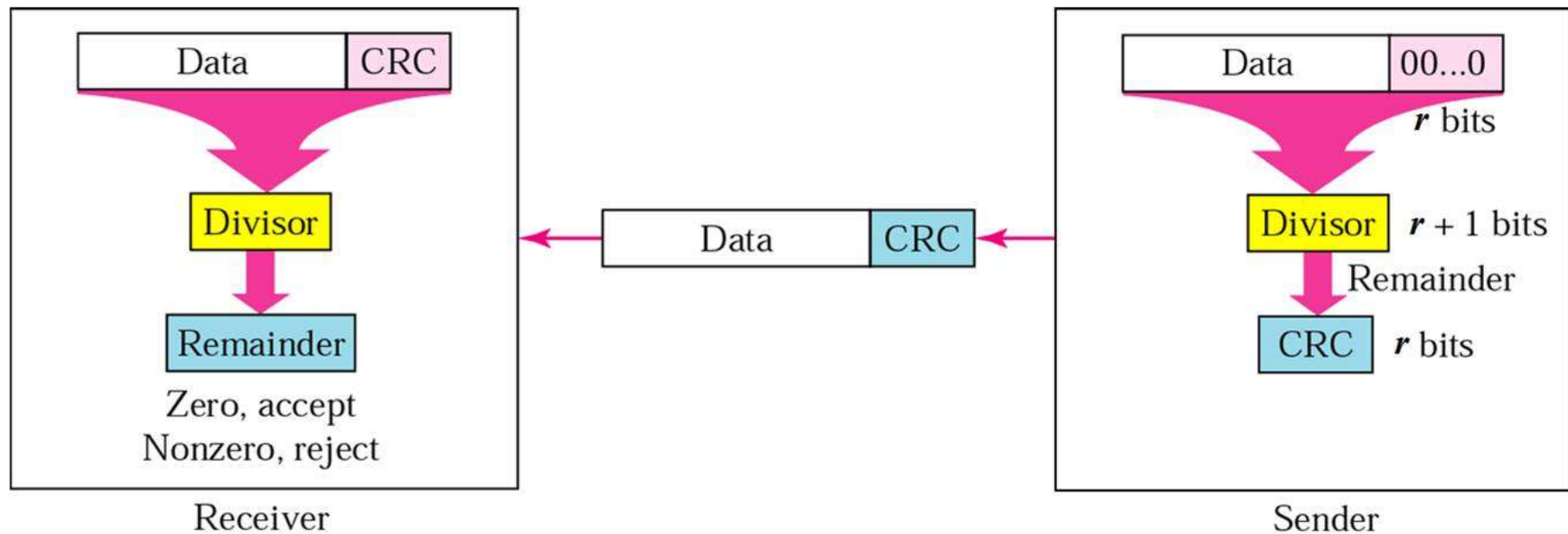
# Polynomial Code (多项式编码)

Also known as **CRC** (Cyclic Redundancy Check, 循环冗余校验), **FCS** (Frame Check Sequence 帧校验序列)

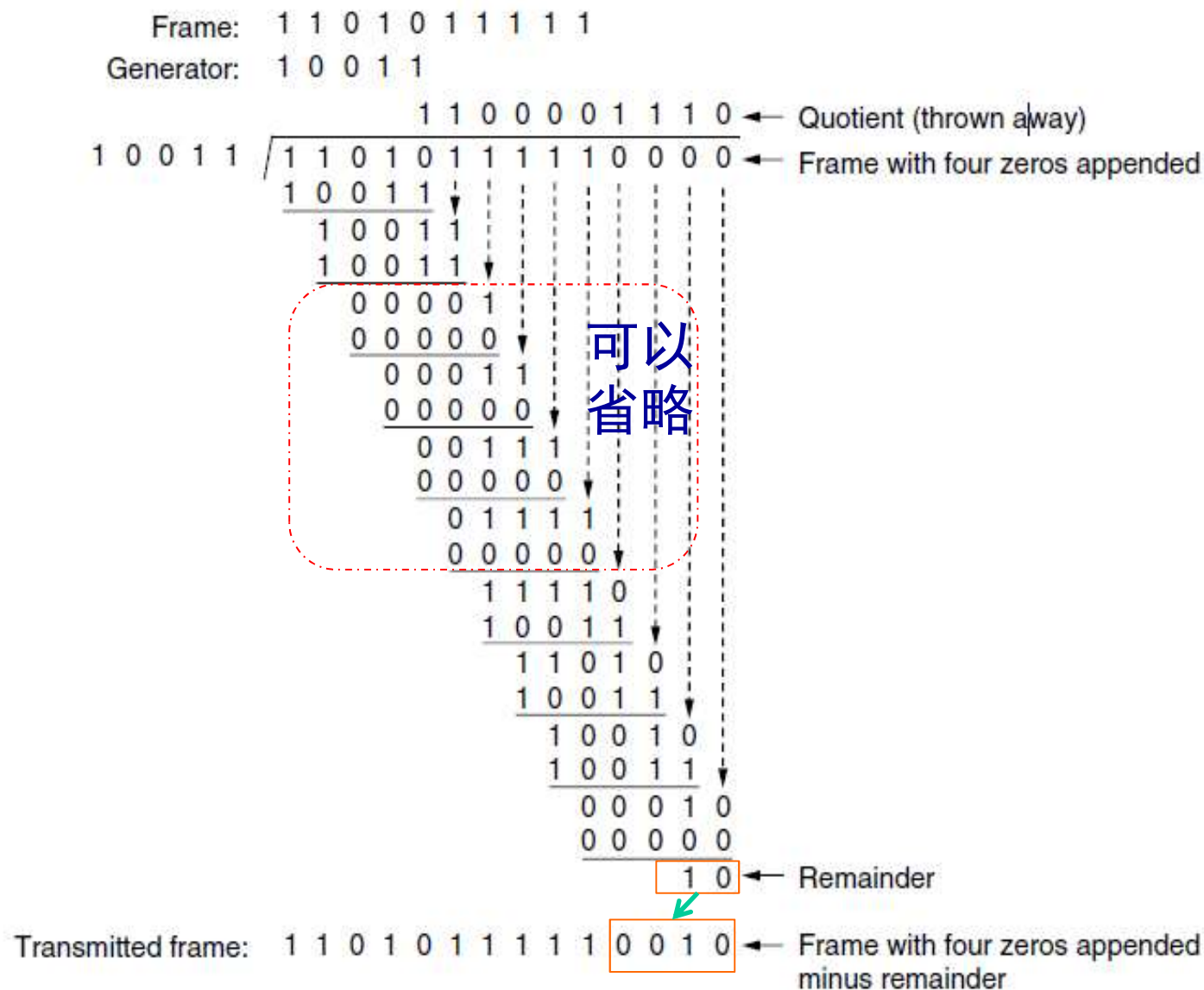
- $M(x)$ : Treating bit strings as representations of polynomials with coefficients of 0 and 1
  - ◆ A  $k$ -bit frame is regarded as the coefficient list for a polynomial with  $k$  terms, ranging from  $x^{k-1}$  to  $x^0$ :  
 $110001 \Rightarrow x^5 + x^4 + x^0$
- Polynomial arithmetic is done modulo 2, addition and subtraction are identical to XOR
- $G(x)$ : Generator polynomial(生成多项式)

# Sender: Computing the Checksum (校验位)

- The algorithm for computing the checksum:
  - ◆ Append  $r$  zero bits to the low-order end of the frame
  - ◆ Divide the bit string corresponding to  $G(x)$
  - ◆  $T(x)$ : Subtract the remainder from the bit string



# Example of CRC Calculation





# Receiver: Verify the Checksum

- If a transmission error occurs, so that instead of the bit string for  $T(x)$  arriving,  $T(x) + E(x)$  arrives.
- The receiver computes  $[T(x) + E(x)]/G(x)$ .  
 $T(x)/G(x)$  is 0, so the result is  $E(x)/G(x)$
- ◆ Those errors that happen to correspond to polynomials containing  $G(x)$  as a factor will slip by; all other errors will be caught.

# The Power of CRC

- $r$  check bits will detect all burst errors of length  $\leq r$
- Two isolated single-bit errors are detected
- Catch all errors consisting of an odd number of inverted bits
- Other burst errors (length  $> r$ )
- ◆ The probability of a bad frame been accepted is  $2^{-r}$

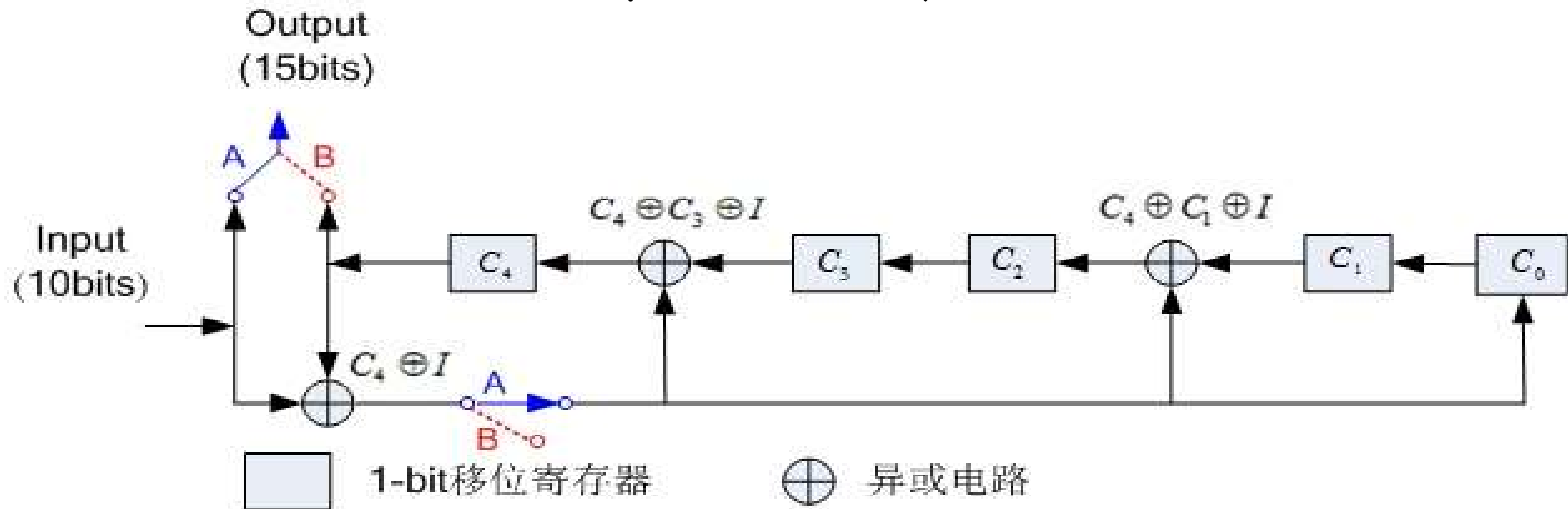
# Generator Polynomial

- Certain polynomials have become international standards
- CRC-12(Telecomm system)  
 $x^{12}+x^{11}+x^3+x^2+x+1$
- CRC-16(Bisync, Modbus, USB, ANSI X3.28, SIA DC-07, many others)  
 $x^{16}+x^{15}+x^2+1$
- CRC-CCITT (HDLC, ITU X.25, V.34/V.41/V.42, PPP-FCS,ZigBee)  
 $x^{16}+x^{12}+x^5+1$
- CRC-32 (ZIP, RAR, IEEE 802 LAN/FDDI, IEEE 1394, PPP-FCS)  
 $x^{32}+x^{26}+x^{23}+x^{22}+x^{16}+x^{12}+x^{11}+x^{10}+x^8+x^7+x^5+x^4+x^2+x^1+1$
- CRC-64

# The Calculation of CRC (Hardware)

## Hardware

- ◆ A simple **shift register circuit**(移位寄存电路) can be constructed to compute and verify the checksums



- ◆  $G(x) = x^5 + x^4 + x^2 + 1$
- ◆ 初始时移位寄存器  $C_0 \sim C_4$  清0
- ◆ 10个信息位发送之后，断开A接通B,发送5位校验和，并将  $C_0 \sim C_4$  清零，为下次发送做好准备

# The Calculation of CRC (Hardware)

|         | $C_4$ | $C_3$ | $C_2$ | $C_1$ | $C_0$ | $C_4 \oplus C_3 \oplus I$ | $C_4 \oplus C_1 \oplus I$ | $C_4 \oplus I$ | $I(\text{Input})$ |
|---------|-------|-------|-------|-------|-------|---------------------------|---------------------------|----------------|-------------------|
| Initial | 0     | 0     | 0     | 0     | 0     | 1                         | 1                         | 1              | 1                 |
| Step 1  | 1     | 0     | 1     | 0     | 1     | 1                         | 1                         | 1              | 0                 |
| Step 2  | 1     | 1     | 1     | 1     | 1     | 1                         | 1                         | 0              | 1                 |
| Step 3  | 1     | 1     | 1     | 1     | 0     | 0                         | 0                         | 1              | 0                 |
| Step 4  | 0     | 1     | 0     | 0     | 1     | 1                         | 0                         | 0              | 0                 |
| Step 5  | 1     | 0     | 0     | 1     | 0     | 1                         | 0                         | 1              | 0                 |
| Step 6  | 1     | 0     | 0     | 0     | 1     | 0                         | 0                         | 0              | 1                 |
| Step 7  | 0     | 0     | 0     | 1     | 0     | 1                         | 0                         | 1              | 1                 |
| Step 8  | 1     | 0     | 0     | 0     | 1     | 1                         | 1                         | 1              | 0                 |
| Step 9  | 1     | 0     | 1     | 1     | 1     | 0                         | 1                         | 0              | 1                 |
| Step10  | 0     | 1     | 1     | 1     | 0     |                           |                           |                |                   |

# 小结

## ■ 差错控制方法

### ◆ 纠错码：汉明码（海明码），纠正1比特翻转错误

- 序号是2的幂次的位置放校验位，其它位置放数据位
- 任何一个数据位的序号都可以写成2的幂次的和，即由对应的校验位进行奇偶校验；该位置的数据位出错时，必然会影响到对应校验位的校验结果

### ◆ 检错码+重传纠错

- 奇偶校验：检测单比特错误/奇数个比特错误，ASCII码使用
- 交错校验：块数据使用， $n*k$ 矩阵，可检测出长度 $\leq n$ 的错误
- CRC（循环冗余校验）

(1)发送方：待校验数据补充 $r$ 个0之后，对生成多项式进行模2除法，余数（ $r$ 位）即为校验信息

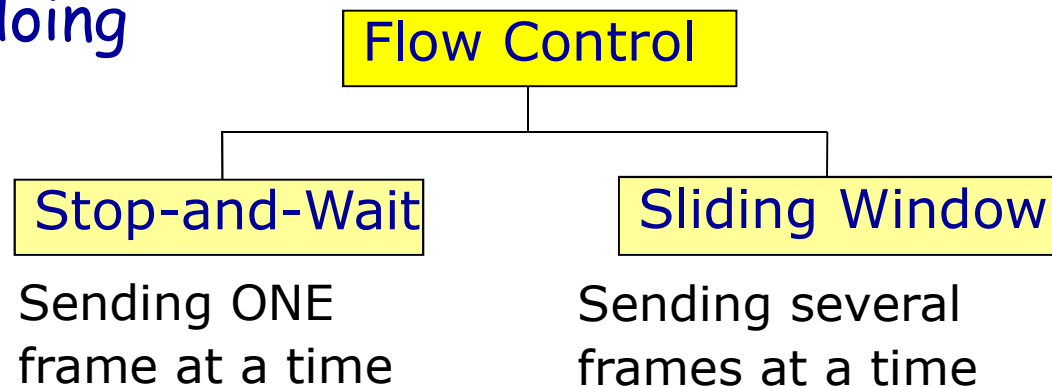
(2)接收方：待校验数据和校验信息一起，对生成多项式进行模2除法，余数不为0即认为有错

# Outline

- Position, Functions & Services of Data Link Layer
- Design Issues
  - ◆ Framing
  - ◆ Error Control
  - ◆ Flow Control (textbook 3.3)
    - Basic flow control protocols
    - Performance of sliding window protocols
- Example Protocols
  - ◆ HDLC
  - ◆ PPP

# Flow control(流量控制)

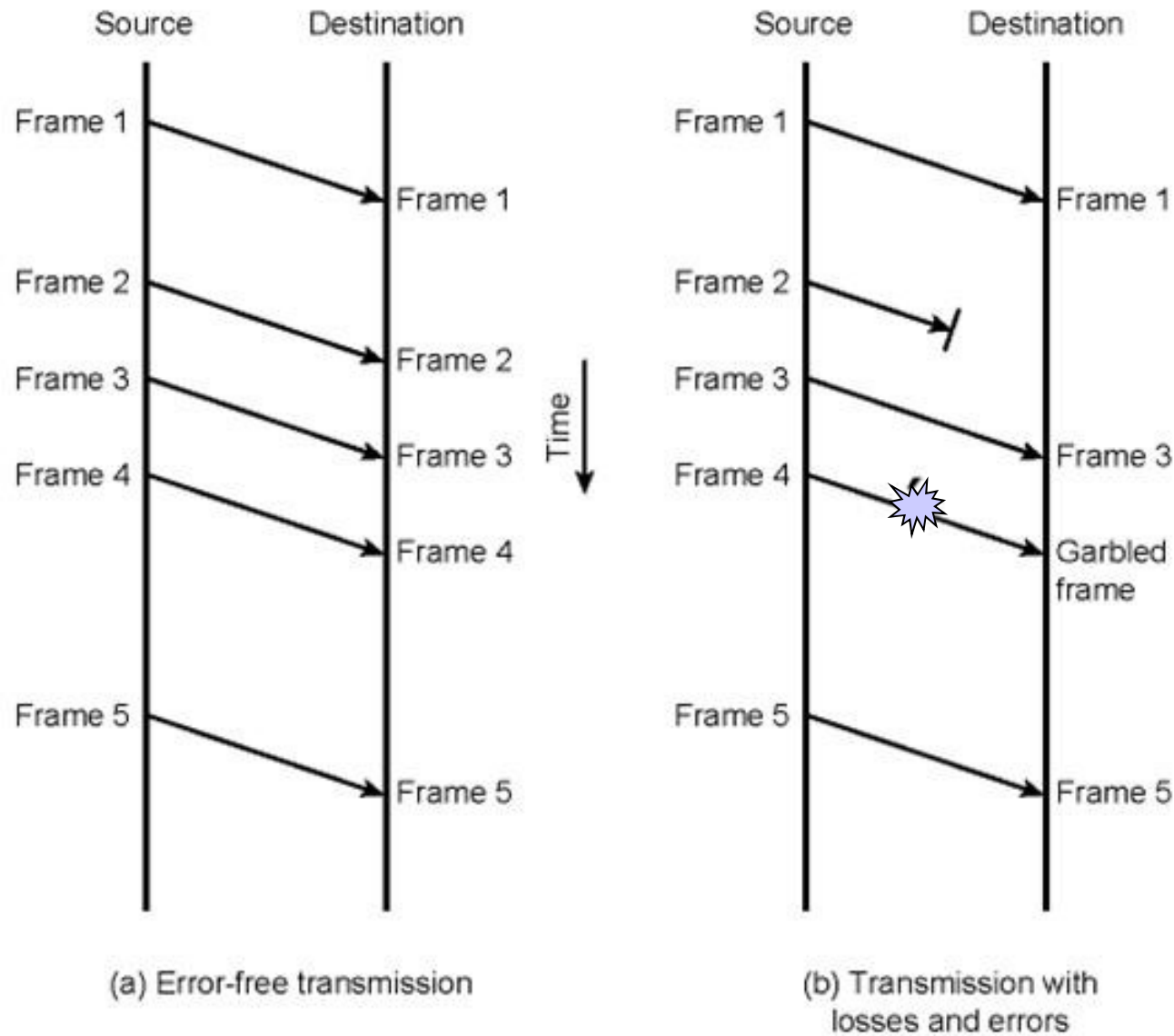
- Ensuring the sending entity does not overwhelm the receiving entity, i.e. preventing buffer overflow
- **Feedback-based flow control**: the receiver sends back information to the sender giving it permission to send more data or at least telling the sender how the receiver is doing



- **Rate-based flow control**: the protocol has a built-in mechanism that limits the transmitting rate of senders, without using feedback from the receiver, to be studied in network layer



# Why we need Flow Control?



# Basic Flow Control Protocols

## ■ Elementary Protocols

- ◆ A Utopian Simplex Protocol
- ◆ A Simplex Stop-and-Wait Protocol for an Error-Free Channel
- ◆ A Simplex Protocol for a Noisy Channel

## ■ Sliding Window Protocols

- ◆ A One-Bit Sliding Window Protocol
- ◆ A Protocol Using Go Back N
- ◆ A Protocol Using Selective Repeat

# Assumptions and lib procedures(1)

## ■ Some assumptions

- ◆ Physical layer, data link layer, and network layer are independent processes
- ◆ Machine A wants to send a long stream of data to machine B, using a reliable, connection-oriented service
- ◆ Machines do not crash

# Assumptions and lib procedures(2)

## ■ Library procedures

- ◆ wait\_for\_event
  - frame\_arrival
  - timeout
  - chsum\_err...
- ◆ from\_network\_layer / to\_network\_layer
- ◆ to\_physical\_layer / from\_physical\_layer
- ◆ timer operations
  - start\_timer
  - stop\_timer
  - timeout...
- ◆ enable\_network\_layer / disable\_network\_layer
- ◆ sequence number operation
  - inc (+1),

# Protocol Definitions

```
#define MAX_PKT 1024                                /* determines packet size in bytes */

typedef enum {false, true} boolean;                  /* boolean type */
typedef unsigned int seq_nr;                          /* sequence or ack numbers */
typedef struct {unsigned char data[MAX_PKT];} packet; /* packet definition */
typedef enum {data, ack, nak} frame_kind;             /* frame_kind definition */

typedef struct {                                     /* frames are transported in this layer */
    frame_kind kind;                                /* what kind of a frame is it? */
    seq_nr seq;                                     /* sequence number */
    seq_nr ack;                                     /* acknowledgement number */
    packet info;                                    /* the network layer packet */
} frame;
```

Some definitions needed in the protocols to follow.  
These are located in the file protocol.h.

```

/* Wait for an event to happen; return its type in event. */
void wait_for_event(event_type *event);

/* Fetch a packet from the network layer for transmission on the channel. */
void from_network_layer(packet *p);

/* Deliver information from an inbound frame to the network layer. */
void to_network_layer(packet *p);

/* Go get an inbound frame from the physical layer and copy it to r. */
void from_physical_layer(frame *r);

/* Pass the frame to the physical layer for transmission. */
void to_physical_layer(frame *s);

/* Start the clock running and enable the timeout event. */
void start_timer(seq_nr k);

/* Stop the clock and disable the timeout event. */
void stop_timer(seq_nr k);

/* Start an auxiliary timer and enable the ack_timeout event. */
void start_ack_timer(void);

/* Stop the auxiliary timer and disable the ack_timeout event. */
void stop_ack_timer(void);

/* Allow the network layer to cause a network_layer_ready event. */
void enable_network_layer(void);

/* Forbid the network layer from causing a network_layer_ready event. */
void disable_network_layer(void);

/* Macro inc is expanded in-line: Increment k circularly. */
#define inc(k) if (k < MAX_SEQ) k = k + 1; else k = 0

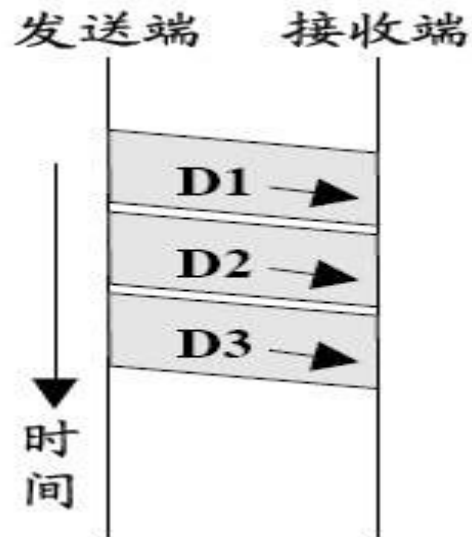
```

# Protocol 1: Utopian Simplex Protocol

## ■ Assumptions

- ◆ Channel is simplex
- ◆ Channel is error-free
- ◆ Receiver has unlimited size of buffers

## ■ No need of error control and flow control



# Protocol 1: A Utopian Simplex Protocol (乌托邦)

/\* Protocol 1 (utopia) provides for data transmission in one direction only, from sender to receiver. The communication channel is assumed to be error free, and the receiver is assumed to be able to process all the input infinitely quickly. Consequently, the sender just sits in a loop pumping data out onto the line as fast as it can. \*/

```
typedef enum {frame arrival} event type;
#include "protocol.h"
```

```
void sender1(void)
{
    frame s;                                /* buffer for an outbound frame */
    packet buffer;                          /* buffer for an outbound packet */

    while (true) {
        from_network_layer(&buffer); /* go get something to send */
        s.info = buffer;             /* copy it into s for transmission */
        to_physical_layer(&s);       /* send it on its way */
    }                                /* Tomorrow, and tomorrow, and tomorrow,
                                   Creeps in this petty pace from day to day
                                   To the last syllable of recorded time
                                   - Macbeth, V, v */
}

void receiver1(void)
{
    frame r;
    event_type event;                    /* filled in by wait, but not used here */

    while (true) {
        wait_for_event(&event);         /* only possibility is frame_arrival */
        from_physical_layer(&r);        /* go get the inbound frame */
        to_network_layer(&r.info);     /* pass the data to the network layer */
    }
}
```



# Protocol 2: Simplex Stop-and-Wait Protocol for an Error-Free Channel

## ■ Assumptions

- ◆ Channel is simplex
- ◆ Channel is error-free
- ◆ Receiver has **limited** size of buffers

## ■ No need of error control

## ■ Flow control is necessary

- ◆ Stop-and-wait (停止等待/停等)
  - Sender sends a frame, and wait
  - Receiver sends a dummy frame(acknowledgement, ACK, 确认/应答)
  - Receiving the ACK, sender sends next frame

# Protocol 2: Simplex Stop-and- Wait Protocol

/\* Protocol 2 (stop-and-wait) also provides for a one-directional flow of data from sender to receiver. The communication channel is once again assumed to be error free, as in protocol 1. However, this time, the receiver has only a finite buffer capacity and a finite processing speed, so the protocol must explicitly prevent the sender from flooding the receiver with data faster than it can be handled. \*/

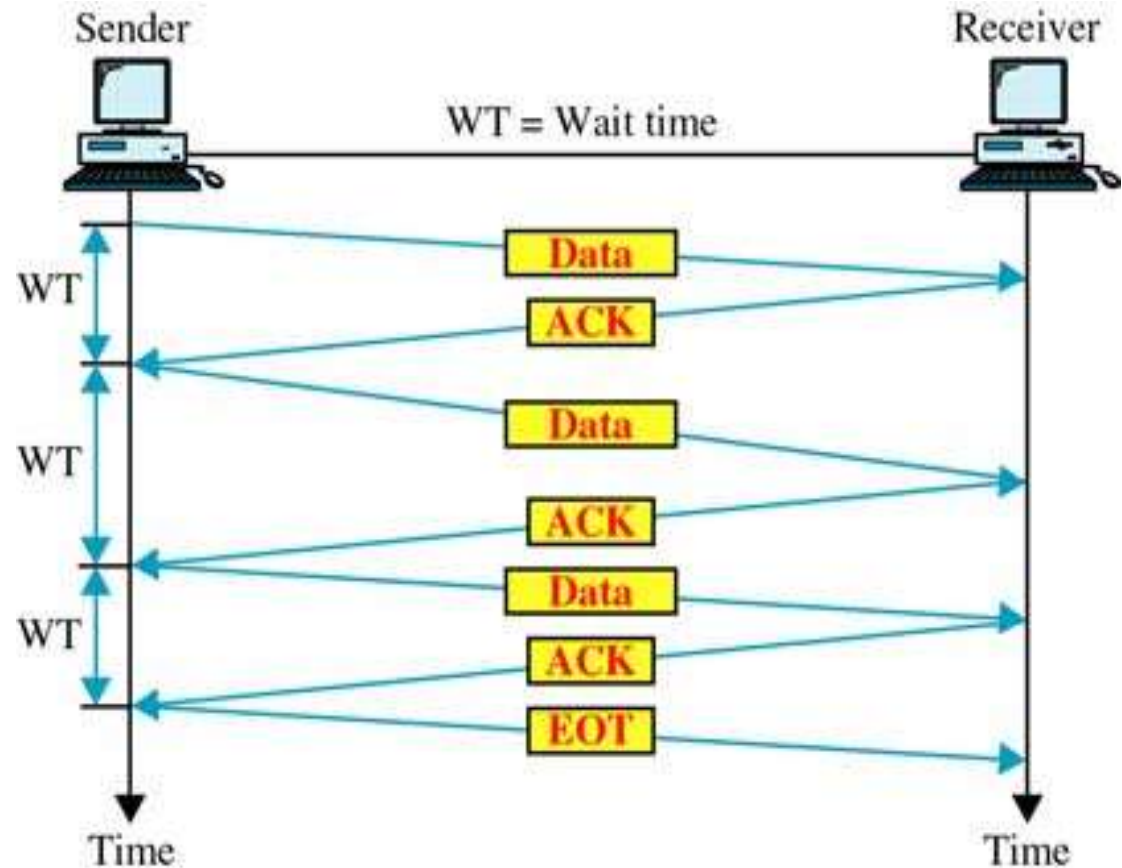
```
typedef enum {frame_arrival} event_type;  
#include "protocol.h"
```

```
void sender2(void)  
{  
    frame s;                                /* buffer for an outbound frame */  
    packet buffer;                          /* buffer for an outbound packet */  
    event_type event;                      /* frame_arrival is the only possibility */  
  
    while (true) {  
        from_network_layer(&buffer);      /* go get something to send */  
        s.info = buffer;                  /* copy it into s for transmission */  
        to_physical_layer(&s);             /* bye bye little frame */  
        wait_for_event(&event);           /* do not proceed until given the go ahead */  
    }  
}
```

```
void receiver2(void)  
{  
    frame r, s;                            /* buffers for frames */  
    event_type event;                      /* frame_arrival is the only possibility */  
    while (true) {  
        wait_for_event(&event);           /* only possibility is frame_arrival */  
        from_physical_layer(&r);          /* go get the inbound frame */  
        to_network_layer(&r.info);        /* pass the data to the network layer */  
        to_physical_layer(&s);            /* send a dummy frame to awaken sender */  
    }  
}
```

# Stop and Wait

- Source transmits a frame
- Destination receives frame and replies with ACK
- Source waits for ACK before sending next frame
- Destination can stop flow by not sending ACK



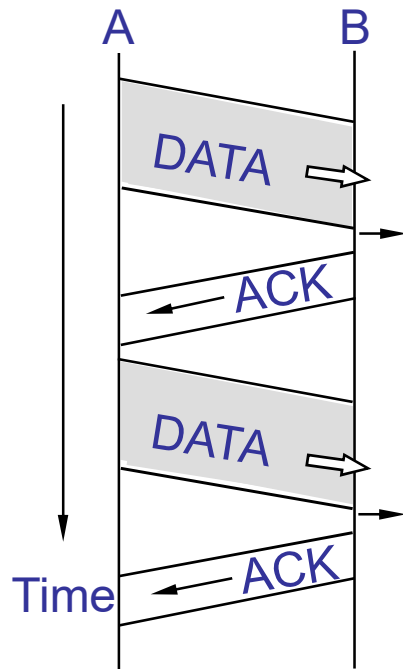
# 基本的流量控制协议

- 流量控制功能：限制发送方的速度，防止淹没接收方
- 流量控制方法：**停止等待**——发送方每发完一帧，即停下来等待；收到接收方反馈（**ACK**），发送方才能继续发送
- 协议**1**：理想信道（无错不丢）
  - ◆ 无需流量控制
- 协议**2**：信道无错，但接收方缓存有限
  - ◆ 流量控制：停止等待

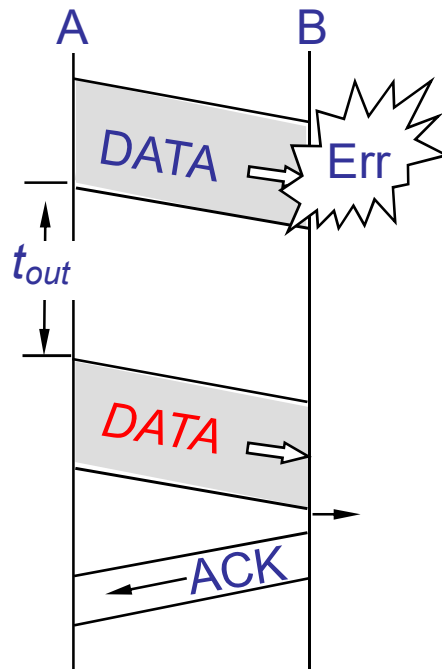
## Problems of protocol 2: If channel is not reliable

- When a frame is damaged
  - ◆ Receiver detects errors with **CRC** and discard the frame
  - ◆ Sender **retransmits** the frame when **timing out**
- When a frame is lost
  - ◆ Data frame lost
    - Sender retransmits when timing out
  - ◆ ACK frame lost
    - Sender retransmits when timing out

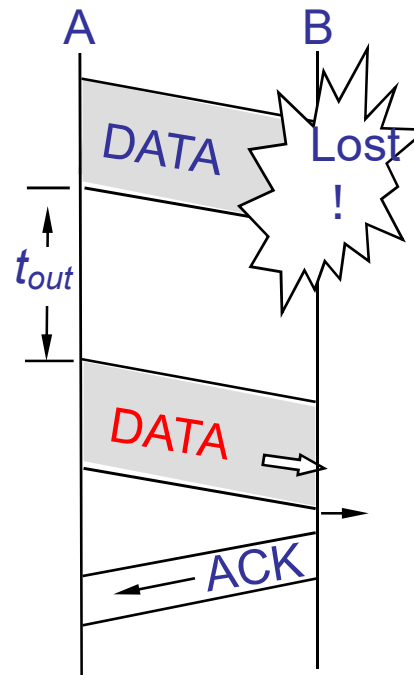
# Stop-And-Wait



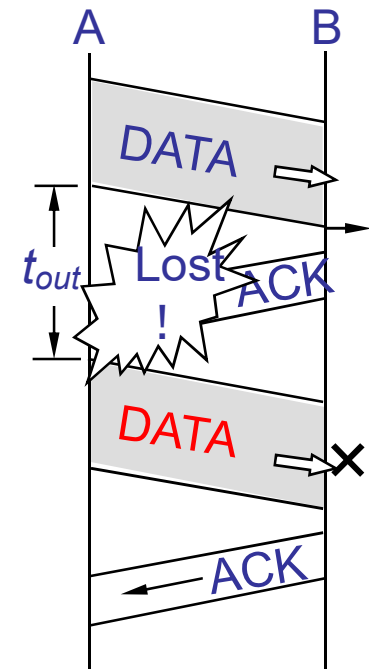
(a) Normal



(b) Data Error



(c) Data Lost



(d) ACK Lost

# More Problems

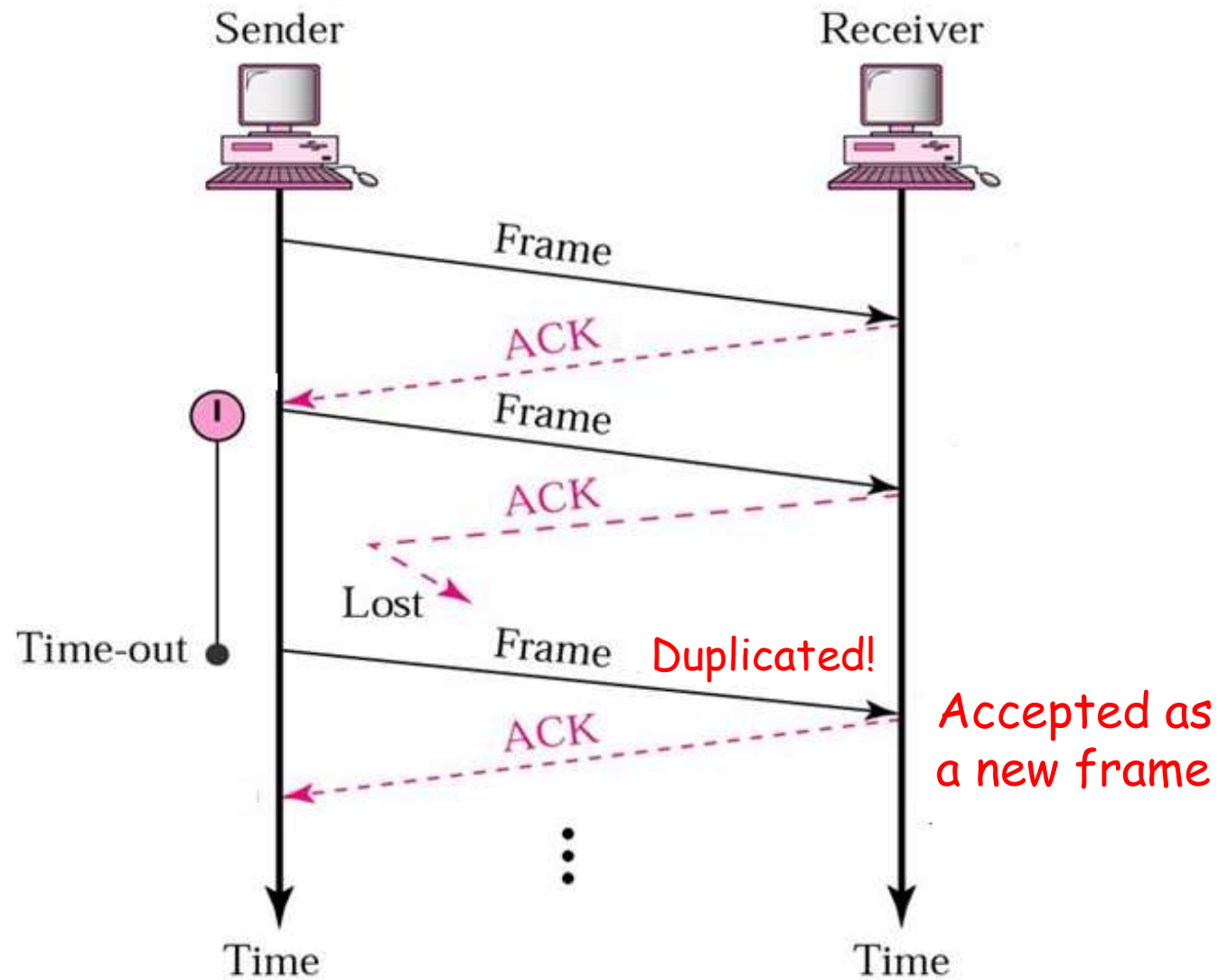
## ■ Case 1: Protocol failed on Receiver

- ◆ The network layer on sender A gives packet 1 to its data link layer. The packet is correctly received at receiver B and passed to the network layer on B. B sends an ACK frame back to A.
- ◆ The **ACK gets lost** completely
- ◆ The data link layer on A eventually times out. Not having received an acknowledgement, it sends the frame containing packet 1 again
- ◆ The **duplicate frame** also arrives B and is passed to the network layer

## ■ Case 2: Protocol failed on Sender

- ◆ A **delayed ACK**

# Problem of lost ACK





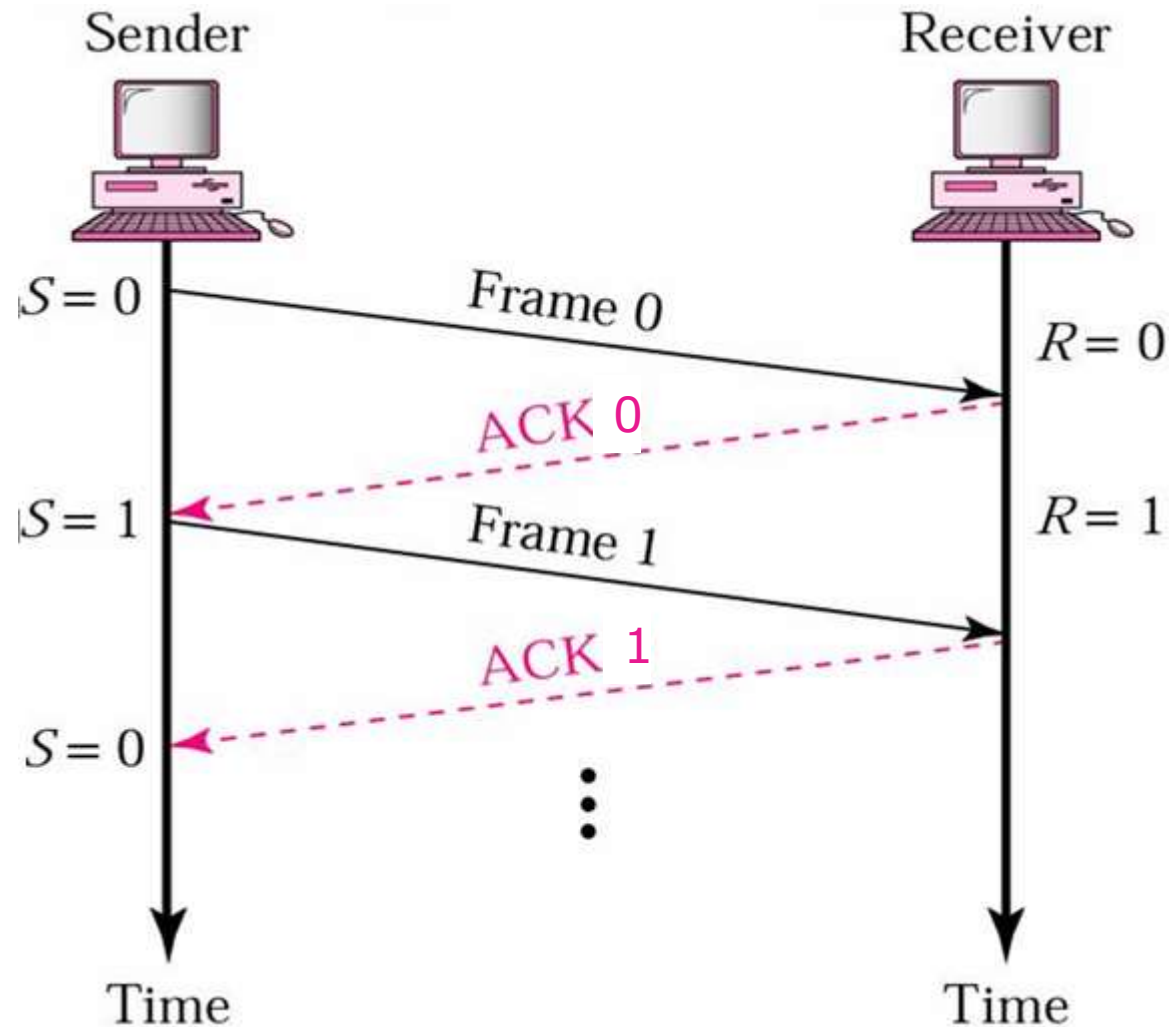
# Solutions: using Sequence Number

- Distinguish a frame that it is seeing for the first time from a retransmission
  - ◆ Put a **sequence number(序号)** in the header of each frame
- What is the minimum number of bits needed for the sequence number?
  - ◆ Data Link Layer: **1 bit**, with 2 sequence numbers (0 and 1) in all
  - ◆ Transport Layer: needing a much larger number space, eg. 32 bits
    - ➔ **ARQ: Automatic Repeat reQuest**  
(自动请求重传)

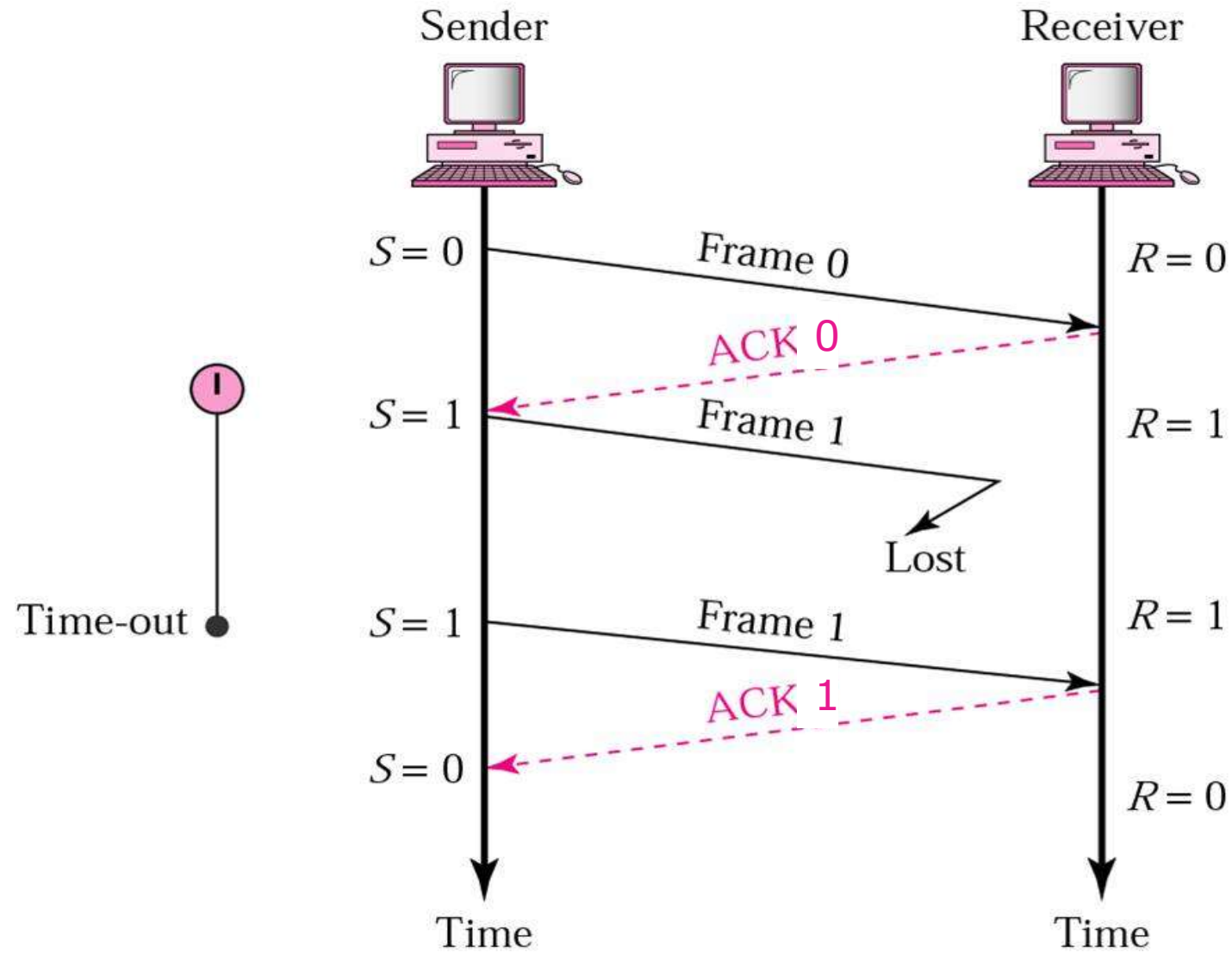
# Procedure of ARQ

- Error detection
  - ◆ Bad **CRC**/FCS (Frame Check Sequence) or lost frame
- Positive acknowledgment (确认)
  - ◆ **ACK** returned by receiver - may not be an ACK for each frame
- Retransmission by sender after timeout
  - ◆ **Sequence number** used to detect duplicated frame
- Negative acknowledgement (否认, optional)
  - ◆ **NAK** returned by receiver and retransmission by sender

# Stop-and-Wait ARQ: Normal Operation

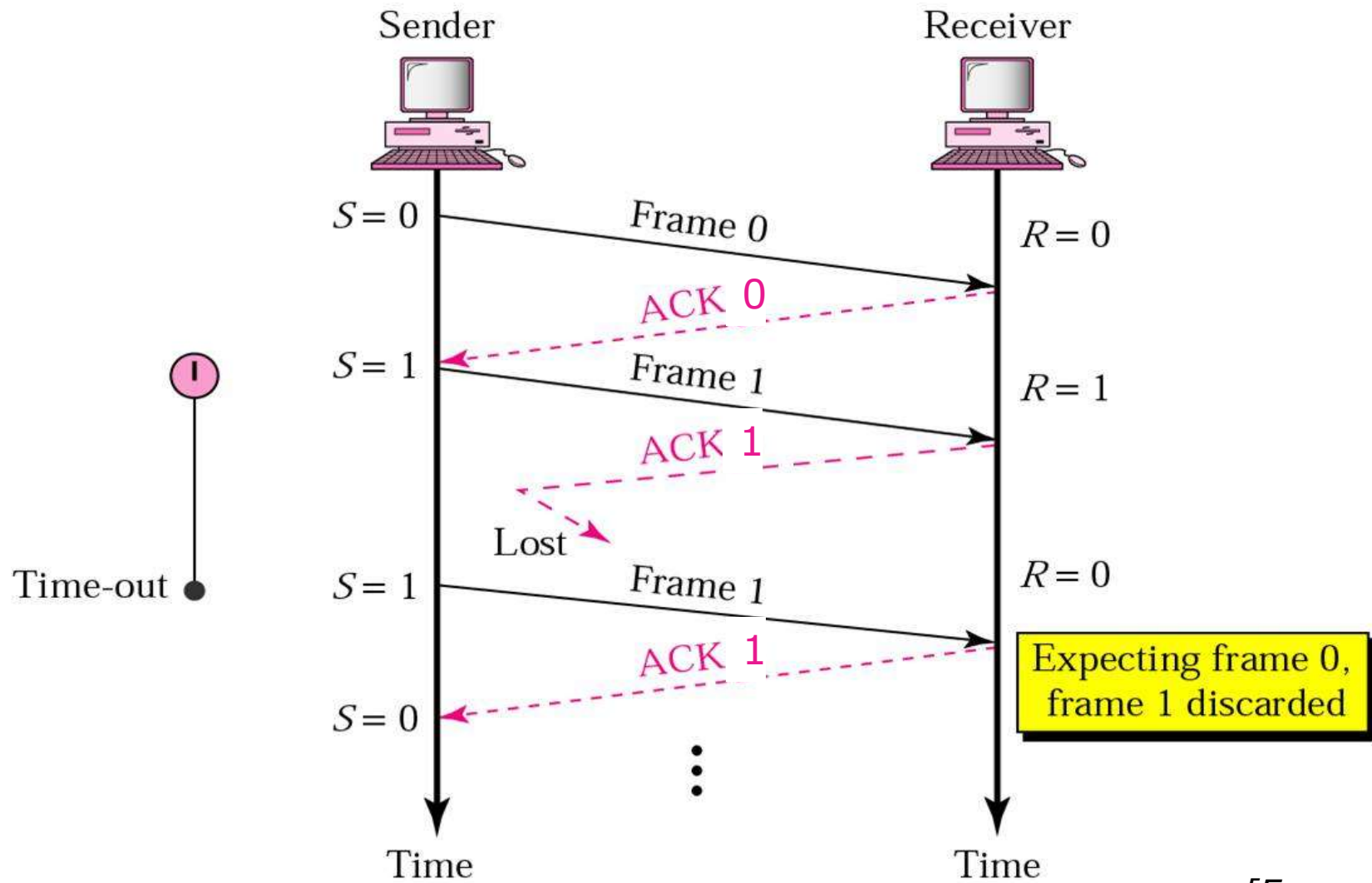


# Case 1: Frame gets lost



[Forouzan]

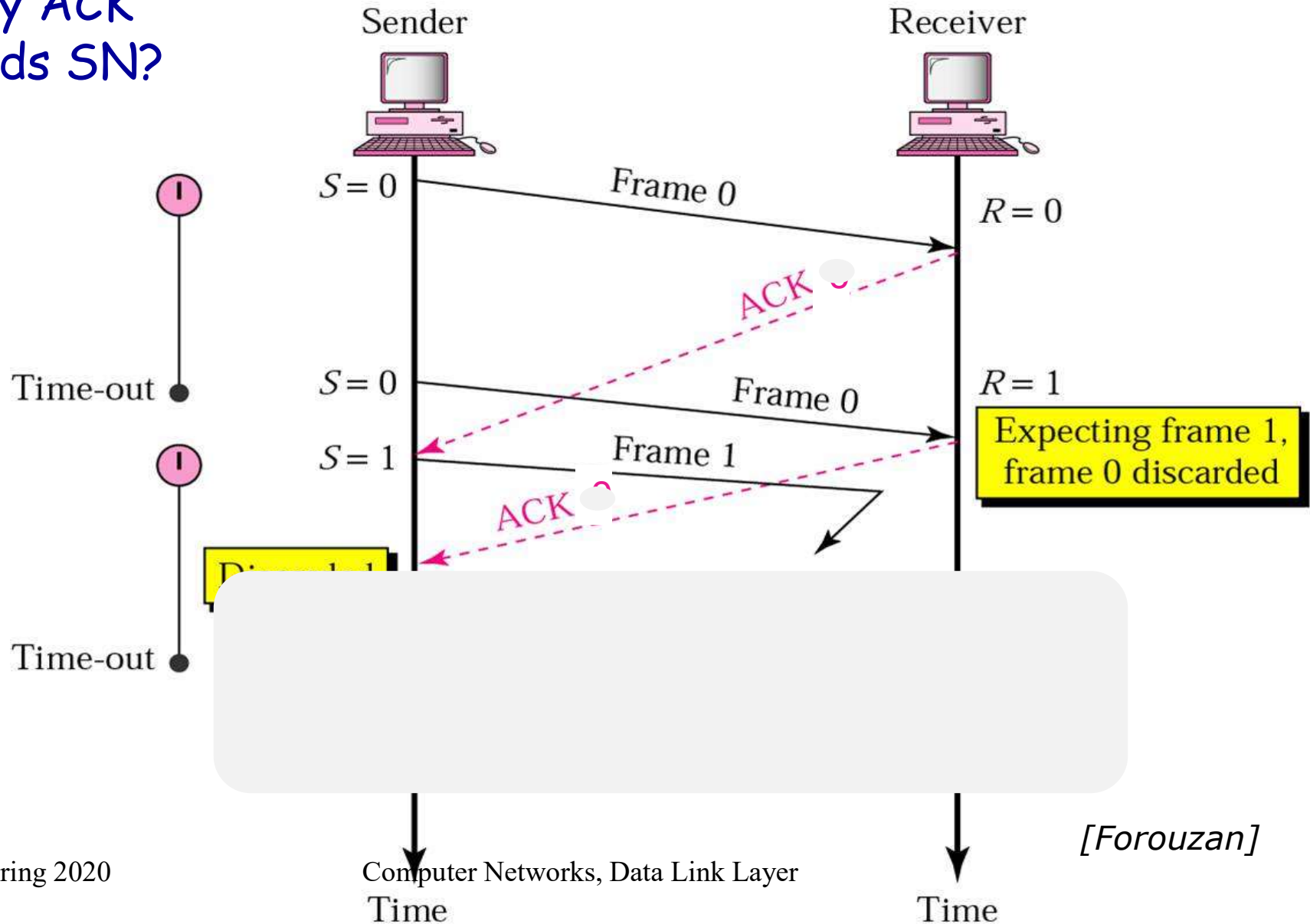
## Case 2: ACK gets lost



[Forouzan]

# Case 3: Delayed ACK

- Why ACK needs SN?



# 单向传输数据的停止等待ARQ协议

- 发送方在数据帧中增加校验字段，发送帧之后启动超时定时器
- 接收方检错，出错则丢弃此帧；如果校验无错，接收方发送确认帧（ACK）
- 收到ACK后，发送方发送下一帧，重启定时器
- 如果定时器超时，发送方重传刚发送的数据帧
- 数据帧中携带序号，接收方根据序号来判断是否是重复的数据帧
- ACK中携带序号，发送方根据序号判断是否是重复的ACK

# Protocol 3: Positive ACK with Retransmission(PAR)

## ■ Assumptions

- ◆ Channel is simplex
- ◆ Channel is **unreliable**
- ◆ Receiver has **limited** size of buffers

## ■ What are enhanced?

- ◆ Sequence number in frame
- ◆ Time out and retransmission



```

/* Protocol 3 (par) allows unidirectional data flow over an unreliable channel. */
#define MAX_SEQ 1 /* must be 1 for protocol 3 */
typedef enum {frame_arrival, cksum_err, timeout} event_type;
#include "protocol.h"

void sender3(void)
{
    seq_nr next_frame_to_send; /* seq number of next outgoing frame */
    frame s; /* scratch variable */
    packet buffer; /* buffer for an outbound packet */
    event_type event;

    next_frame_to_send = 0; /* initialize outbound sequence numbers */
    from_network_layer(&buffer); /* fetch first packet */
    while (true) {
        s.info = buffer; /* construct a frame for transmission */
        s.seq = next_frame_to_send; /* insert sequence number in frame */
        to_physical_layer(&s); /* send it on its way */
        start_timer(s.seq); /* if answer takes too long, time out */
        wait_for_event(&event); /* frame_arrival, cksum_err, timeout */
        if (event == frame_arrival) {
            from_physical_layer(&s); /* get the acknowledgement */
            if (s.ack == next_frame_to_send) {
                stop_timer(s.ack); /* turn the timer off */
                from_network_layer(&buffer); /* get the next one to send */
                inc(next_frame_to_send); /* invert next_frame_to_send */
            }
        }
    }
}

```

Sender

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |                                                                                                                                                                                                                                                                                                                                                                   |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> void receiver3(void) {     seq_nr frame_expected;     frame r, s;     event_type event;      frame_expected = 0;     while (true) {         wait_for_event(&amp;event);         if (event == frame_arrival) {             from_physical_layer(&amp;r);             if (r.seq == frame_expected) {                 to_network_layer(&amp;r.info);                 inc(frame_expected);             }             s.ack = 1 - frame_expected;             to_physical_layer(&amp;s);         }     } } </pre> | <pre> #ack= #frame-just-received  /* possibilities: frame_arrival, cksum_err */ /* a valid frame has arrived. */ /* go get the newly arrived frame */ /* this is what we have been waiting for. */ /* pass the data to the network layer */ /* next time expect the other sequence nr */  /* tell which frame is being acked */ /* send acknowledgement */ </pre> |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

Receiver

A positive acknowledgement with retransmission protocol.  
(PAR)

## 考虑差错控制:

### Sender

```
从网络层获取一个分组放入buffer
发送buffer中的数据, 新启动定时器
label1:
wait_for_event()
switch (event) {
case 收到了坏帧(校验和错):
    重发缓冲在buffer里的数据, 重新启动定时器
case 定时器超时:
    重发缓冲在buffer里的数据, 重新启动定时器
case 收到校验和正确的帧:
    if (ack序号正确) {
        关闭旧定时器
        从网络层获取下一个分组放入buffer
        发送buffer中的数据, 启动新的定时器
    } else
        重发缓冲在buffer里的数据, 重新启动定时器
}
goto label1
```

## 考虑差错控制: Receiver

```
frame_expected=0
while(true) {
    wait_for_event()
    switch(event) {
    case 坏帧:
        do_nothing; or send NAK
    case 收到校验和正确的数据帧:
        if(序号==frame_expected) {
            向网络层上交分组
            回ACK(序号为frame_expected)
            inc(frame_expected)
        } else {
            回ACK (序号为1-frame_expected)
        }
    }
}
```

# About Timer

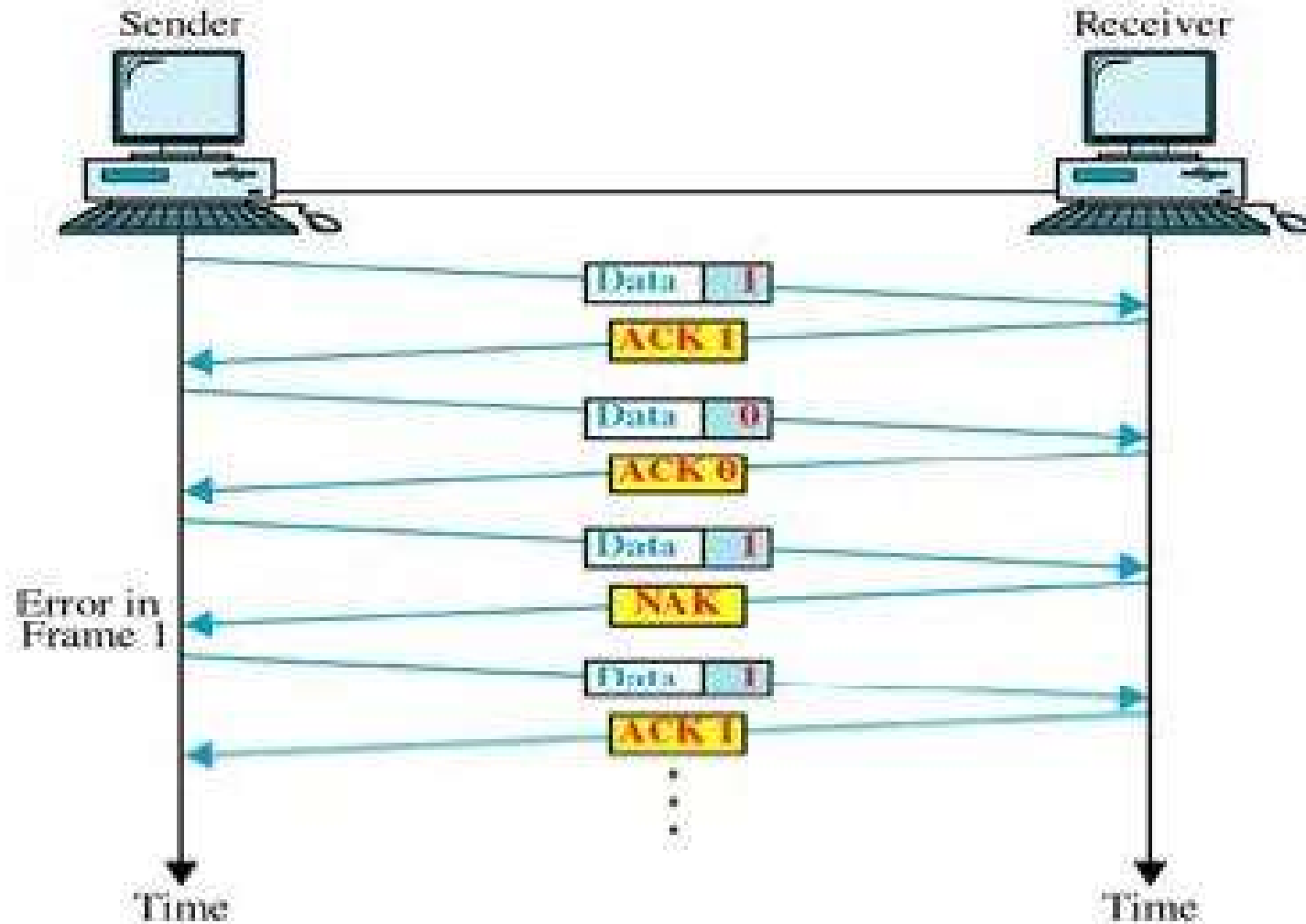
## ■ Timeout

- ◆ If the timeout interval is set too short, the sender will transmit unnecessary frames. While these extra frames will not affect the correctness of the protocol, they will hurt performance

## ■ About NAK

- ◆ Is NAK mandatory?
- ◆ Good things using NAK?

# Stop-and-Wait ARQ: Optional NAK



# Basic Flow Control Protocols

## ■ Elementary Protocols

- ◆ A Utopian Simplex Protocol
- ◆ A Simplex Stop-and-Wait Protocol for an Error-Free Channel
- ◆ A Simplex Protocol for a Noisy Channel

## ■ Sliding Window Protocols

- ◆ A One-Bit Sliding Window Protocol
- ◆ A Protocol Using Go Back N
- ◆ A Protocol Using Selective Repeat

# Sliding-Window(滑动窗口) Flow Control: Principles

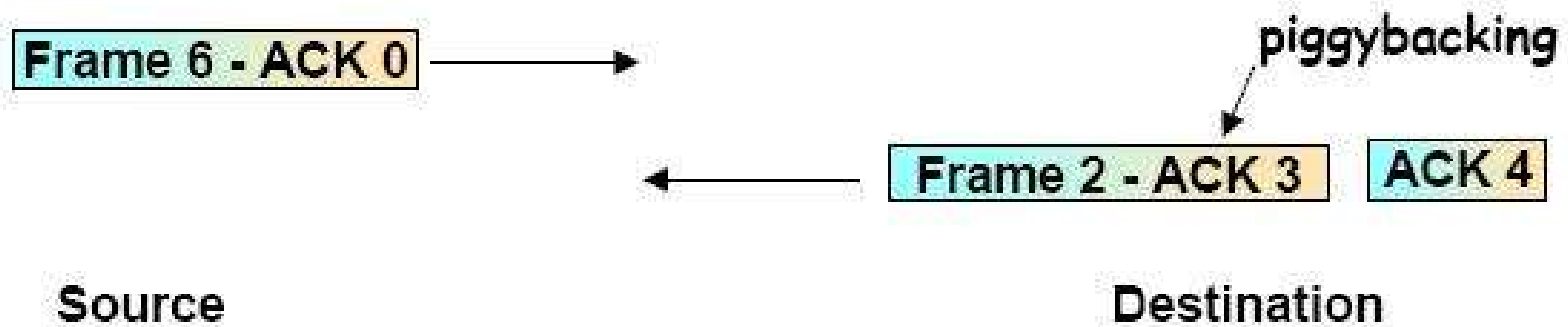
- Full-duplex(全双工) data transmission, allowing piggybacking(捎带应答) 
- Allow multiple frames to be in transit at a time
- Sender can send up to  $W$  frames without ACK
  - ◆  $W$  is the sending window size
- Destination may be able to buffer multiple frames
- Sequence number bounded by size of field ( $n$ )
  - ◆ Frames are numbered modulo  $2^n$  ( $0 \dots 2^n - 1$ )
  - ◆ Window size does not have to be max allowed





# Piggybacking(捎带应答/确认)

- When a data frame arrives, instead of immediately sending a separate control frame (ACK), the receiver restrains itself and waits until the network layer passes it the next packet
- ACK is attached to the outgoing data frame (using the ACK field in the frame header)



# Sending window & Receiving window

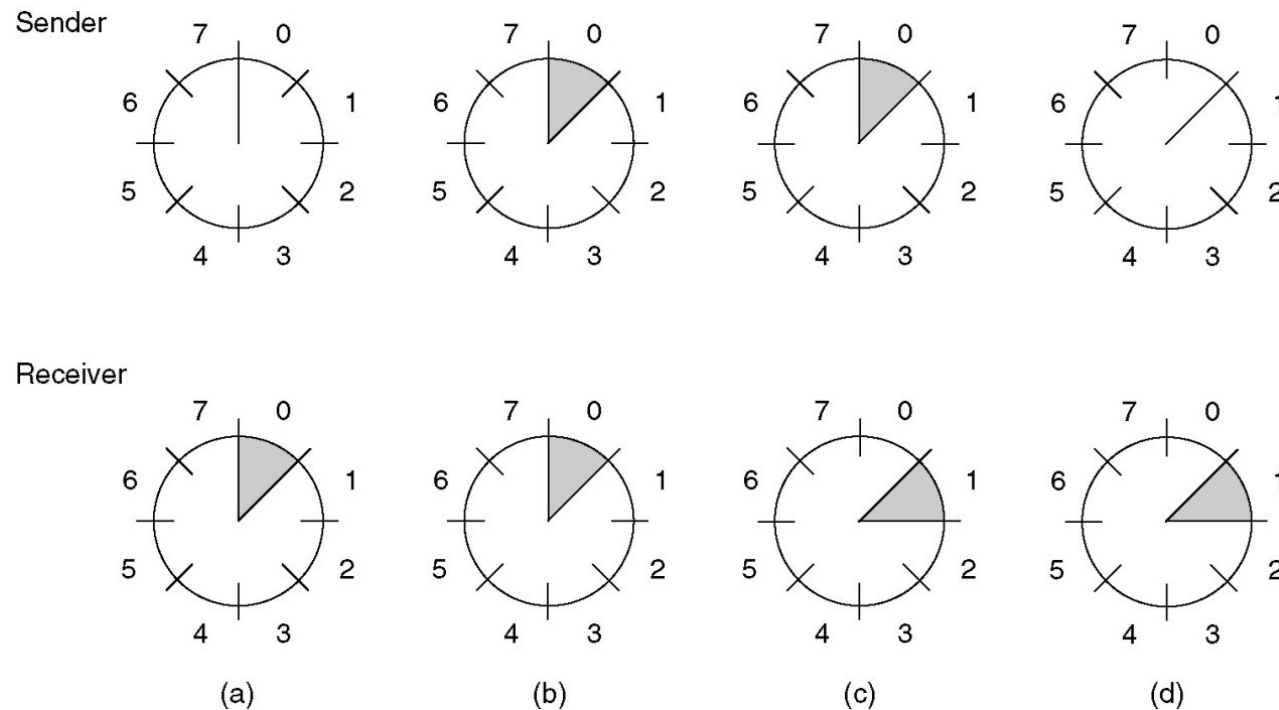
## ■ Sending window

- ◆ At any instant of time, the sender maintains a set of sequence numbers corresponding to frames it is permitted to send.
- ◆ These frames are said to fall within the sending window

## ■ Receiving window

- ◆ Receiver also maintains a receiving window corresponding to the set of frames it is permitted to accept

# Sliding Window of Size 1



A sliding window of size 1, with a 3-bit sequence number.

(a) Initially.

(b) After the first frame has been sent.

(c) After the first frame has been received.

(d) After the first ACK has been received.

# One-Bit Sliding Window Protocol (1)

```
/* Protocol 4 (sliding window) is bidirectional. */
#define MAX_SEQ 1 /* must be 1 for protocol 4 */
typedef enum {frame_arrival, cksum_err, timeout} event_type;
#include "protocol.h"
void protocol4 (void)
{
    seq_nr next_frame_to_send; /* 0 or 1 only */
    seq_nr frame_expected; /* 0 or 1 only */
    frame r, s; /* scratch variables */
    packet buffer; /* current packet being sent */
    event_type event;

    next_frame_to_send = 0; /* next frame on the outbound stream */
    frame_expected = 0; /* frame expected next */
    from_network_layer(&buffer); /* fetch a packet from the network layer */
    s.info = buffer; /* prepare to send the initial frame */
    s.seq = next_frame_to_send; /* insert sequence number into frame */
    s.ack = 1 - frame_expected; /* piggybacked ack */
    to_physical_layer(&s); /* transmit the frame */
    start_timer(s.seq); /* start the timer running */
}
```

# One-Bit Sliding Window Protocol (2)

```
while (true) {  
    wait_for_event(&event);           /* frame_arrival, cksum_err, or timeout */  
    if (event == frame_arrival) {     /* a frame has arrived undamaged. */  
        from_physical_layer(&r);      /* go get it */  
  
        if (r.seq == frame_expected) { /* handle inbound frame stream. */  
            to_network_layer(&r.info); /* pass packet to network layer */  
            inc(frame_expected);        /* invert seq number expected next */  
        }  
  
        if (r.ack == next_frame_to_send) { /* handle outbound frame stream. */  
            stop_timer(r.ack);           /* turn the timer off */  
            from_network_layer(&buffer); /* fetch new pkt from network layer */  
            inc(next_frame_to_send);     /* invert sender's sequence number */  
        }  
  
        s.info = buffer;                /* construct outbound frame */  
        s.seq = next_frame_to_send;    /* insert sequence number into it */  
        s.ack = 1 - frame_expected;   /* seq number of last received frame */  
        to_physical_layer(&s);          /* transmit a frame */  
        start_timer(s.seq);             /* start the timer running */  
    }  
}
```

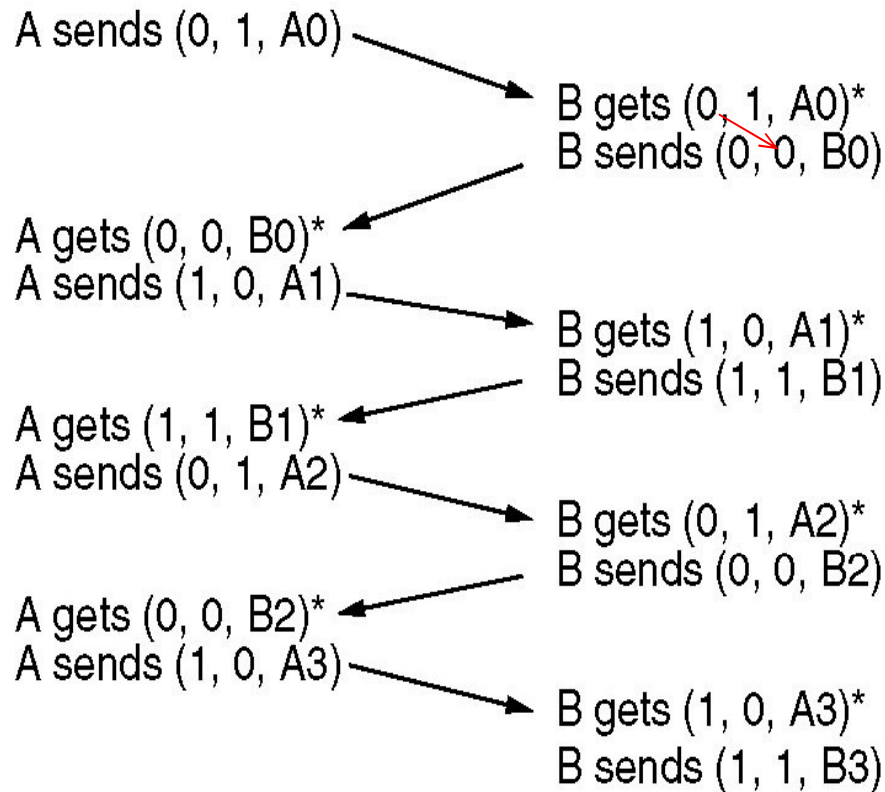
收到正确序号帧，提交网络层，接收窗口前移1格

上帧已正确到达，获取网络层新包，帧序号+1

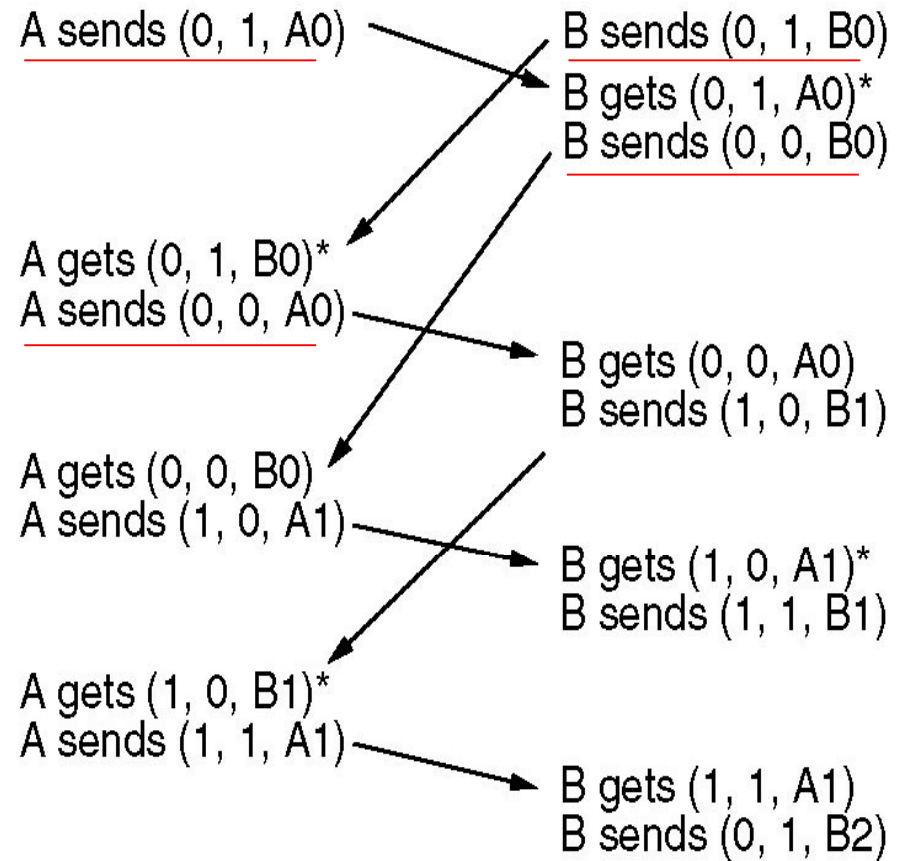
2020春

计算机网络, 数据链路层

# Peculiar Situation of Protocol 4: Simultaneously Sending



Normal case



Abnormal case: frames sent twice

The notation is (seq, ack, packet)

#ack= #frame-just-received

\* indicates where a network layer accepts a packet

# 实用的停止等待ARQ协议

- 发送方在数据帧中增加**校验**字段，发送帧之后启动**超时定时器**
- 接收方检错，出错则丢弃此帧；如果校验无错，接收方发送确认帧（**ACK**）或者在回传的数据帧中**捎带确认**
- 收到确认后，发送方发送下一帧，重启定时器
- 如果定时器超时，发送方**重传**刚发送的数据帧
- **数据帧**中携带**发送序号**，接收方根据序号来判断是否是重复的数据帧
- 发送方根据**ACK序号**判断是否是重复的ACK



## Problem of Protocol 4: low efficiency

- Consider a 50-kbps satellite channel with a 500-msec round-trip propagation delay, send 1000-bit frames
  - ◆  $t=0$ : sender starts sending
  - ◆  $t=20$  msec: the frame has been completely sent
  - ◆  $t=270$  msec: frame fully arrived at the receiver
  - ◆  $t=520$  msec: ACK arrived back at the sender, under the best of circumstances (no waiting in the receiver and a short acknowledgement frame).
- Conclusion
  - ◆ Sender was blocked during 500/520 or 96% of the time. only 4% of the available bandwidth was used



# Basic Flow Control Protocols

## ■ Elementary Protocols

- ◆ An Unrestricted Simplex Protocol
- ◆ A Simplex Stop-and-Wait Protocol
- ◆ A Simplex Protocol for a Noisy Channel

## ■ Sliding Window Protocols

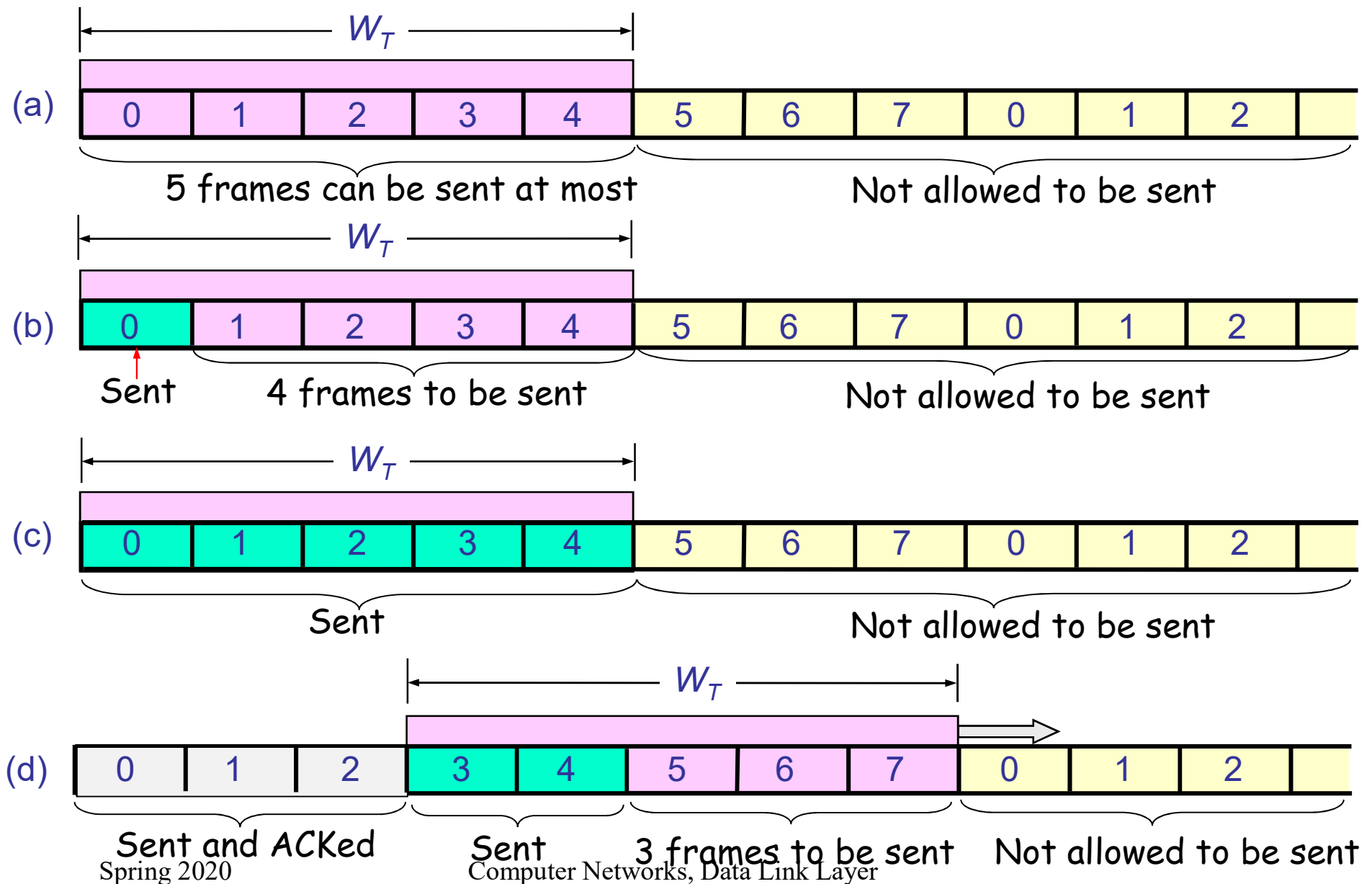
- ◆ A One-Bit Sliding Window Protocol
- ◆ A Protocol Using Go Back N
- ◆ A Protocol Using Selective Repeat

# Go Back N（回退N步）：Principles

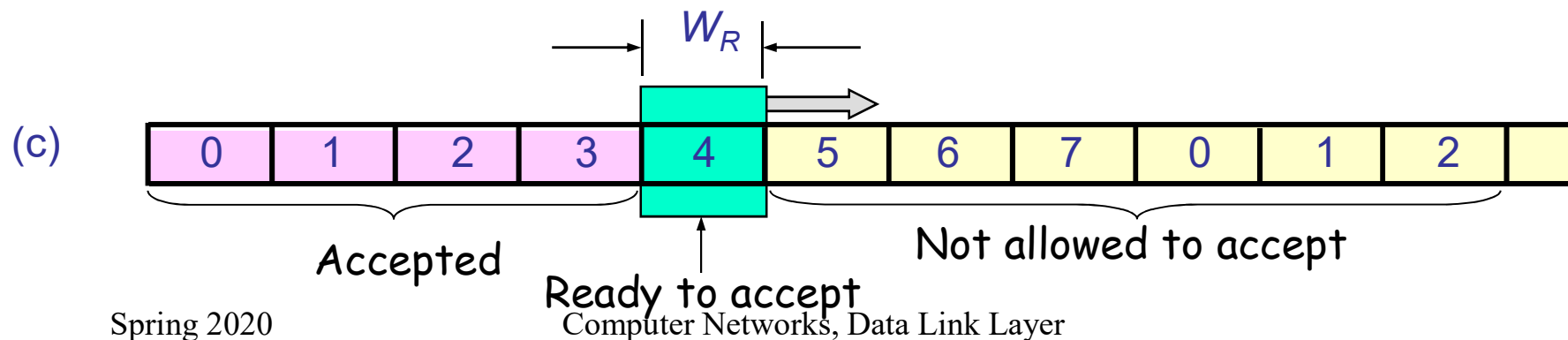
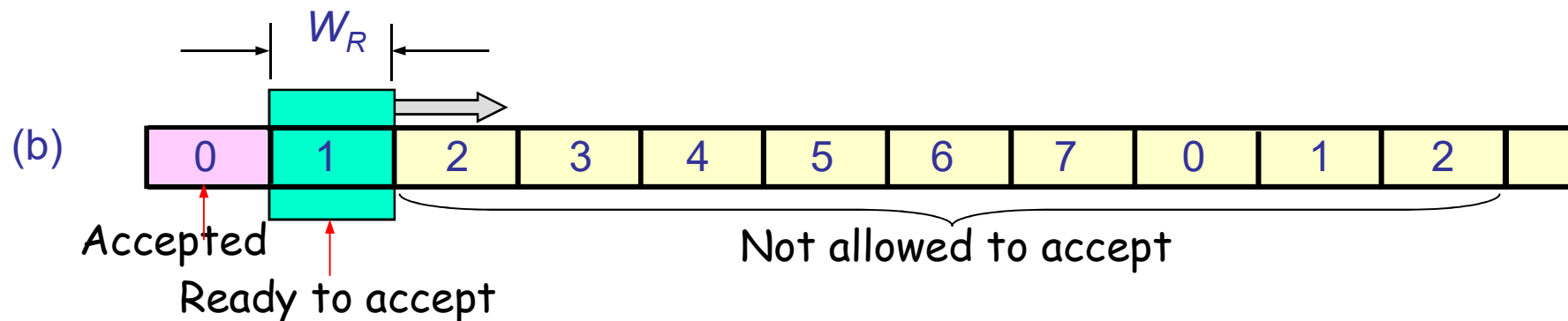
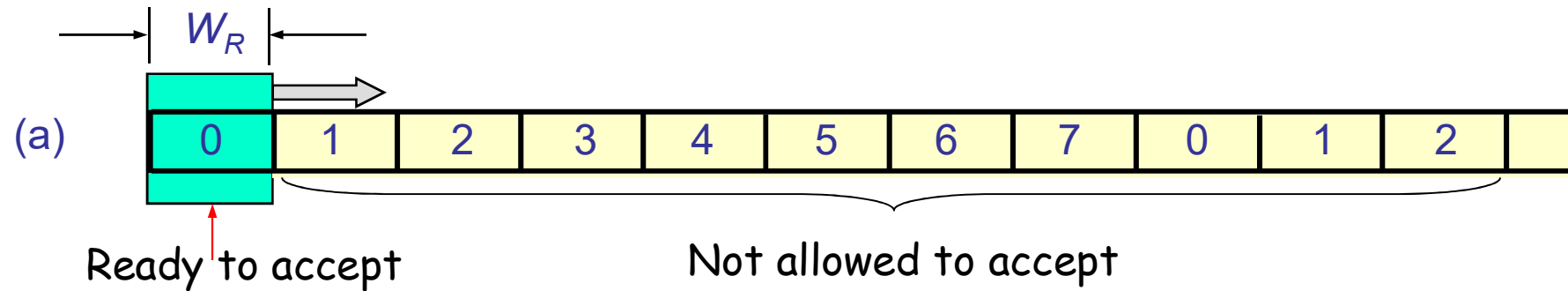
Based on sliding window

- ◆ Sending window: buffers for outstanding frames
- ◆ Receiving window: buffers to receive frames
- Use window to control number of outstanding frames
  - ◆ Sender is allowed to send multiple frames before receiving ACK: **pipelining**(流水线) 
- If Receiver detects error, discard that frame and all future frames until error frame received correctly
- Transmitter must **go back and retransmit that frame and all subsequent frames** 

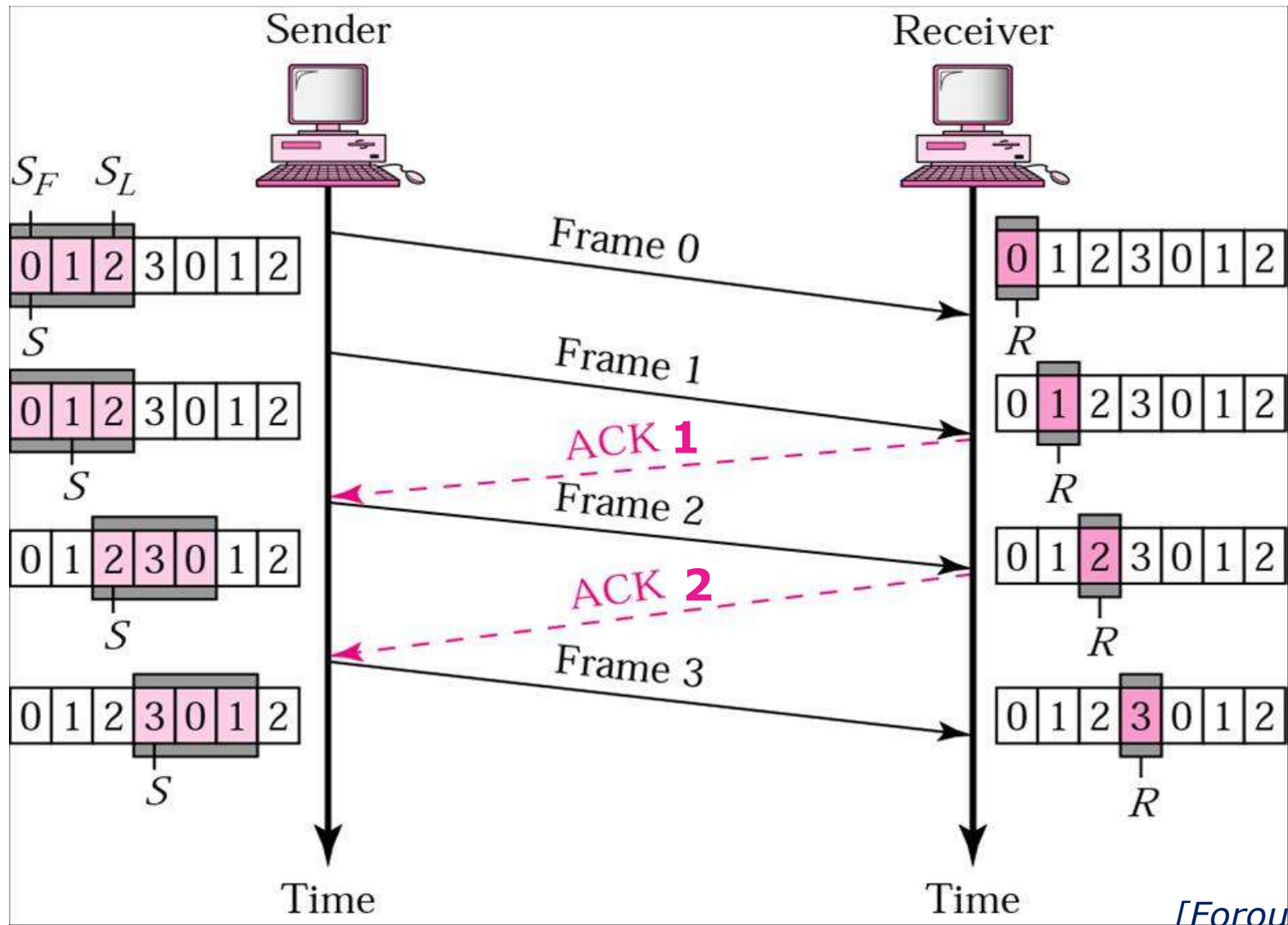
# Idea of Sending Window in GBN: $W_T > 1$



# Idea of Receiving Window in GBN: $W_R=1$

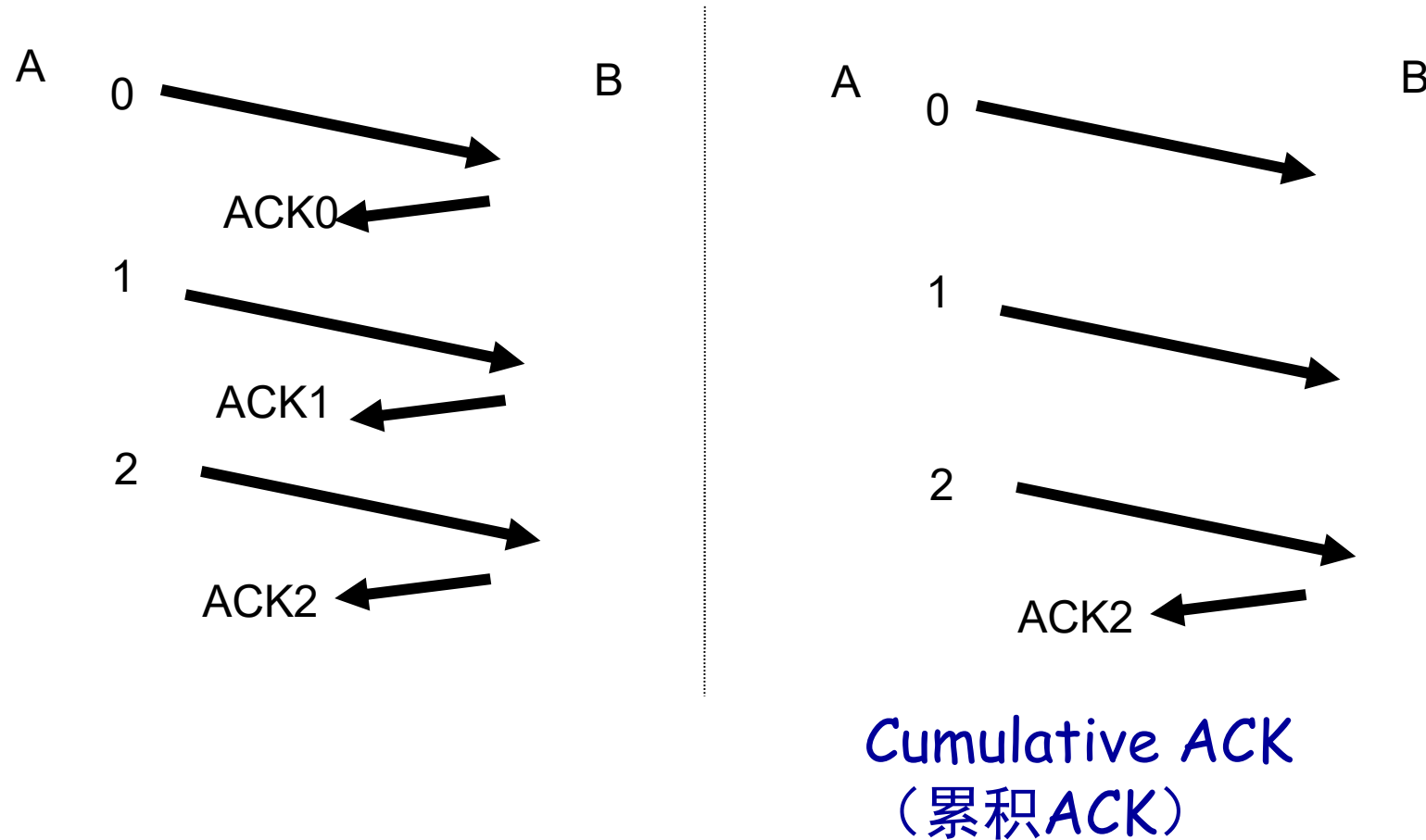


# Go Back N: Normal Operation

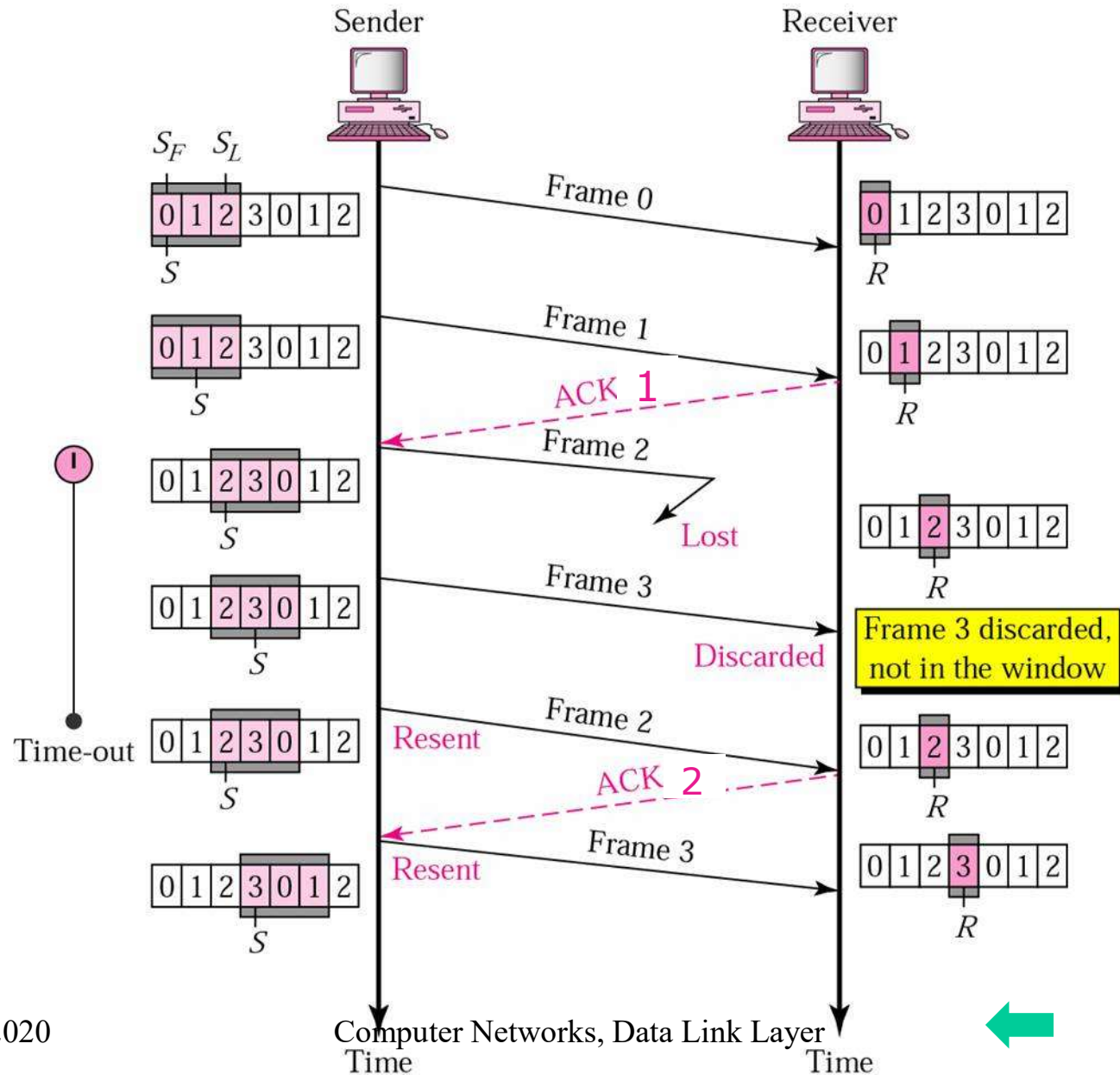


[Forouzan]

# Go Back N: No Need to ACK Every Frame

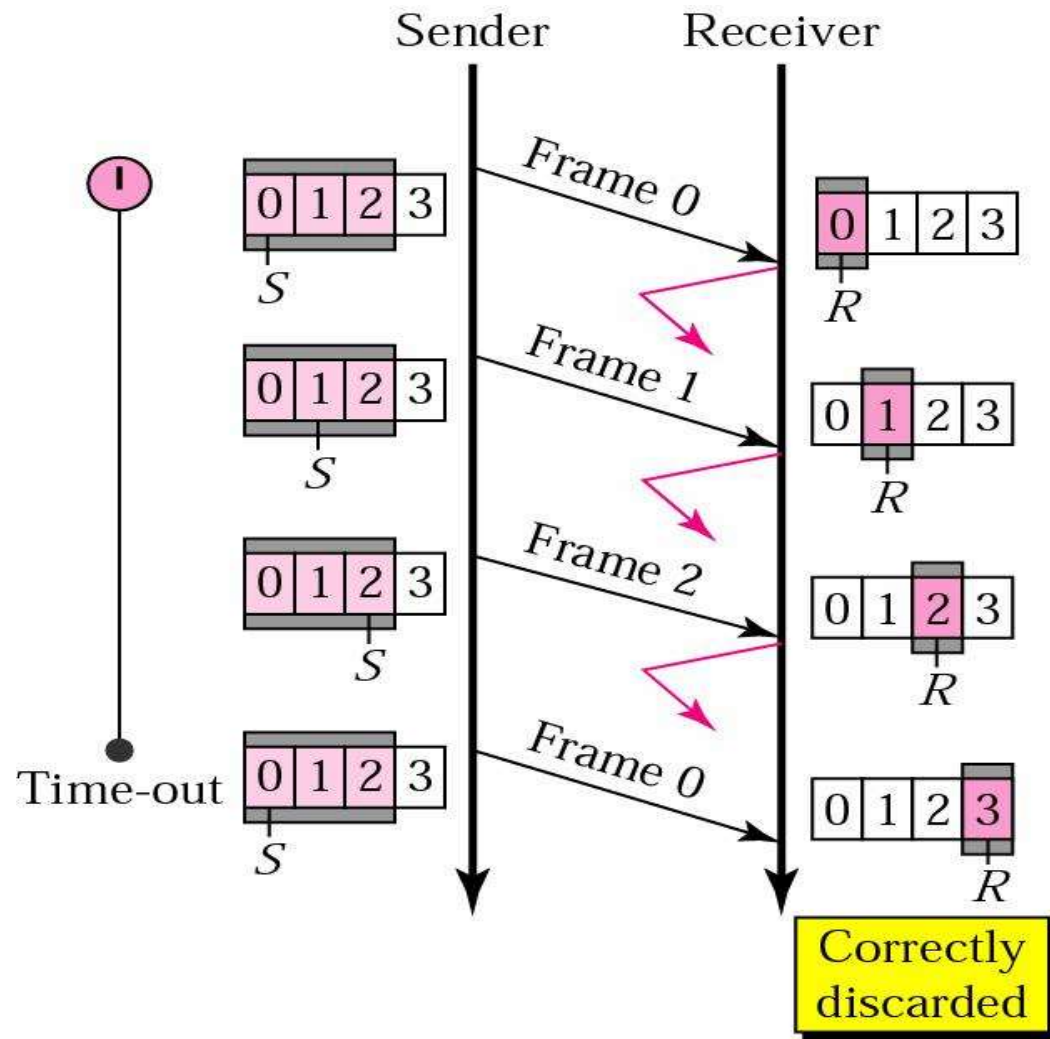


# Go Back N: Frame Lost



[Forouzan]

# Go Back N: ACK Lost





# Go Back N: Frame Damaged

- If Data Frame is Damaged
  - ◆ Receiver detects error in frame  $i$
  - ◆ Receiver discard that frame  $i$  and all subsequent
  - ◆ When timeout, sender retransmits frame  $i$  and all subsequent
- If ACK Frame is Damaged
  - ◆ Receiver gets frame  $i$  and send acknowledgement  $ACK(i)$  which is lost
  - ◆ Acknowledgements are cumulative, so next  $ACK(i+n)$  may arrive before sender times out on frame  $i$ , thus no need for retransmission
  - ◆ If sender times out, retransmits frame  $i$  and all subsequent

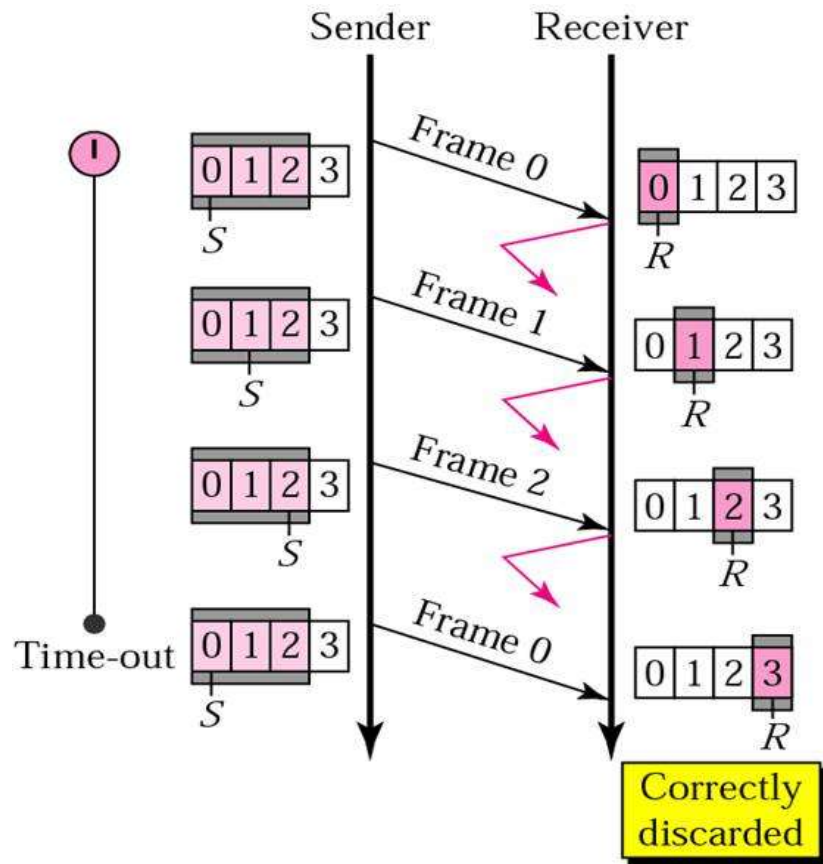
# Channel Efficiency of Go Back N

- Consider a 50-kbps satellite channel with a 500-msec round-trip propagation delay, send 1000-bit frames,  $W_T=26$ 
  - ◆  $t=0$ : sender starts sending
  - ◆  $t=20$  ms: the 1<sup>st</sup> frame has been completely sent
  - ◆  $t=270$  ms: frame fully arrived at the receiver
  - ◆  $t= 520$  ms: the 26<sup>th</sup> frame has been sent
  - ◆  $t=520$  ms: ACK arrived back at the sender
- Channel efficiency
  - ◆  $520/520=100\%$
  - ◆ Pipelining (流水线)

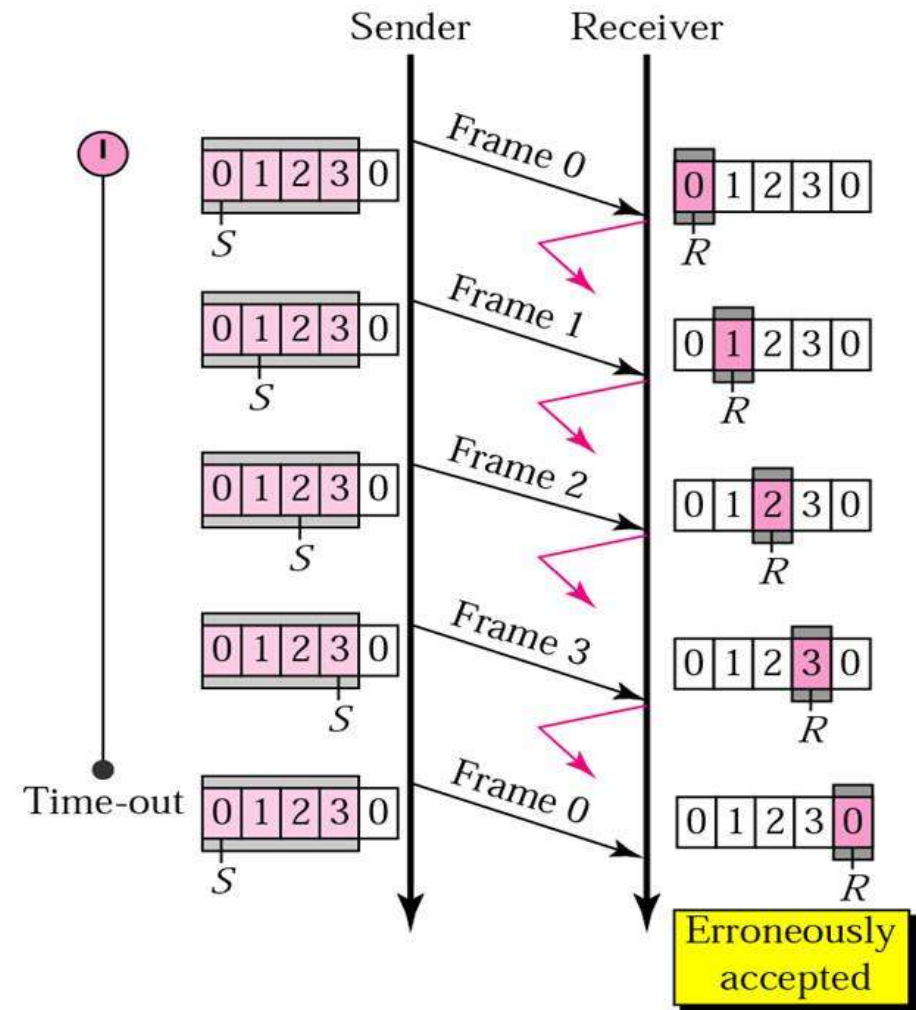
# Maximum of Sending Window Size

- Can window size be  $2^n$  for  $n$ -bit sequence number?
- Considering following case:
  - The sender sends frames 0 through 7.
  - The receiver gets all 8 frames and sends back ACKs.
  - All ACKs are lost.
  - The sender re-transmit frame 0.
- Problem: Can receiver detect that it is a duplicate frame?
- 原则
  - ◆ 当发送方发送完发送窗口内所有帧，在未收到ACK之前，接收方正确接收到所有发送来的帧，接收窗口向前推移，此时，必须保证发送窗口与接收窗口在序号上不能重叠
  - ◆ For  $n$ -bit sequence number,  $W_T \leq 2^n - 1$

# Maximum of Sending Window Size: Example of 2-bit SN( $m=2$ )



a. Window size  $< 2^m$

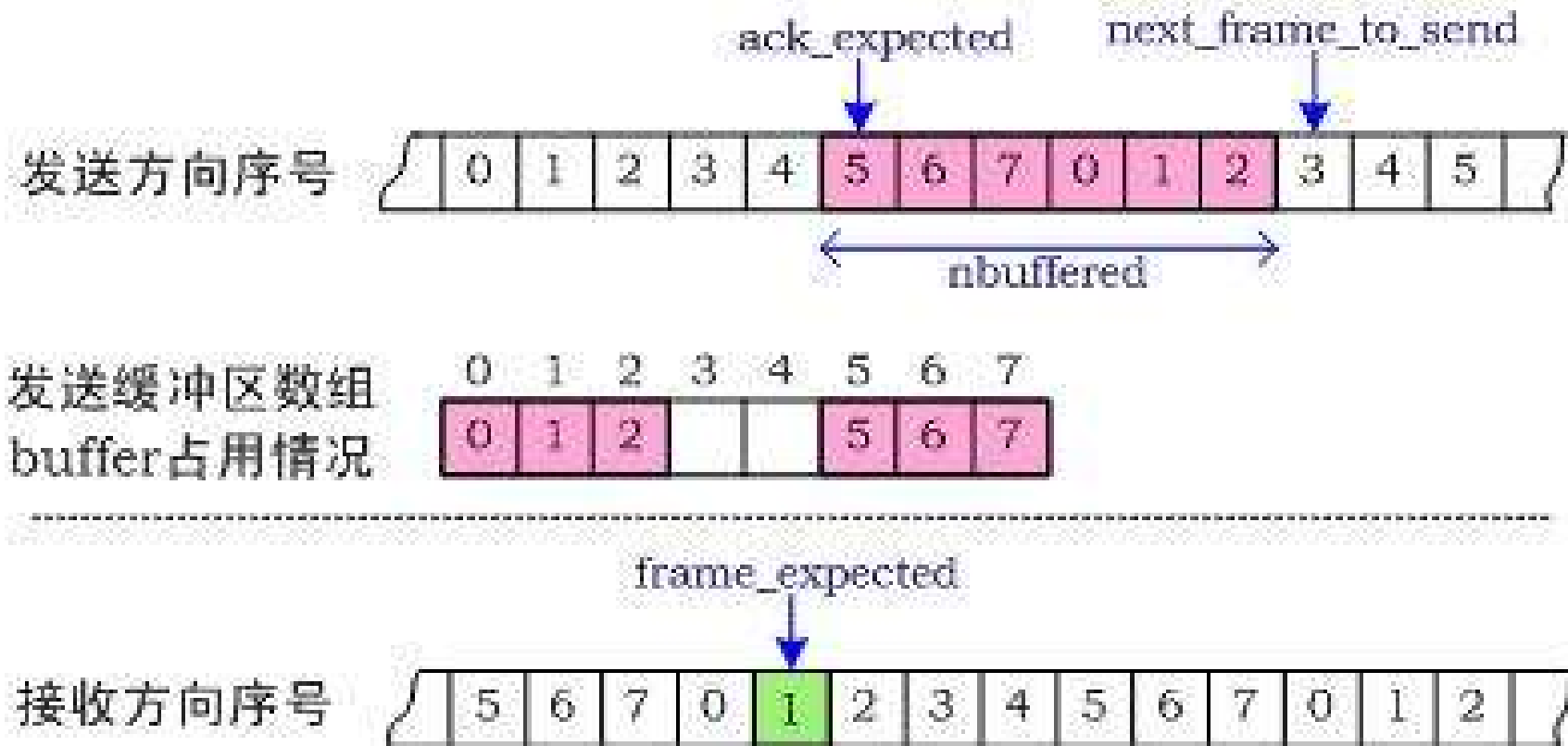


b. Window size  $= 2^m$

[Forouzan]

# Protocol 5: Go Back N Protocol

3-bit Sequence Number, Max sending window= 7



/\* Protocol 5 (Go-back-n) allows multiple outstanding frames. The sender may transmit up to MAX\_SEQ frames without waiting for an ack. In addition, unlike in the previous protocols, the network layer is not assumed to have a new packet all the time. Instead, the network layer causes a network\_layer\_ready event when there is a packet to send. \*/

```
#define MAX_SEQ 7
```

```
typedef enum {frame_arrival, cksum_err, timeout, network_layer_ready} event_type;
```

```
#include "protocol.h"
```

```
static boolean between(seq_nr a, seq_nr b, seq_nr c)
```

```
{
```

```
/* Return true if a <= b < c circularly; false otherwise. */
```

```
if (((a <= b) && (b < c)) || ((c < a) && (a <= b)) || ((b < c) && (c < a)))
```

```
    return(true);
```

```
    else
```

```
        return(false);
```

```
}
```

```
static void send_data(seq_nr frame_nr, seq_nr frame_expected, packet buffer[])
```

```
{
```

```
/* Construct and send a data frame. */
```

```
    frame s;
```

```
/* scratch variable */
```

```
    s.info = buffer[frame_nr];
```

```
/* insert packet into frame */
```

```
    s.seq = frame_nr;
```

```
/* insert sequence number into frame */
```

```
    s.ack = (frame_expected + MAX_SEQ) % (MAX_SEQ + 1); /* piggyback ack */
```

```
    to_physical_layer(&s);
```

```
/* transmit the frame */
```

```
    start_timer(frame_nr);
```

```
/* start the timer running */
```

```
} Spring 2020
```

```
Computer Networks, Data Link Layer
```

Upper Window Edge+1

Lower Window Edge

```
void protocol5(void)
{
    seq_nr next_frame_to_send; /* MAX_SEQ > 1; used for outbound stream */
    seq_nr ack_expected; /* oldest frame as yet unacknowledged */
    seq_nr frame_expected; /* next frame expected on inbound stream */
    frame r; /* scratch variable */
    packet buffer[MAX_SEQ + 1]; /* buffers for the outbound stream */
    seq_nr nbuffered; /* # output buffers currently in use */
    seq_nr i; /* used to index into the buffer array */
    event_type event;

    enable_network_layer(); /* allow network_layer_ready events */
    ack_expected = 0; /* next ack expected inbound */
    next_frame_to_send = 0; /* next frame going out */
    frame_expected = 0; /* number of frame expected inbound */
    nbuffered = 0; /* initially no packets are buffered */
}
```



```

while (true) {
    wait_for_event(&event);          /* four possibilities: see event_type above */

    switch(event) {
        case network_layer_ready:    /* the network layer has a packet to send */
            /* Accept, save, and transmit a new frame. */
            from_network_layer(&buffer[next_frame_to_send]); /* fetch new packet */
            nbuffered = nbuffered + 1; /* expand the sender's window */
            send_data(next_frame_to_send, frame_expected, buffer); /* transmit the frame */
            inc(next_frame_to_send); /* advance sender's upper window edge */
            break;

        case frame_arrival:          /* a data or control frame has arrived */
            from_physical_layer(&r); /* get incoming frame from physical layer */

            if (r.seq == frame_expected) {
                /* Frames are accepted only in order. */
                to_network_layer(&r.info); /* pass packet to network layer */
                inc(frame_expected); /* advance lower edge of receiver's window */
            }
    }
}

```



```

/* Ack n implies n - 1, n - 2, etc. Check for this. */
while (between(ack_expected, r.ack, next_frame_to_send)) {
    /* Handle piggybacked ack. */
    nbuffered = nbuffered - 1; /* one frame fewer buffered */
    stop_timer(ack_expected); /* frame arrived intact; stop timer */
    inc(ack_expected); /* contract sender's window */
}
break;

```

```

case cksum_err: break; /* just ignore bad frames */

```

```

case timeout: /* trouble; retransmit all outstanding frames */
    next_frame_to_send = ack_expected; /* start retransmitting here */
    for (i = 1; i <= nbuffered; i++) {
        send_data(next_frame_to_send, frame_expected, buffer); /* resend 1 frame */
        inc(next_frame_to_send); /* prepare to send the next one */
    }
}

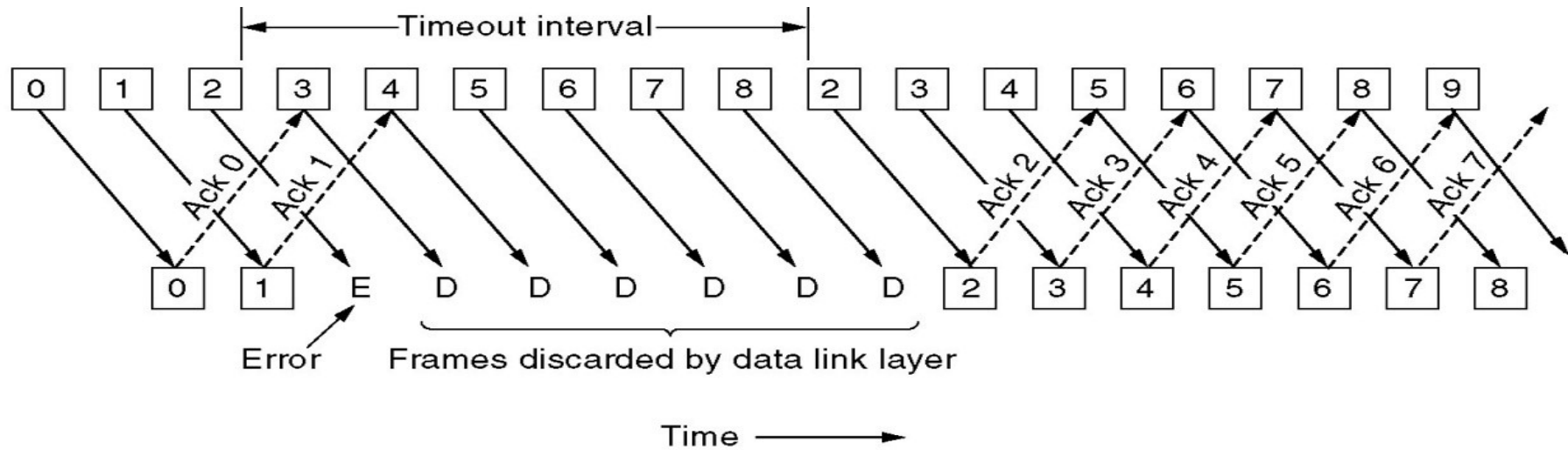
```

```

if (nbuffered < MAX_SEQ)
    enable_network_layer();
else
    disable_network_layer();
}

```

# Problems of Go Back N



Receiver's window size is 1.

# Basic Flow Control Protocols

## ■ Elementary Protocols

- ◆ An Unrestricted Simplex Protocol
- ◆ A Simplex Stop-and-Wait Protocol
- ◆ A Simplex Protocol for a Noisy Channel

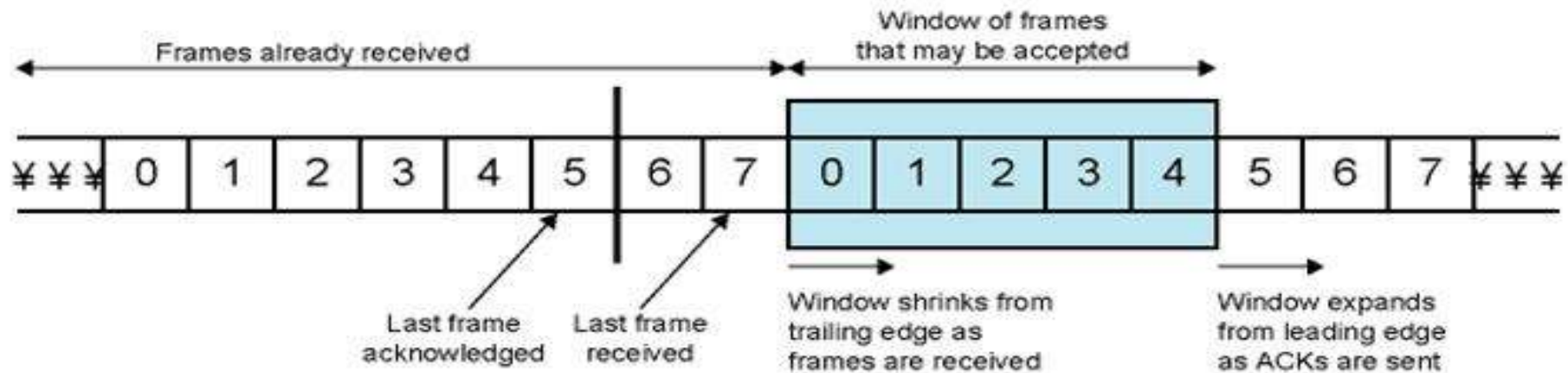
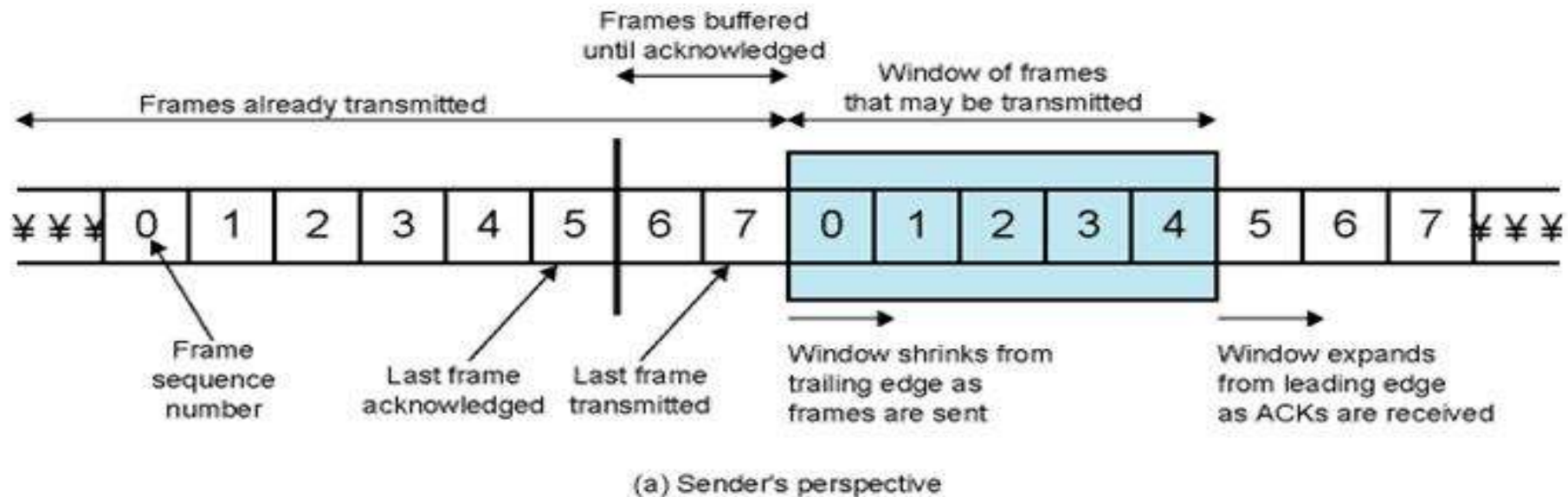
## ■ Sliding Window Protocols

- ◆ A One-Bit Sliding Window Protocol
- ◆ A Protocol Using Go Back N
- ◆ A Protocol Using Selective Repeat

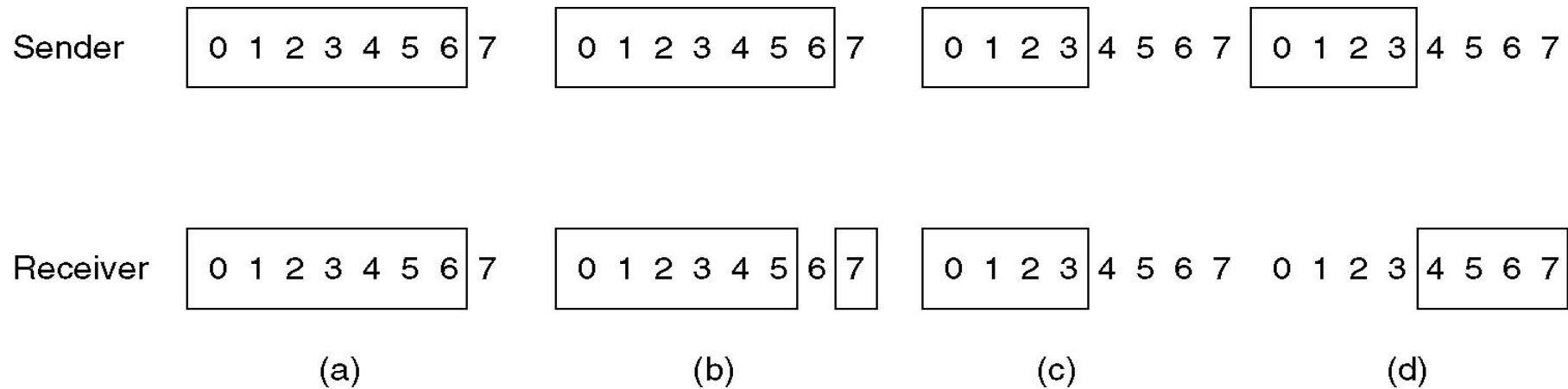
# Selective Repeat (选择重传) ARQ

- Also called *Selective Reject*
- Only rejected frames are retransmitted
- Subsequent frames are accepted by the receiver and buffered
- Minimizes retransmission
- Receiver must maintain large enough buffer
- More complex

# Sliding Window: General View



# Window Size of Selective Repeat



(a) Initial situation with a window size 7

(b) After 7 frames sent and received, but not acknowledged.

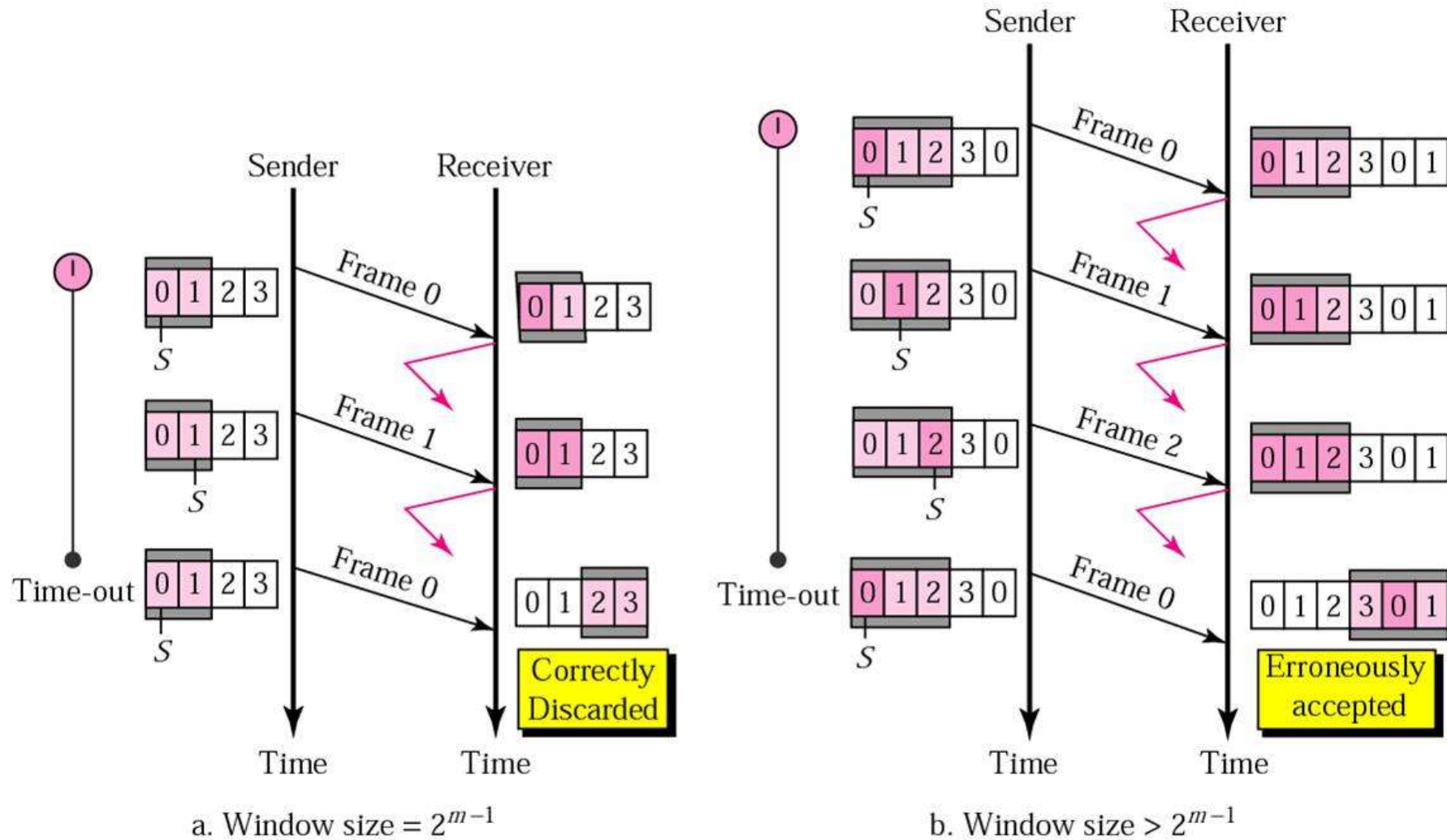
(c) Initial situation with a window size of 4.

(d) After 4 frames sent and received, but not acknowledged

# Maximum Window Size of Selective Repeat

- The maximum window size should be at most half the range of the sequence numbers
  - ◆  $W_T + W_R \leq 2^n$
  - ◆  $W_T \geq W_R$
  - ◆ In practice, usually  $W_T = W_R = 2^{n-1}$
- How many buffers must the receiver have?
  - ◆ The number of buffers needed is equal to the window size, not to the range of sequence numbers

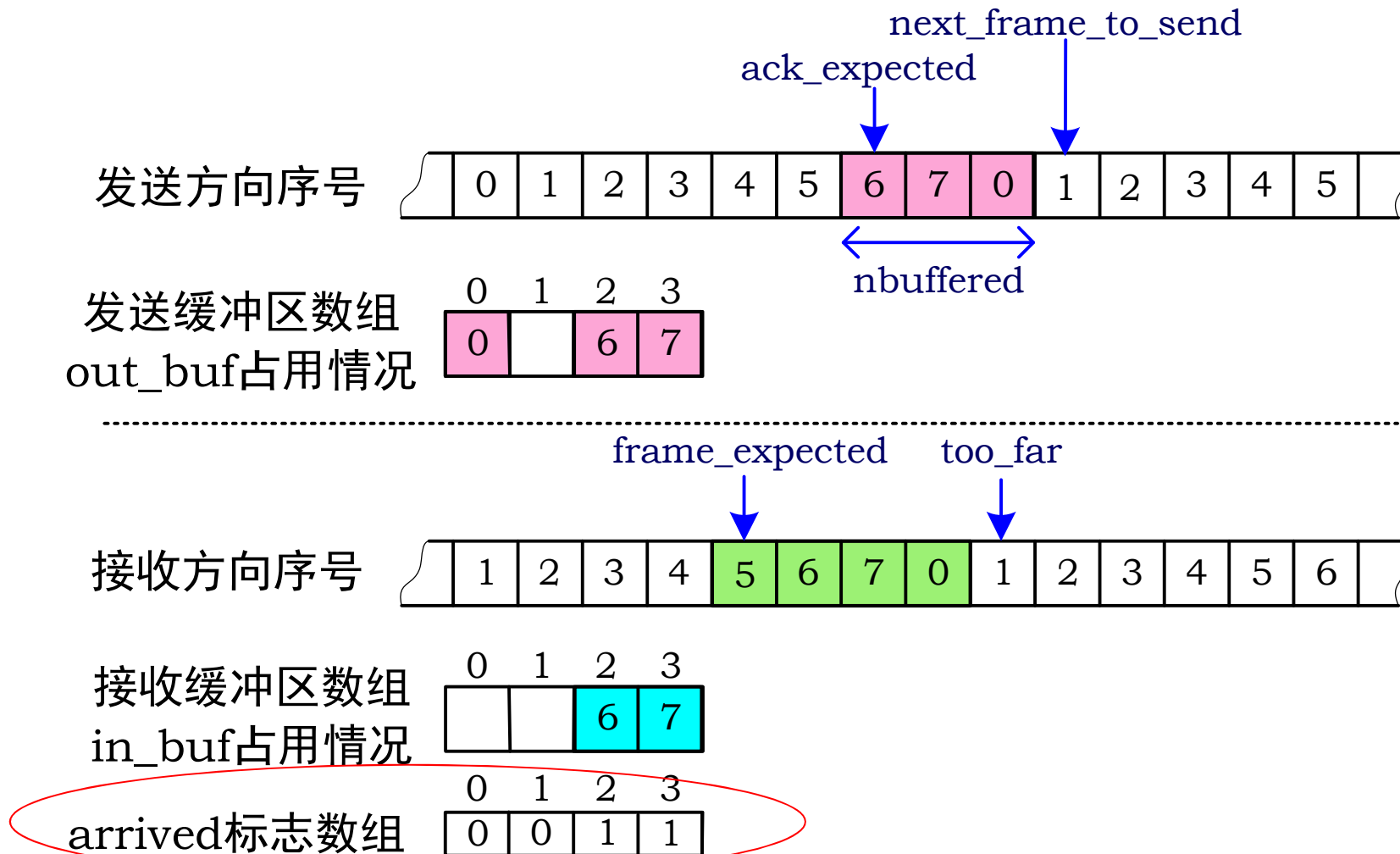
# Maximum of Sending Window Size: Example of 2-bit SN(m=2)





# Protocol 6: Selective Repeat

发送窗口最大为4，接收窗口最大为4



# 对Protocol 5的改进和效率

## ■ 设置ACK定时器

- ◆ 在没有数据要发送时，发送ACK帧
- ◆ 用于回送ACK的定时器操作
  - start\_ack\_timer和stop\_ack\_timer,
- ◆ 数据重传所用的定时器操作
  - start\_timer与stop\_timer
- ◆ ACK定时器时限的设计

## ■ NAK帧

## ■ 协议参数与线路利用率

- ◆ 超时定时器时限的设计（考虑线路往返时间延迟，ACK定时器时限，对方物理层发送排队时延）
- ◆ 滑动窗口，即MAX\_SEQ大小的设置

# ACK Timer

## ■ ACK timer

- ◆ After an in-sequence data frame arrives, an auxiliary timer is started by `start_ack_timer`. If no reverse traffic has presented before this timer expires, a separate ACK frame is sent
- ◆ The timeout associated with the auxiliary timer be appreciably shorter than the timer used for timing out data frames

# NAK

- Whenever the receiver has reason to suspect that an error has occurred, it sends NAK frame back to the sender
  - ◆ NAK frame is a request for retransmission
- There are two cases when the receiver should be suspicious
  - ◆ A **damaged frame** has arrived
  - ◆ A frame **other than the expected** one arrived
- Avoid making multiple requests for retransmission of the same frame
  - ◆ Receiver keeps track of whether a NAK has already been sent for a given frame

/\* Protocol 6 (Selective repeat) accepts frames out of order but passes packets to the network layer in order. Associated with each outstanding frame is a timer. When the timer expires, only that frame is retransmitted, not all the outstanding frames, as in protocol 5. \*/

```
#define MAX_SEQ 7                                /* should be  $2^n - 1$  */
#define NR_BUFS ((MAX_SEQ + 1)/2)
typedef enum {frame_arrival, cksum_err, timeout, network_layer_ready, ack_timeout} event_type;
#include "protocol.h"
boolean no_nak = true;                            /* no nak has been sent yet */
seq_nr oldest_frame = MAX_SEQ + 1;               /* initial value is only for the simulator */

static boolean between(seq_nr a, seq_nr b, seq_nr c)
{
    /* Same as between in protocol 5, but shorter and more obscure. */
    return ((a <= b) && (b < c)) || ((c < a) && (a <= b)) || ((b < c) && (c < a));
}

static void send_frame(frame_kind fk, seq_nr frame_nr, seq_nr frame_expected, packet buffer[])
{
    /* Construct and send a data, ack, or nak frame. */
    frame s;                                       /* scratch variable */

    s.kind = fk;                                  /* kind == data, ack, or nak */
    if (fk == data) s.info = buffer[frame_nr % NR_BUFS];
    s.seq = frame_nr;                             /* only meaningful for data frames */
    s.ack = (frame_expected + MAX_SEQ) % (MAX_SEQ + 1);
    if (fk == nak) no_nak = false;                /* one nak per frame, please */
    to_physical_layer(&s);                       /* transmit the frame */
    if (fk == data) start_timer(frame_nr % NR_BUFS);
    stop_ack_timer();                           /* no need for separate ack frame */
}
```

```

void protocol6(void)
{
    seq_nr ack_expected;           /* lower edge of sender's window */
    seq_nr next_frame_to_send;     /* upper edge of sender's window + 1 */
    seq_nr frame_expected;         /* lower edge of receiver's window */
    seq_nr too_far;                /* upper edge of receiver's window + 1 */
    int i;                         /* index into buffer pool */
    frame r;                       /* scratch variable */
    packet out_buf[NR_BUFS];       /* buffers for the outbound stream */
    packet in_buf[NR_BUFS];        /* buffers for the inbound stream */
    boolean arrived[NR_BUFS];      /* inbound bit map */
    seq_nr nbuffered;              /* how many output buffers currently used */
    event_type event;

    enable_network_layer();         /* initialize */
    ack_expected = 0;              /* next ack expected on the inbound stream */
    next_frame_to_send = 0;        /* number of next outgoing frame */
    frame_expected = 0;
    too_far = NR_BUFS;
    nbuffered = 0;                 /* initially no packets are buffered */
    for (i = 0; i < NR_BUFS; i++) arrived[i] = false;

```



```

while (true) {
    wait_for_event(&event);                /* five possibilities: see event_type above */
    switch(event) {
        case network_layer_ready:          /* accept, save, and transmit a new frame */
            nbuffered = nbuffered + 1;      /* expand the window */
            from_network_layer(&out_buf[next_frame_to_send % NR_BUFS]); /* fetch new packet */
            send_frame(data, next_frame_to_send, frame_expected, out_buf); /* transmit the frame */
            inc(next_frame_to_send);        /* advance upper window edge */
            break;

        case frame_arrival:                /* a data or control frame has arrived */
            from_physical_layer(&r);         /* fetch incoming frame from physical layer */
            if (r.kind == data) {
                /* An undamaged frame has arrived. */
                if ((r.seq != frame_expected) && no_nak)
                    send_frame(nak, 0, frame_expected, out_buf); else start_ack_timer();
                if (between(frame_expected, r.seq, too_far) && (arrived[r.seq % NR_BUFS] == false)) {
                    /* Frames may be accepted in any order. */
                    arrived[r.seq % NR_BUFS] = true;    /* mark buffer as full */
                    in_buf[r.seq % NR_BUFS] = r.info;  /* insert data into buffer */
                    while (arrived[frame_expected % NR_BUFS]) {
                        /* Pass frames and advance window. */
                        to_network_layer(&in_buf[frame_expected % NR_BUFS]);
                        no_nak = true;
                        arrived[frame_expected % NR_BUFS] = false;
                        inc(frame_expected); /* advance lower edge of receiver's window */
                        inc(too_far);       /* advance upper edge of receiver's window */
                        start_ack_timer();   /* to see if a separate ack is needed */
                    }
                }
            }
    }
}

```

```

if((r.kind==nak) && between(ack_expected,(r.ack+1)%(MAX_SEQ+1),next_frame_to_send))
    send_frame(data, (r.ack+1) % (MAX_SEQ + 1), frame_expected, out_buf);

while (between(ack_expected, r.ack, next_frame_to_send)) {
    nbuffered = nbuffered - 1;          /* handle piggybacked ack */
    stop_timer(ack_expected % NR_BUFS); /* frame arrived intact */
    inc(ack_expected);                  /* advance lower edge of sender's window */
}
break;

case cksum_err:
    if (no_nak) send_frame(nak, 0, frame_expected, out_buf); /* damaged frame */
    break;

case timeout:
    send_frame(data, oldest_frame, frame_expected, out_buf); /* we timed out */
    break;

case ack_timeout:
    send_frame(ack,0,frame_expected, out_buf); /* ack timer expired; send ack */
}

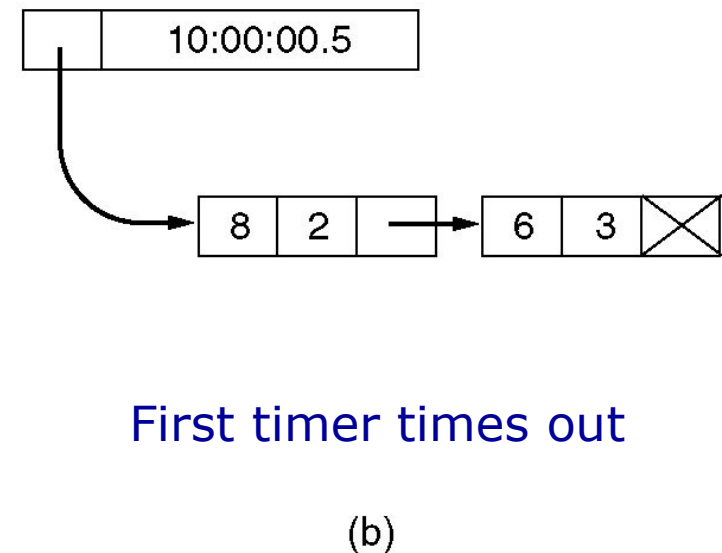
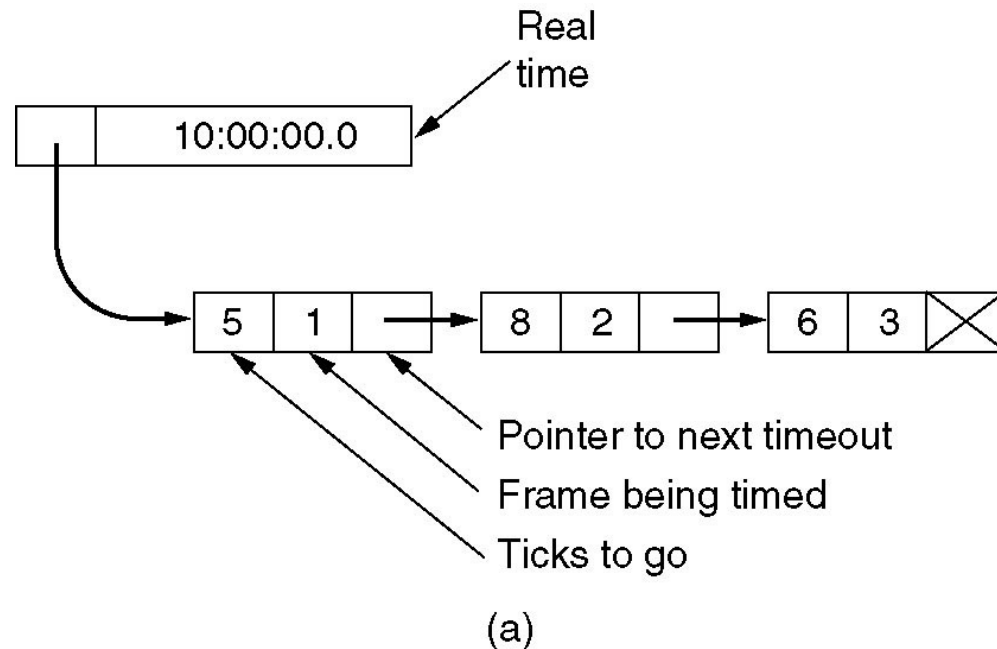
if (nbuffered < NR_BUFS) enable_network_layer(); else disable_network_layer();
}
}

```



# Timer Queue in Software

Simulation of multiple timers in software.



- Clock ticks once every 100ms
- 3 pending timeout at 10:00:00.5, 10:00:01.3, 10:00:01.9 respectively

# Summary of Flow Control Protocols

- Protocol 1: ideal channel, no flow control
- Protocol 2, error-free channel, stop and wait
  - ◆ ACK from receiver
- Protocol 3, error-free but frame may be lost
  - ◆ Using Timer and 1-bit Sequence Number
- Protocol 4, 1-bit sliding window (**Stop-and-Wait**)
  - ◆ full duplex, Piggybacking
- Protocol 5, **Go Back N** sliding window
  - ◆ Sender window size>1, Receiver window size=1
  - ◆ For n-bit SN,  $W_T \leq 2^n - 1$
- Protocol 6, **Selective Repeat** sliding window
  - ◆ Sender window size>1, Receiver window size>1
  - ◆ For n-bit SN, usually  $W_T = W_R = 2^{n-1}$
  - ◆ ACK Timer and NAK

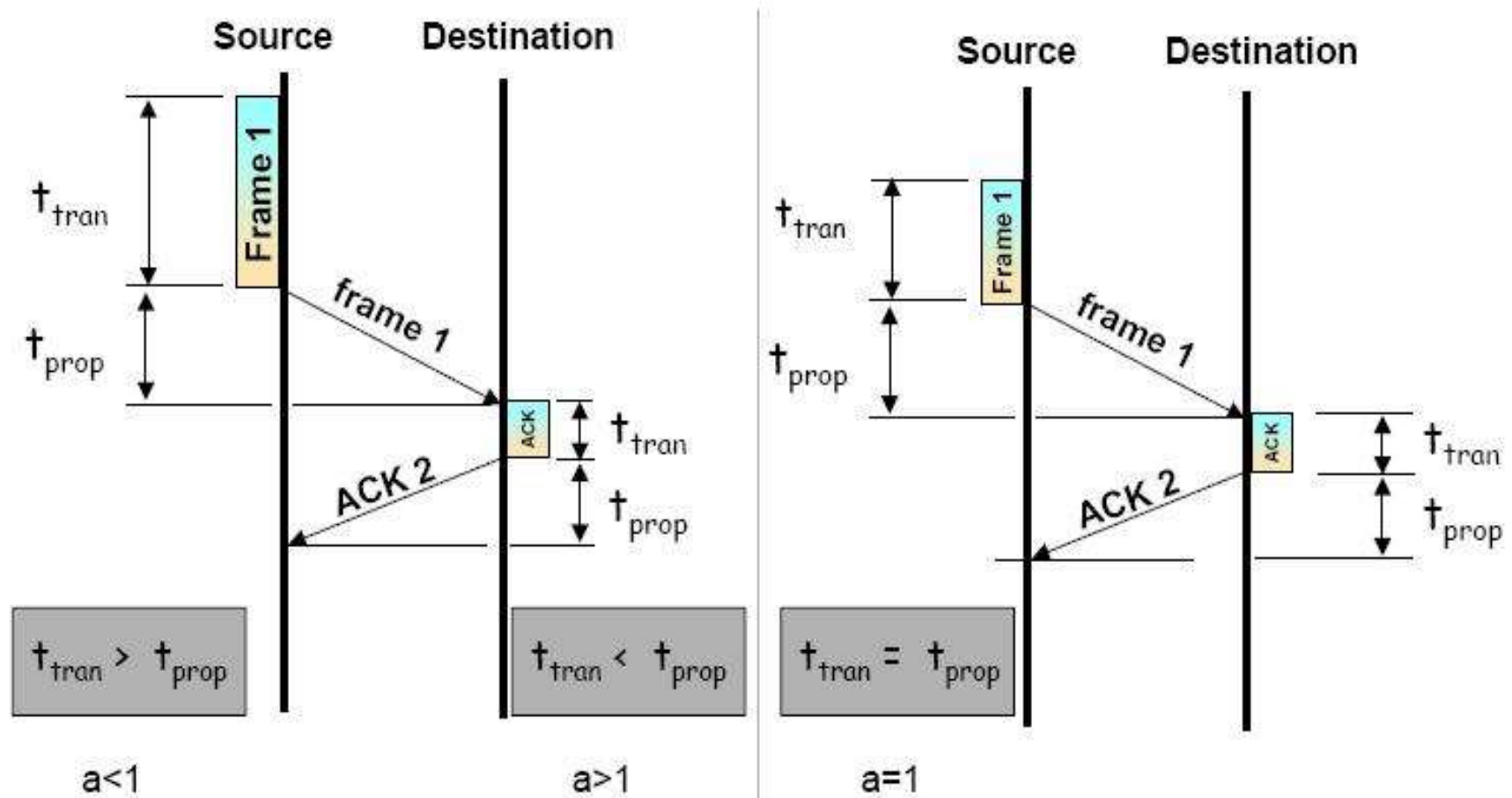
# 总结

|      | 乌托邦协议 | 无错信道停等协议 | 停等协议      | 1比特滑动窗口协议 | GoBackN   | 选择重传 |
|------|-------|----------|-----------|-----------|-----------|------|
| 信道条件 | 理想信道  |          |           |           |           |      |
| 协议功能 |       | 流量控制     | 差错控制、流量控制 | 差错控制、流量控制 | 差错控制、流量控制 |      |
| 通信模式 | 单工    | 单工       | 单工        | 双工        | 双工        | 双工   |
| 确认方式 |       |          |           |           |           |      |
| 序号   | 无     | 无        |           |           |           |      |
| 窗口尺寸 |       |          |           |           |           |      |
| 信道效率 |       |          |           |           |           |      |
| 帧种类  |       |          |           |           |           |      |
| 典型协议 |       |          |           |           |           |      |
| 适用场景 |       |          |           |           |           |      |

# Outline

- Position, Functions & Services of Data Link Layer
- Design Issues
  - ◆ Framing
  - ◆ Error Control
  - ◆ Flow Control
    - Basic flow control protocols
    - Performance of sliding window protocols
- Example Protocols
  - ◆ HDLC
  - ◆ PPP

# Model of Frame Transmission



$$a = t_{\text{prop}} / t_{\text{tran}}$$

# Transfer Time & Round Trip Time

## ■ Frame transfer time (txfr)

- ◆ Time for receiver to receive the entire frame

$$t_{xfr} = t_{tran} + t_{prop} + t_{queue/switching}$$

$$t_{tran} = \frac{\text{number of frame bits [bits]}}{\text{data rate [bps]}}$$

$$t_{prop} = \frac{\text{link length [m]}}{v_{prop} \text{ [m/s]}}$$

## ■ Round Trip Time (RTT)

- ◆ May not be twice the one-way delay if data frame and ACK frame travel over different links

# Performance of Error-free Stop and Wait (1)

- The total time to send the data frame

$$T_F = t_{\text{tran}} + t_{\text{prop}} + t_{\text{proc}} + t_{\text{ack}} + t_{\text{prop}} + t_{\text{proc}}$$

- ◆  $T_F$  : The time to send one frame and receive an acknowledgement
- ◆  $t_{\text{tran}}$  : time to transmit a frame
- ◆  $t_{\text{prop}}$  : propagation time from Source to Destination
- ◆  $t_{\text{proc}}$  : processing time
- ◆  $t_{\text{ack}}$  : time to transmit an acknowledgement.

# Performance of Error-free Stop and Wait (2)

- Assume  $t_{\text{proc}}$  and  $t_{\text{ack}}$  are relatively negligible, then

$$T_F = t_{\text{tran}} + 2t_{\text{prop}}$$

- Utilization

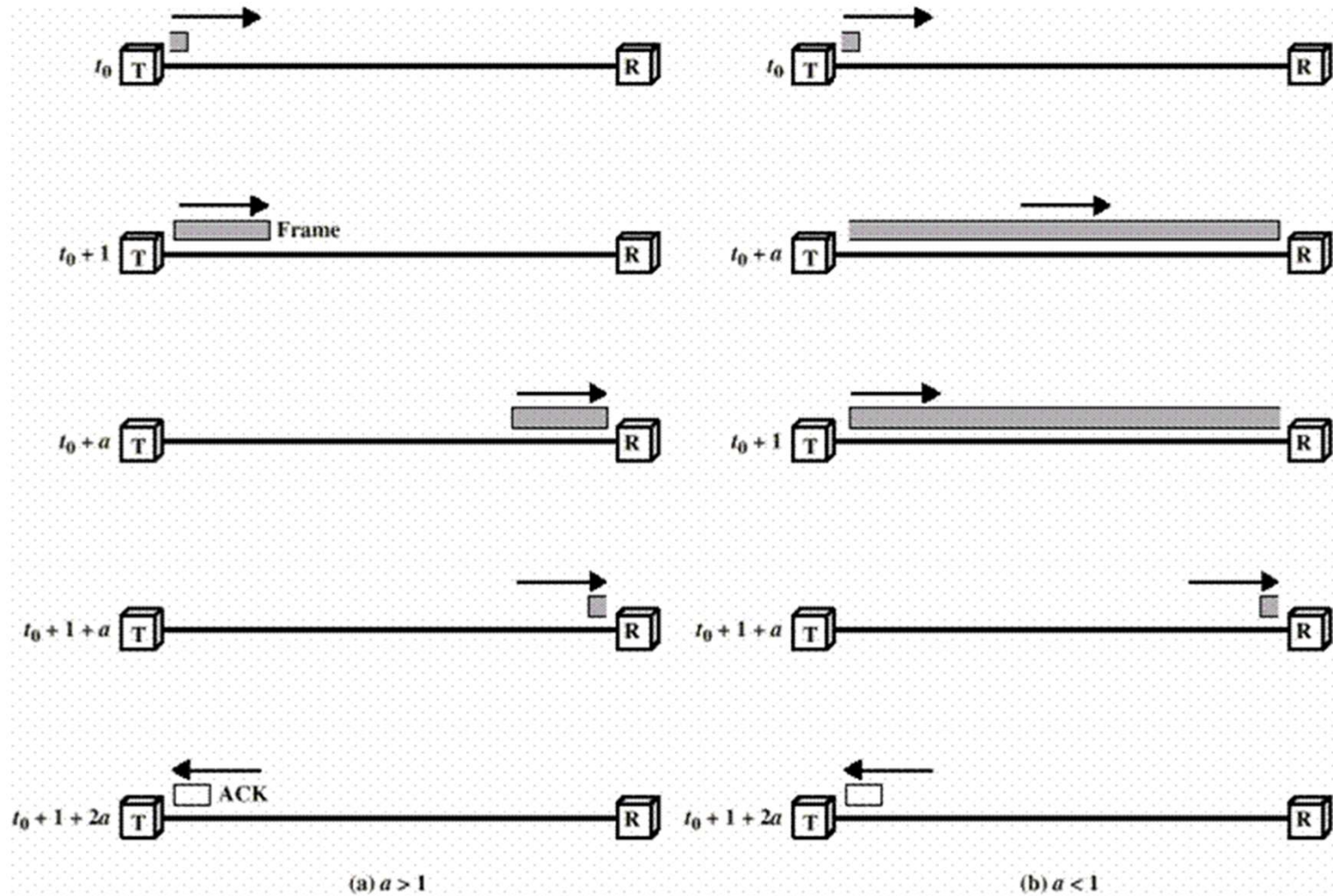
$$U = \frac{t_{\text{tran}}}{2t_{\text{prop}} + t_{\text{tran}}} = \frac{1}{1 + 2a}$$

$$\text{where } a = \frac{\text{propagation time}}{\text{transmission time}}$$



# Stop and Wait: Performance

Normalize the transmission time to **1**, and the propagation time is ***a***



# Performance of Stop and Wait: example

## ■ Stop-and-wait without error

□ Satellite Link: Propagation Delay  $t_{\text{prop}} = 270 \text{ ms}$

Frame Size = 4000 bits = 500 bytes

Data rate = 56 kbps  $\Rightarrow t_{\text{tran}} = 4/56 = 71 \text{ ms}$

$\alpha = t_{\text{prop}}/t_{\text{tran}} = 270/71 = 3.8$

$U = 1/(2\alpha+1) = 0.12$

□ Short Link: 1 km = 5  $\mu\text{s}$ ,

Rate=10 Mbps,

Frame=500 bytes  $\Rightarrow t_{\text{tran}} = 4\text{k}/10\text{M} = 400 \mu\text{s}$

$\alpha = t_{\text{prop}}/t_{\text{tran}} = 5/400 = 0.012 \Rightarrow U = 1/(2\alpha+1) = 0.98$

# Stop and Wait: Line Utilization

- Channel capacity is  $b$  bits/sec
- Frame size  $l$  bits
- Round-trip propagation time  $R$  sec
- Line Utilization

$$\frac{\frac{l}{b}}{\frac{l}{b} + R} = \frac{l}{l + bR}$$

◆ If  $l < bR$ , the efficiency will be less than 50 percent

# Performance of Stop and Wait: More

## ■ Stop-and-wait with error

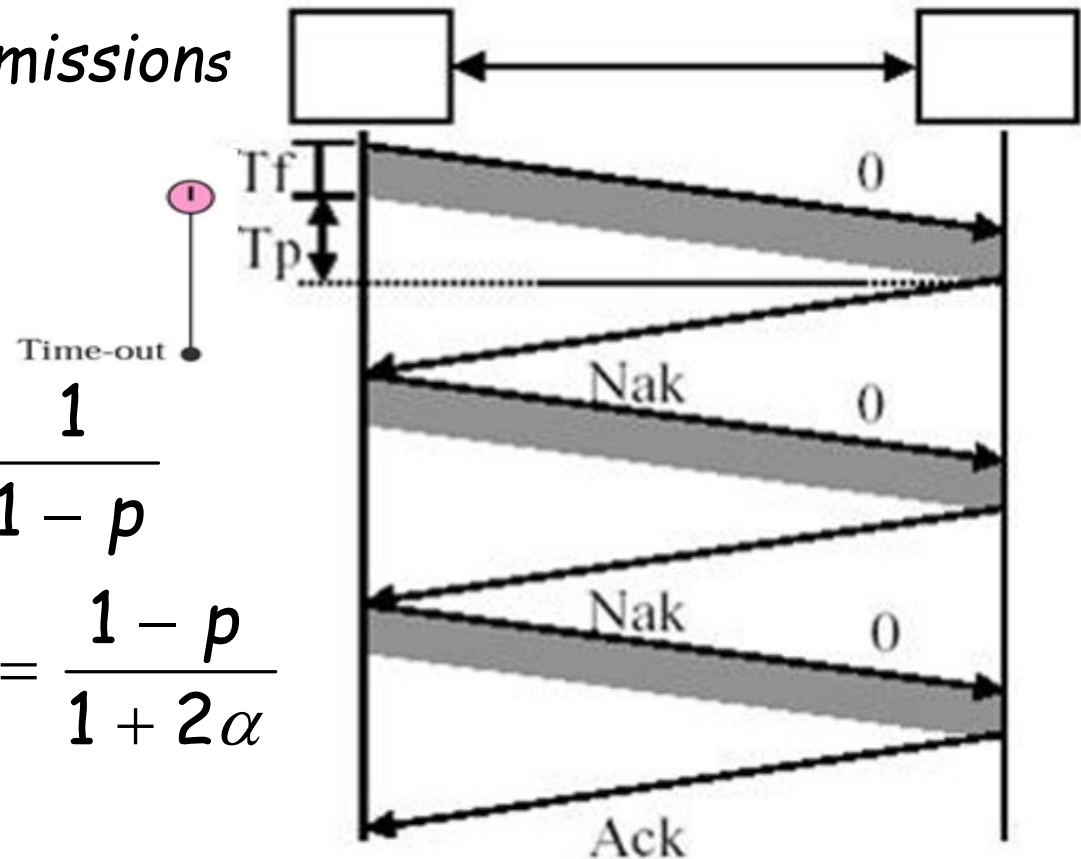
◆  $p$  = Probability of frame error

◆  $N_r$  = number of transmissions

$$\alpha = \frac{T_{prop}}{T_{tran}}$$

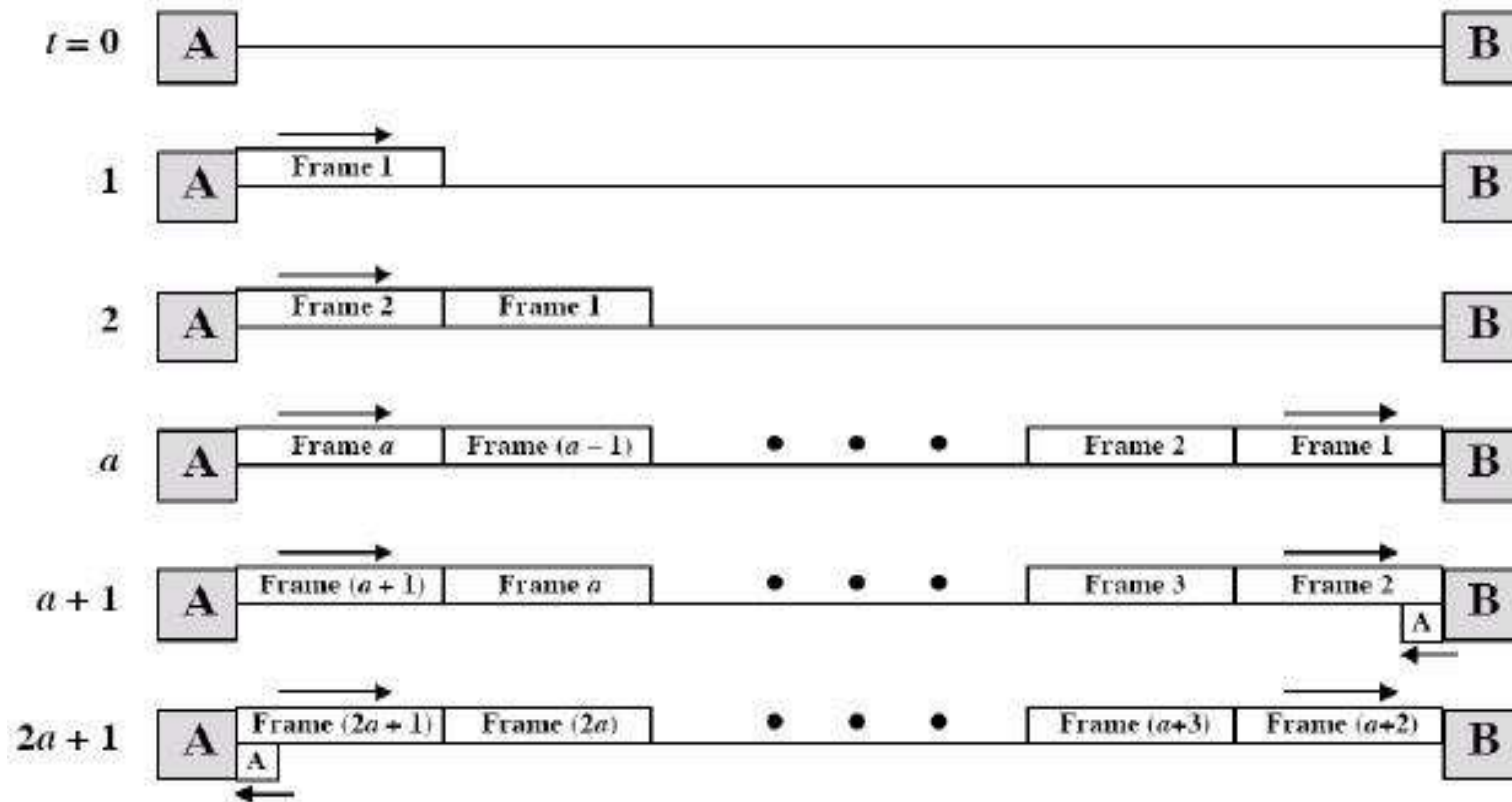
$$N_r = \sum_{i=1}^{\infty} ip^{i-1}(1-p) = \frac{1}{1-p}$$

$$U = \frac{T_{tran}}{N_r(T_{tran} + 2T_{prop})} = \frac{1-p}{1+2\alpha}$$



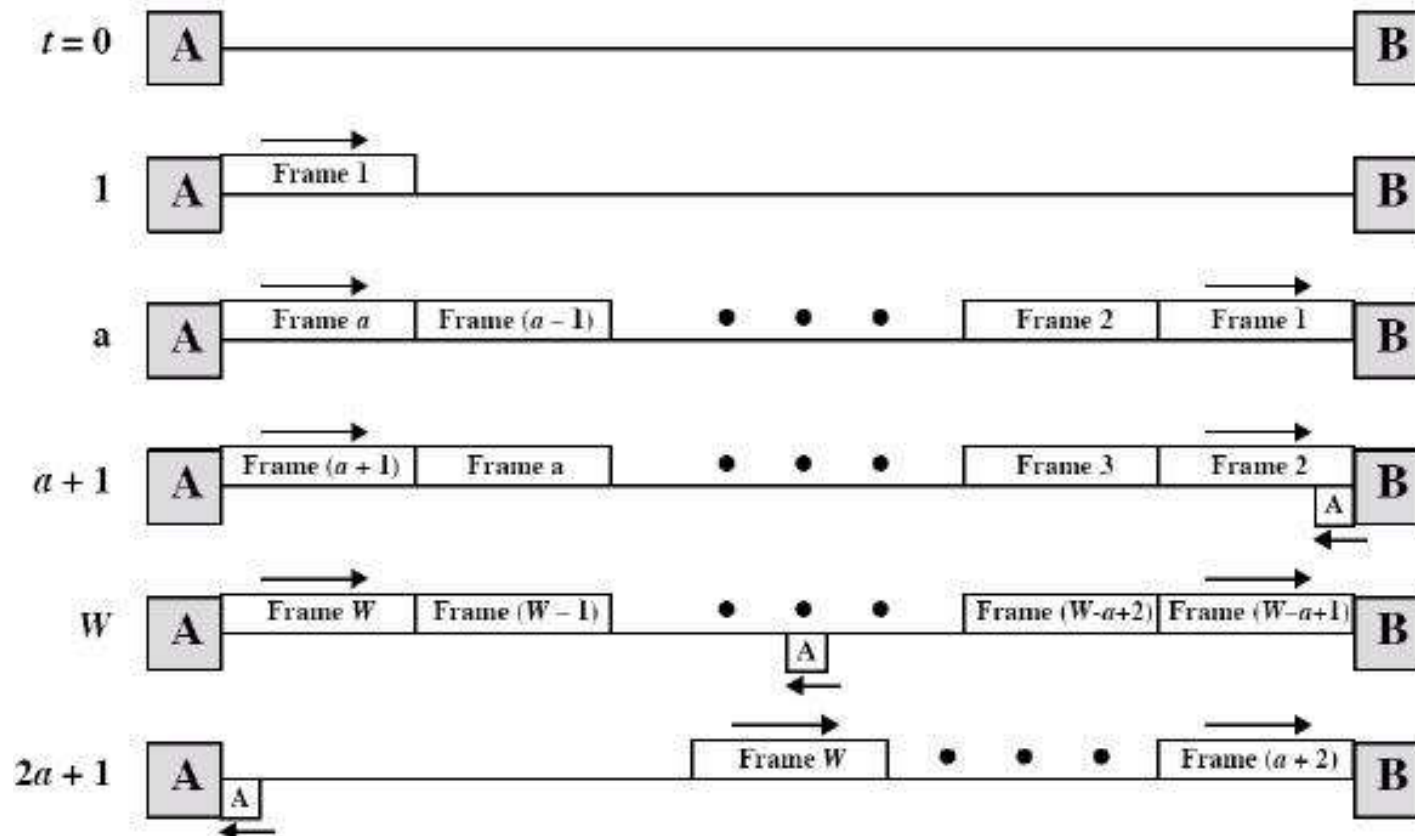
# Performance of Error-Free Sliding-Window Flow Control (1)

- Normalize the transmission time to 1, and the propagation time is  $a$ 
  - ◆ Case 1:  $W \geq 2a + 1$ , where  $W$  is window size



# Performance of Error-Free Sliding-Window Flow Control (2)

## Case 2: $W < 2a + 1$



## Performance of Error-Free Sliding-Window Flow Control (3)

- Therefore we can express the utilization

$$U = \begin{cases} 1 & W \geq 2a+1 \\ \frac{W}{2a+1} & W < 2a+1 \end{cases}$$

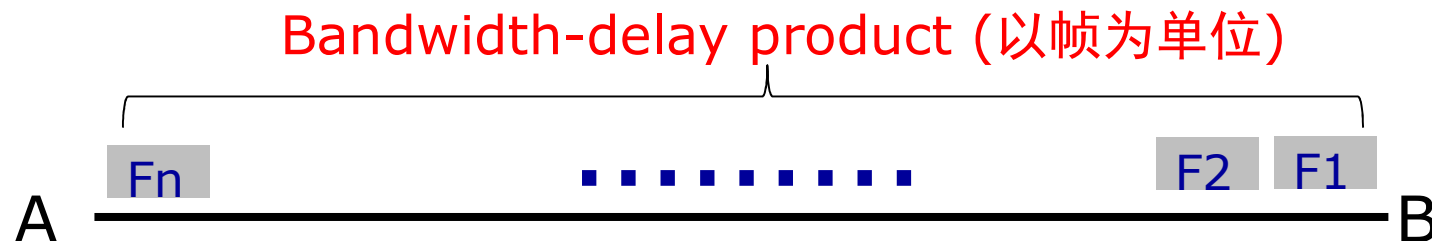
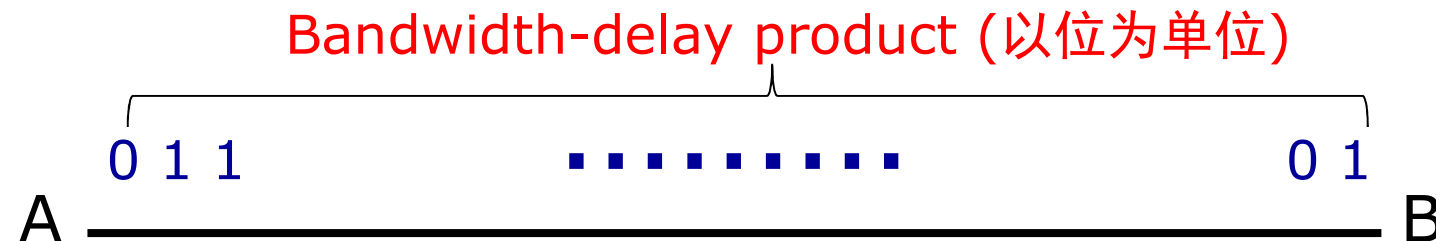
- Another formula in textbook

$$\text{link utilization} \leq \frac{w}{1 + 2BD}$$

其实是一样的

# Bandwidth-delay product(带宽时延积)

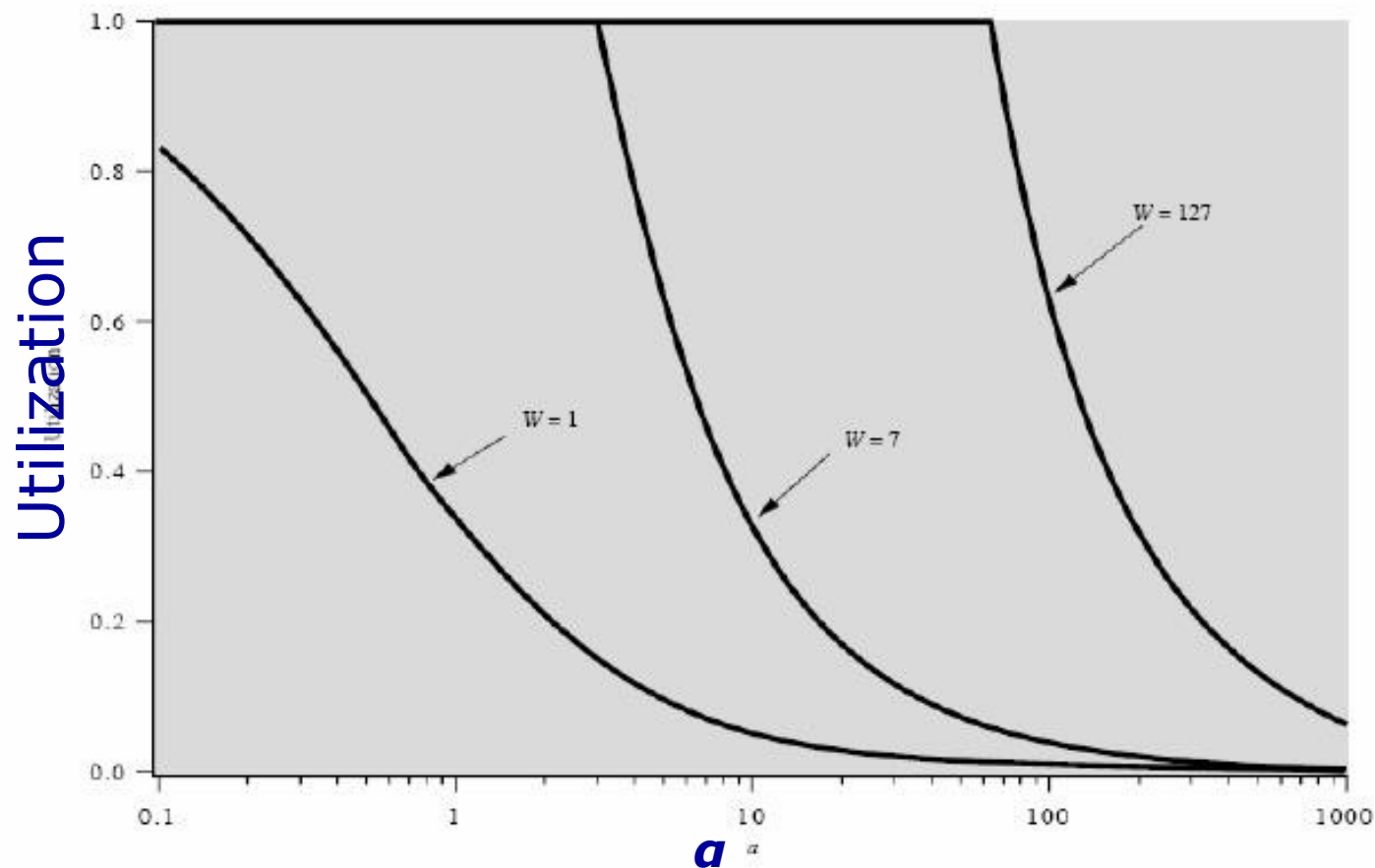
- Bandwidth: maximum bit rate(bps)
- Delay: propagation delay (传播时延)
- Bandwidth-delay product: 在单程传播时延的时间内, 信道可以容纳多少位数据





# Performance of Error-Free Sliding-Window Flow Control (4)

- Sliding Window Utilization as a function of  $\alpha$



# Performance of Error-Free Sliding-Window Flow Control: Piggybacking

- Piggybacking: (捎带确认/搭载ACK)
- The reverse traffic (data frame) transfer time must be considered.

◆  $U = W / (2a + 2) = W / 2(a + 1)$

# Review of Flow Control

## ■ Protocols in practice

- ◆ Protocol 4, 1-bit sliding window
  - $W_T = W_R = 1$
- ◆ Protocol 5, Go Back N sliding window
  - Sender window size  $> 1$ , Receiver window size  $= 1$
  - For n-bit SN,  $W_T \leq 2^n - 1$
- ◆ Protocol 6, Selective Repeat sliding window
  - Sender window size  $> 1$ , Receiver window size  $> 1$
  - For n-bit SN,  $W_T = W_R \leq 2^{n-1}$

## ■ Performance of the protocols

- ◆ Stop-and-wait
  - Error-free channel:  $U = 1/(1+2a)$
  - With Errors:  $U = (1-p)/(1+2a)$
- ◆ Go-back N and Selective Reject, Error-free
- ◆ Piggybacking:  $U = W/2(a+1)$

$$U = \begin{cases} 1 & W \geq 2a + 1 \\ \frac{W}{2a + 1} & W < 2a + 1 \end{cases}$$

# Outline

- Position, Functions & Services of Data Link Layer
- Design Issues
  - ◆ Framing
  - ◆ Error Control
  - ◆ Flow Control
- Example Protocols
  - ◆ HDLC
  - ◆ SLIP/PPP

# Data Link Types

## ■ Point to point links(点到点链路)

- ◆ One sender, one receiver
- ◆ Address may not be necessary
- ◆ Reliable communication
- ◆ Sample protocols: HDLC, PPP...

## ■ Broadcast links(广播链路)

- ◆ Many senders and potential receivers.
- ◆ Accessing a shared medium
- ◆ Addressing is necessary
- ◆ Sample protocols: CSMA/CD, CSMA/CA.....

# Example of Point-to-point Data Link Protocols

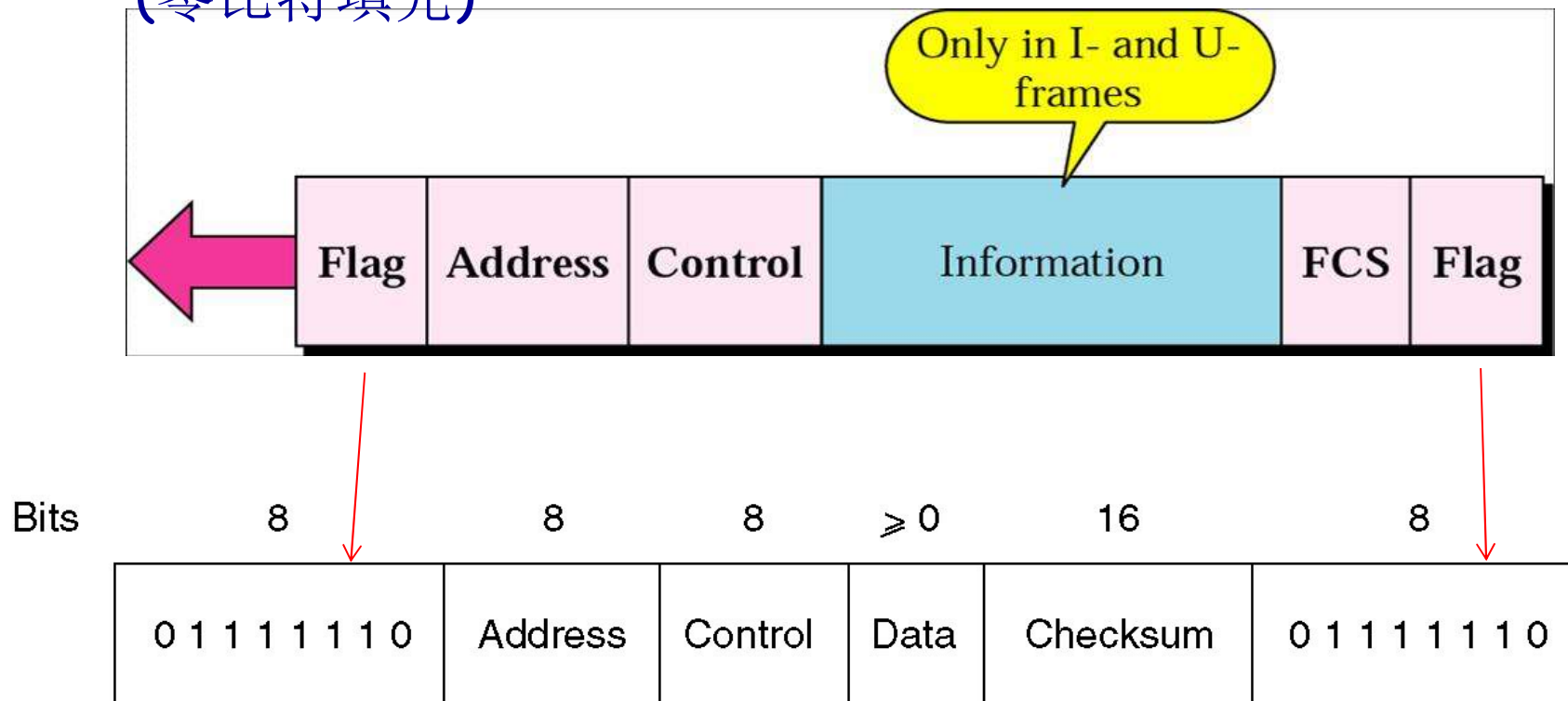
- HDLC (High-Level Data Link Control)  
(高级数据链路控制)
- The Data Link Layer in the Internet
  - ◆ SLIP (Serial Line IP)
  - ◆ PPP (Point-to-Point Protocol)

# HDLC: History

- IBM SDLC
  - ◆ Synchronous Data Link Control
- ANSI ADCCP
  - ◆ Advanced Data Communication Control Procedure
- ISO HDLC
  - ◆ High-level Data Link Control
- CCITT(ITU-T) LAPB
  - ◆ LAP (Link Access Procedure)
  - ◆ LAPB (Link Access Procedure - Balanced mode)
- LAPD (ISDN D-Channel)
- LAN LLC sublayer

# HDLC: Frame Structure

- Synchronous transmission 同步传输
- Bit-oriented, bit stuffing for transparent transmission (零比特填充)

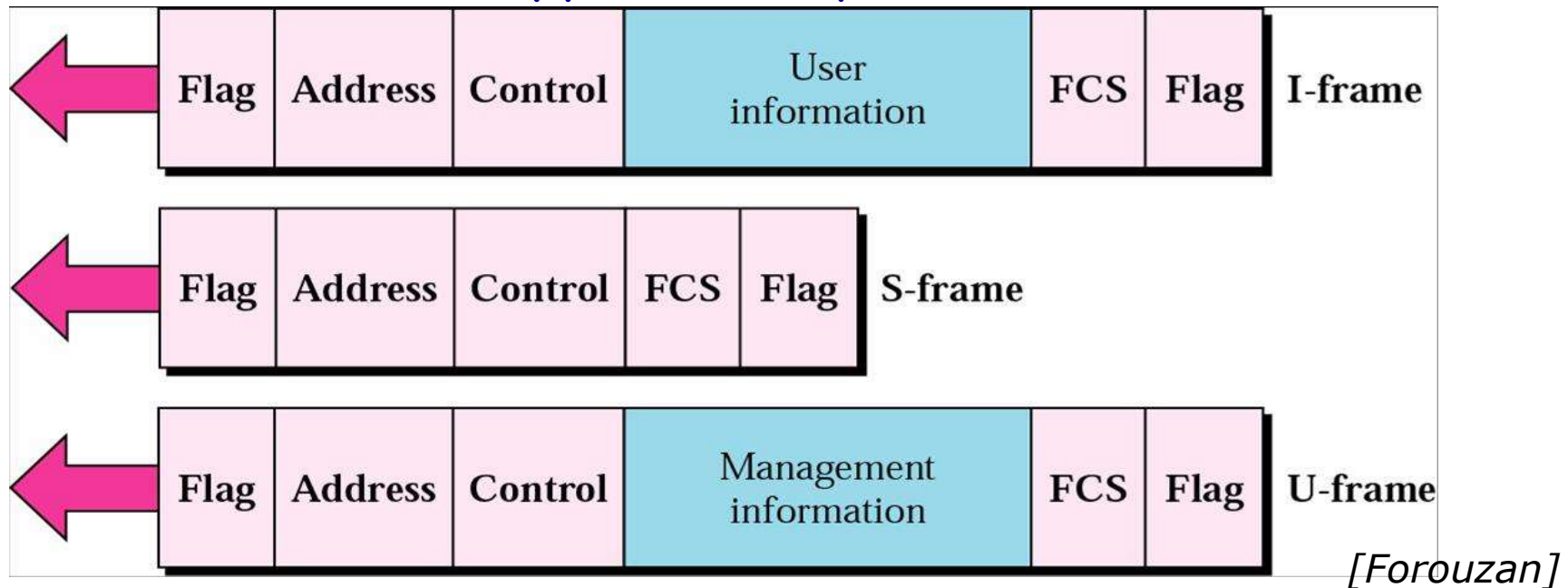


[Forouzan]

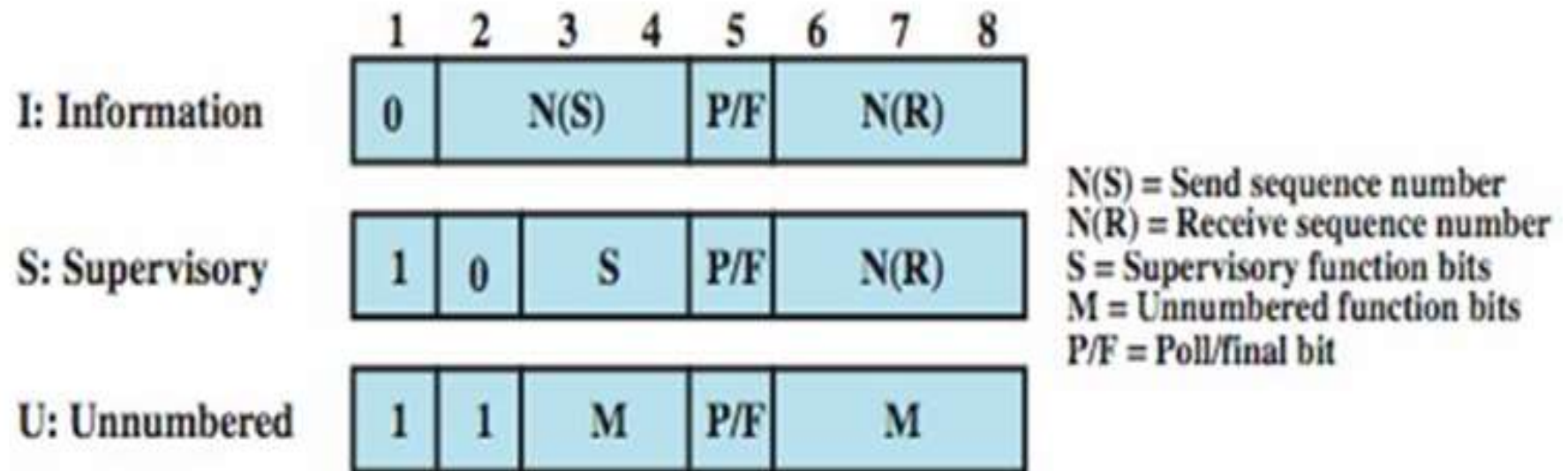


# HDLC: Frame Types

- Information: data to be transmitted to next layer up
  - ◆ Flow and error control piggybacked on information frames 信息帧
- Supervisory: ACK or NAK when piggyback not used 监督帧
- Unnumbered: supplementary link control 无编号帧



# HDLC: Control Fields



# Example of Point-to-point Data Link Protocols

- HDLC (High-Level Data Link Control)
  - ◆ 帧首尾标志: “01111110”, 零比特填充
  - ◆ CRC校验
  - ◆ 流量控制和差错控制:
    - GoBackN ARQ和选择重传ARQ
- The Data Link Layer in the Internet
  - ◆ SLIP (Serial Line IP)
  - ◆ PPP (Point-to-Point Protocol)

# SLIP – Serial Line Internet Protocol(串行线IP)

- Used to connect to a remote server
- RFC1055
- Framing: Byte Stuffing 字节填充
  - Use 0xC0 as a flag char
  - Use 0xDB 0xDC to replace 0xC0
  - Use 0xDB 0xDB to replace 0xDB
- Problems
  - No error control
  - Only support IP
  - Don't assign IP address dynamically
  - No authentication

# Example of Point-to-point Data Link Protocols

- HDLC (High-Level Data Link Control)
- The Data Link Layer in the Internet
  - ◆ SLIP (Serial Line IP)
  - ◆ PPP (Point-to-Point Protocol 点对点协议)

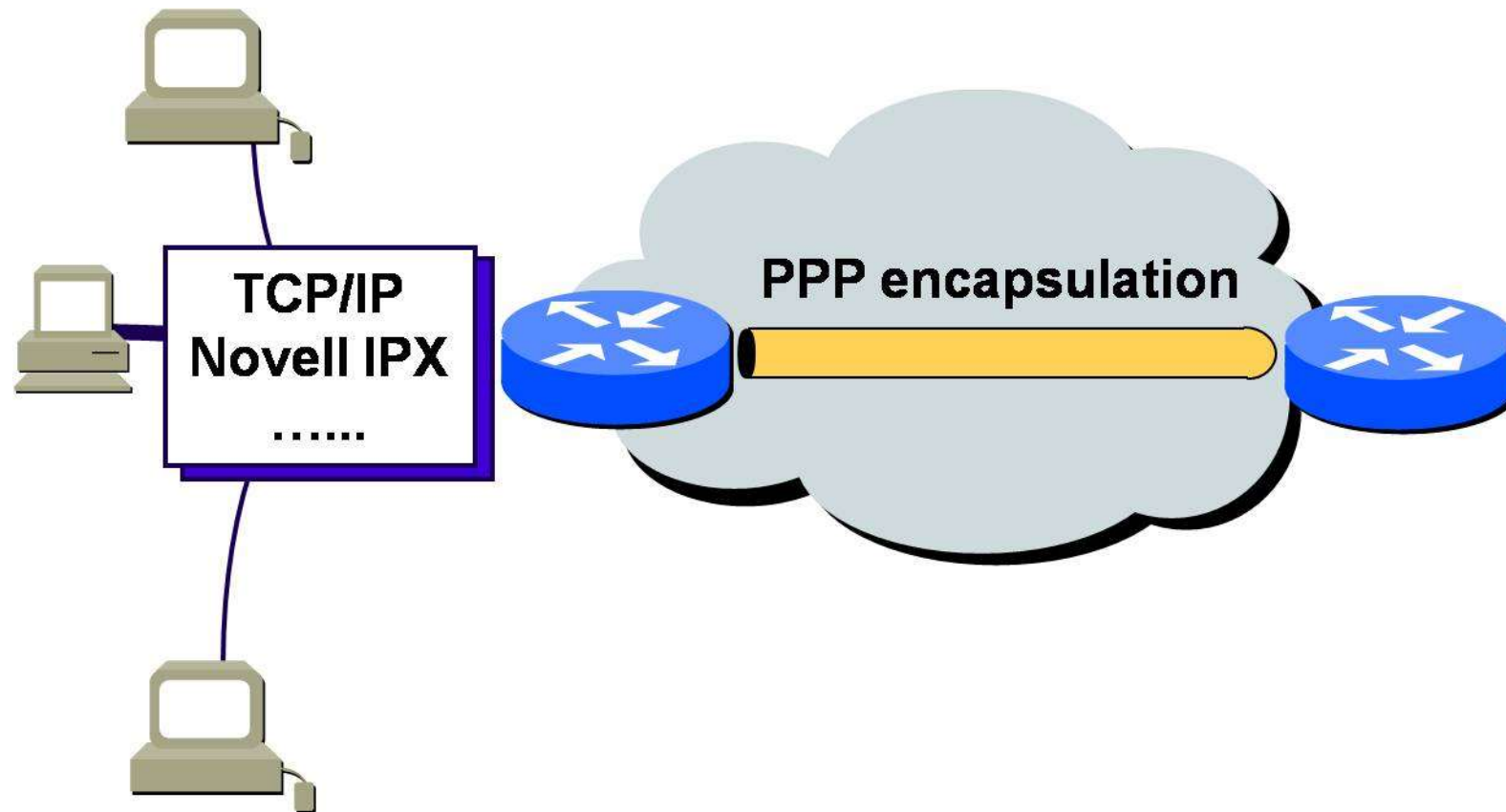
# PPP vs. HDLC

- Point-point link vs. Multiple configurations
- Byte oriented vs. Bit oriented
  - ◆ Byte stuffing vs bit stuffing
- Connectionless unacknowledged service vs. Reliable connection-oriented service

# PPP compared with HDLC

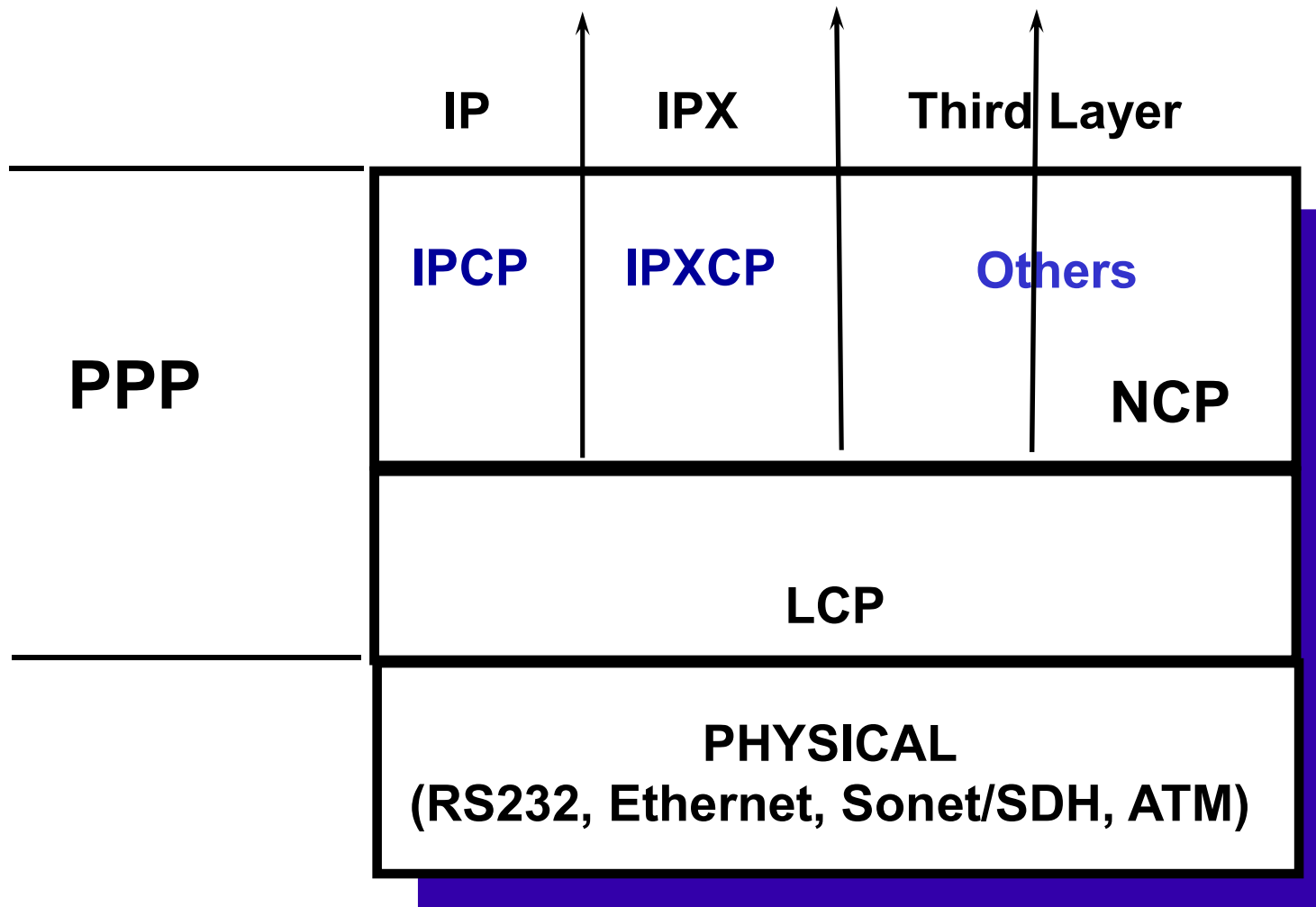
- **Simpler**
- **What are enhanced**
  - ◆ Working with numerous network layer protocols
  - ◆ Negotiation of connections
  - ◆ User authentication
  - ◆ Negotiation of data compression
  - ◆ Physical layer can be asynchronous or synchronous
- **What are removed**
  - ◆ Error correction
  - ◆ Flow control
  - ◆ Sequence number

# PPP: Multiple network protocols





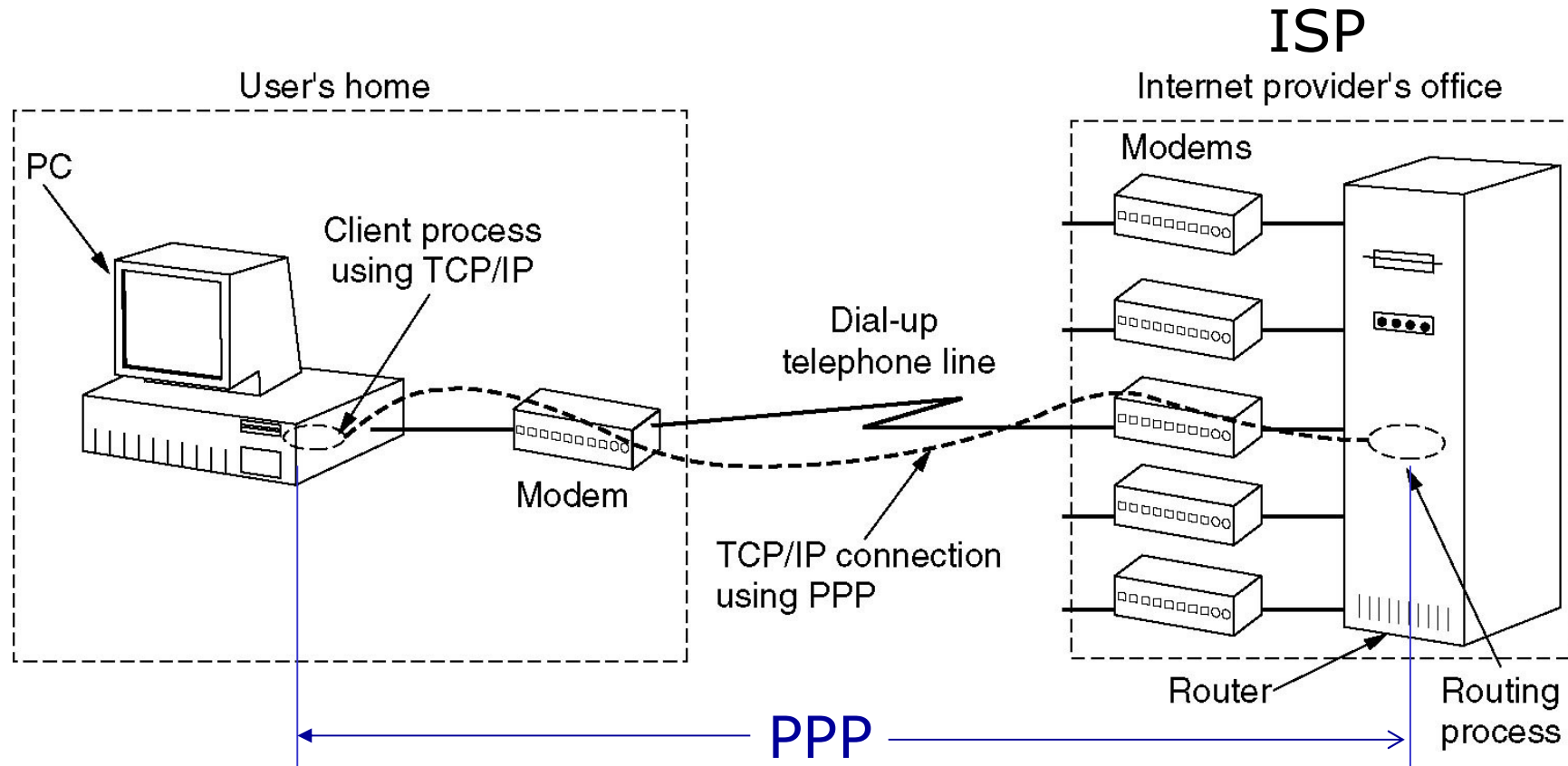
# PPP: Sub-layers(1)



## PPP: Sub Layers (2)

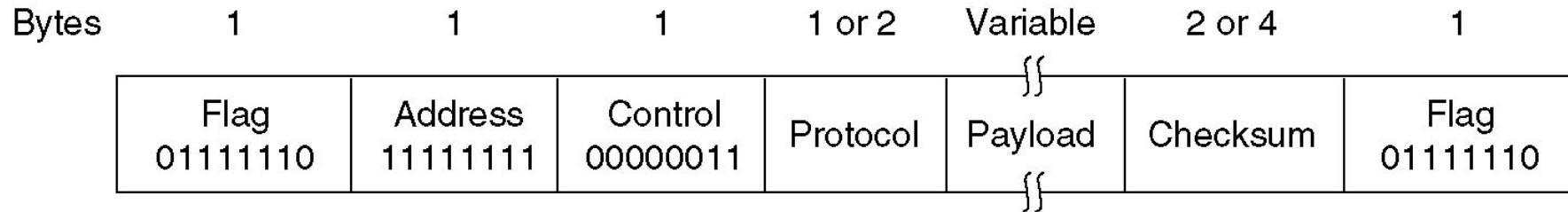
- Protocol specifications:
  - ◆ RFC1661,1662,1663
- Layer 1
  - ◆ Synchronous circuits (bit-oriented encodings, PPP over SONET)
  - ◆ Asynchronous circuits (byte-oriented encodings)
  - ◆ PPPoE (PPP over Ethernet, ADSL)
- LCP: Link Control Protocol
  - ◆ Error detection (CRC)
  - ◆ Permits authentication
- NCP: Network Control Protocol
  - ◆ negotiates network-layer options, such as IP address

# PPP used in dial-up access to Internet



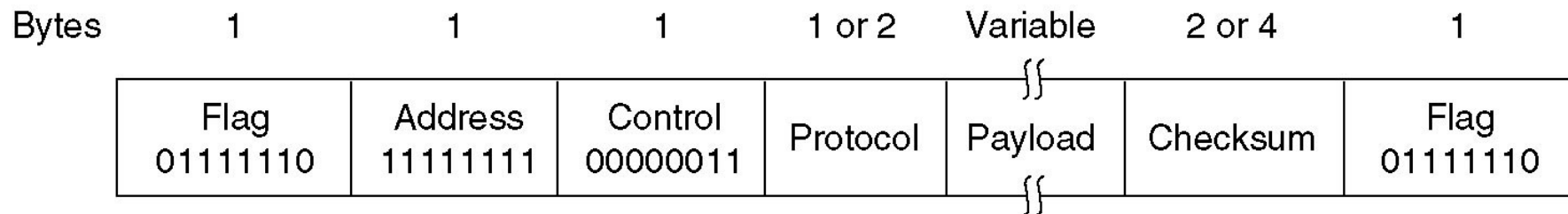
A home personal computer as an Internet host

# PPP for PPPoE: Frame Format (1)



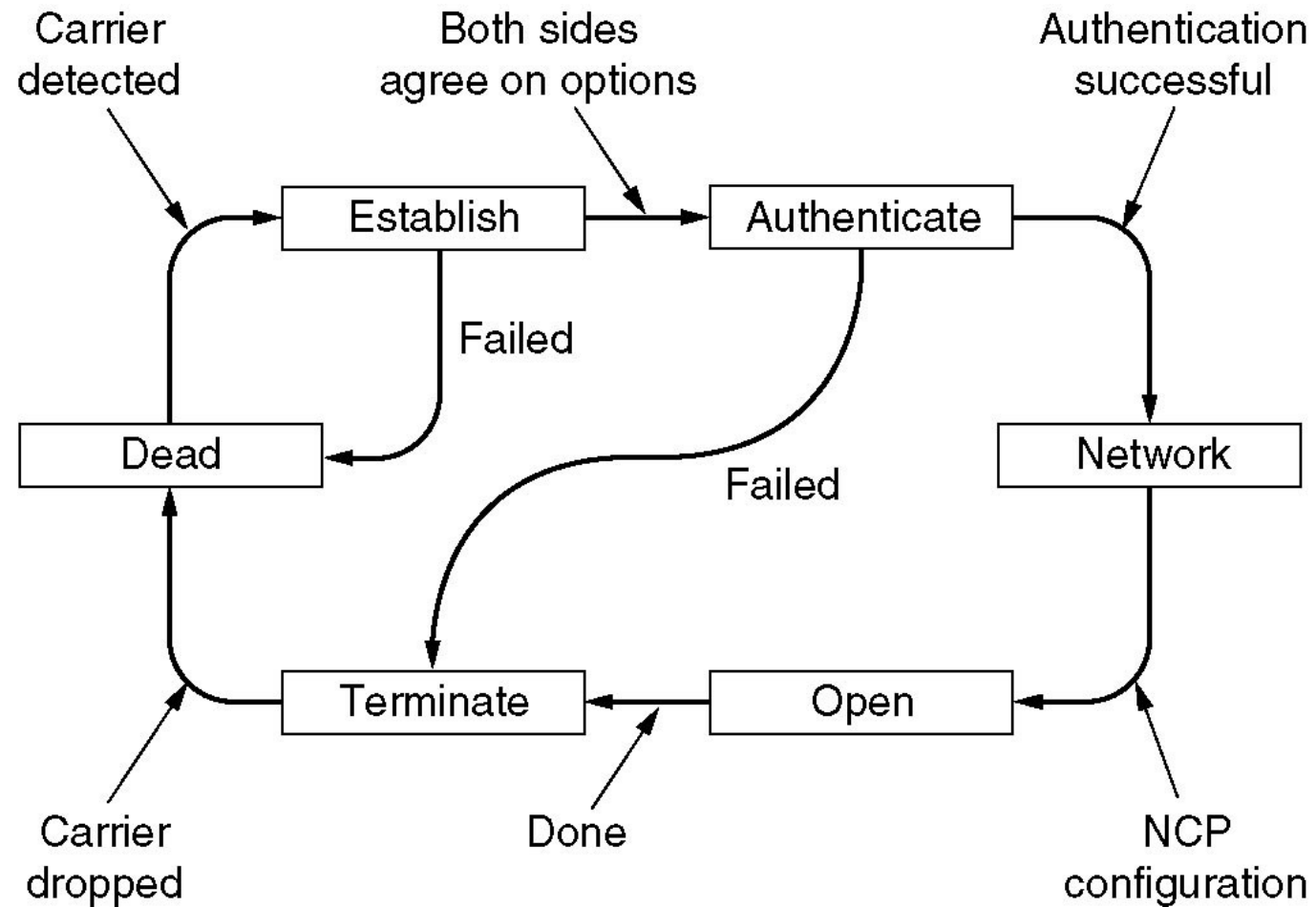
- Unnumbered mode operation 无序号
- Byte stuffing 字节填充
  - ◆ 0x7D as escape byte
  - ◆ 0x7E → 0x7D 0x5E
  - ◆ 0x7D → 0x7D 0x5D
  - ◆ Adding 0x7D before an ASCII control character ( $\leq 0x20$ )
    - 0x03 → 0x7D 0x03

# PPP for PPPoE: Frame Format(2)

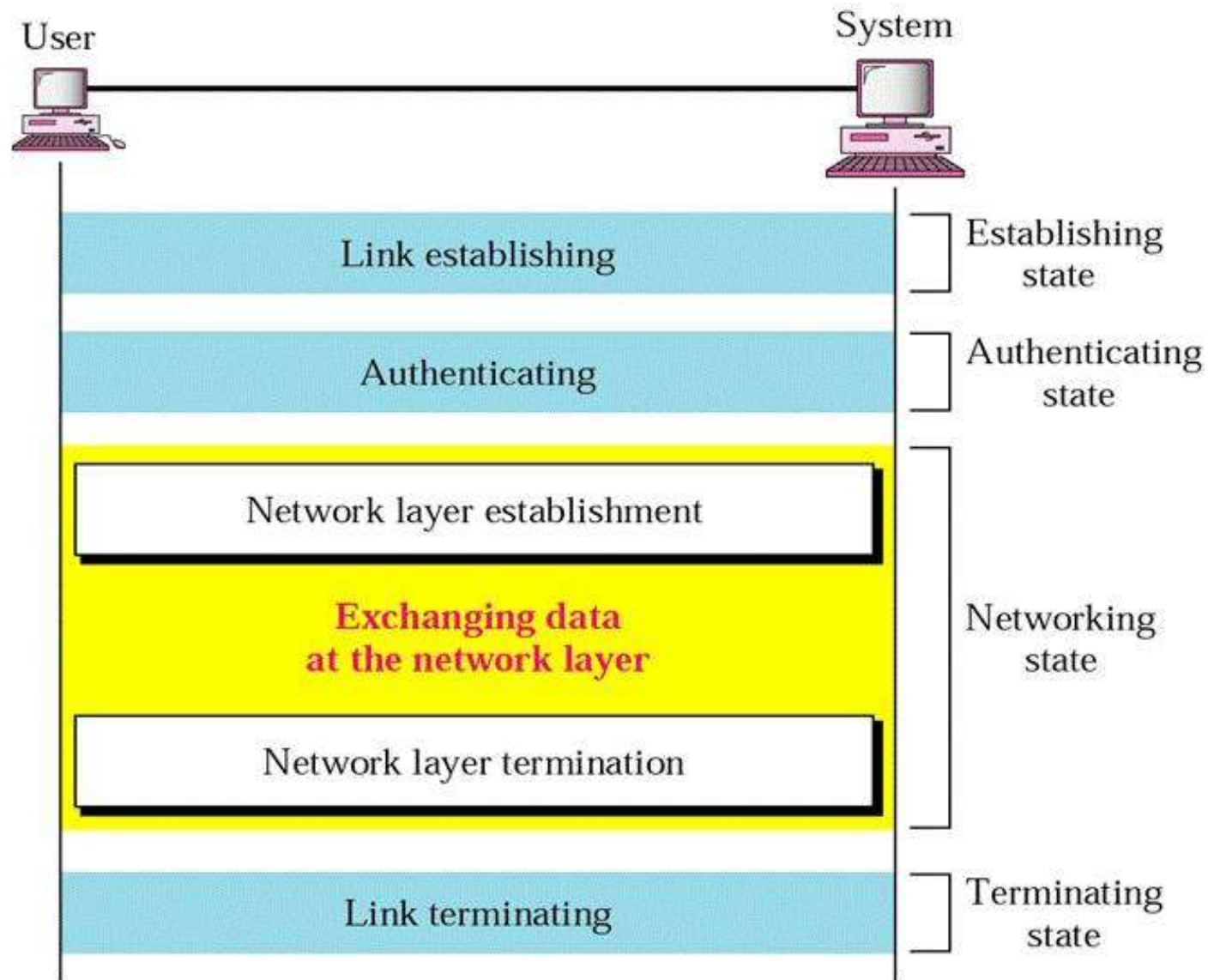


- *Address*: 0xFF indicates "all stations"
- *Control*: 0x03 indicates unnumbered mode
- *Protocol*: indicating type of payload, supporting LCP, NCP, IP, IPX, AppleTalk...
- *Checksum*: using CRC to check errors

# PPP: Phase Diagram



# PPP: An Example



# Key points of Data Link Layer

## ■ Functions and services of data link layer

## ■ Framing method

- ◆ Byte stuffing
- ◆ Bit stuffing
- ◆ Physical layer coding violations

## ■ Error Control

- ◆ Correcting Code: Hamming code
- ◆ Error Detecting Code: CRC

## ■ Flow control: ARQ

- ◆ Stop-and-wait:  $W_T = W_R = 1, U = 1/(1+2a)$
  - ◆ Go-back-N:  $1 < W_T < 2^m, W_R = 1$
  - ◆ Selective Repeat:  $W_T = W_R = 2^{m-1}$
- $\longrightarrow U = \begin{cases} 1 & W \geq 2a+1 \\ \frac{W}{2a+1} & W < 2a+1 \end{cases}$

## ■ Practical protocols

- ◆ HDLC: Usage, frame types
- ◆ PPP: Usage, frame structures



# Terms

| 缩写          | 英文全称                               | 中文       |
|-------------|------------------------------------|----------|
|             | access control                     | 访问(接入)控制 |
| ACK         | Acknowledgement                    | 确认(帧)    |
|             | addressing                         | 寻址       |
| ARQ         | Automatic Repeat reQuest           | 自动请求重发   |
| ATM         | Asynchronous Transfer Mode         | 异步传输模式   |
|             | bandwidth-delay product            | 带宽时延积    |
| BER         | Bit Error Rate                     | 误码率      |
|             | bit rate                           | 数据率(比特率) |
|             | bit stuffing                       | (零)比特填充  |
|             | byte stuffing                      | 字节填充     |
|             | burst error                        | 突发差错     |
|             | carrier                            | 载波       |
|             | character count                    | 字符计数法    |
|             | Character Stuffing                 | 字符填充     |
| Spring 2020 | codeword                           | 码字       |
|             | Computer Networks, Data Link Layer |          |

| 缩写  | 英文全称                            | 中文            |
|-----|---------------------------------|---------------|
|     | cumulative acknowledgement      | 累积 <b>ACK</b> |
| CRC | Cyclic Redundancy Check         | 循环冗余校验        |
|     | data link layer                 | 数据链路层         |
|     | dial-up                         | 拨号            |
|     | encapsulation                   | 封装            |
|     | error control                   | 差错控制          |
|     | error syndrome                  | 差错综合症         |
|     | error correction                | 差错纠正          |
|     | Error Correcting Code           | 纠错码           |
|     | error detection                 | 差错检测          |
|     | Ethernet                        | 以太网           |
|     | flag byte                       | 标志字节          |
| FCS | Frame Check Sequence            | 帧校验序列         |
| FDM | Frequency Division Multiplexing | 频分复用          |

| 缩写  | 英文全称                        | 中文        |
|-----|-----------------------------|-----------|
|     | flow control                | 流量控制      |
| FEC | Forward Error Correction    | 前向纠错      |
|     | Feedback-based flow control | 基于反馈的流量控制 |
|     | frame                       | 帧         |
|     | framing                     | 成帧        |
|     | frame header                | 帧头        |
|     | full-duplex                 | 全双工       |
|     | Generator polynomial        | 生成多项式     |
|     | Go-Back-N                   | 回退N步(协议)  |
|     | Hamming code                | 汉明(海明)码   |
|     | hamming distance            | 汉明距离      |
|     | HUB                         | 集线器       |
| LAN | Local Area Network          | 局域网       |
| LAP | Link Access Procedure       | 链路接入规程    |

| 缩写          | 英文全称                                  | 中文           |
|-------------|---------------------------------------|--------------|
| LAPB        | Link Access Procedure - Balanced mode | 链路接入规程——平衡模式 |
| LAPD        | Link Access Procedure, on D channel   | D信道上的链路接入规程  |
| LCP         | Link Control Protocol                 | 链路控制协议       |
|             | Line Utilization                      | 线路利用率        |
| LLC         | Logical Link Control                  | 逻辑链路控制       |
| MAC         | Media Access Control                  | 媒体(介质)访问控制   |
|             | Manchester Encoding                   | 曼彻斯特编码       |
| HDLC        | High-Level Data Link Control          | 高级数据链路控制     |
|             | hop                                   | 跳            |
|             | hop by hop                            | 逐跳           |
| NAK         | Negative acknowledgement              | 否认(帧)        |
| NCP         | Network Control Protocol              | 网络控制协议       |
| NIC         | Network Interface Card                | 网络接口卡        |
|             | Odd & Even Parity                     | 奇偶校验         |
| Spring 2020 | Computer Networks, Data Link Layer    |              |

| 缩写    | 英文全称                                         | 中文          |
|-------|----------------------------------------------|-------------|
| PAR   | Positive Acknowledgement with Retransmission | 带确认的重传      |
|       | packet                                       | 分组/包        |
|       | Physical layer coding violations             | 物理层编码违例法    |
|       | Packet Switching                             | 分组交换        |
|       | Parity check                                 | 奇偶校验        |
|       | payload                                      | 净荷（数据）      |
|       | piggybacking                                 | 捎带确认(应答)    |
|       | pipelining                                   | 流水线         |
| PPP   | Point-to-Point Protocol                      | 点对点协议       |
| PPPoE | PPP over Ethernet                            | 以太网之上的点对点协议 |
|       | Polynomial Code                              | 多项式编码       |
|       | preamble                                     | 前导码         |
|       | propagation delay                            | 传播时延        |
|       | rate-based flow control                      | 基于速率的流量控制   |
|       | round-trip propagation time                  | 环回传播时间      |

| 缩写    | 英文全称                        | 中文       |
|-------|-----------------------------|----------|
| SLIP  | Serial Line IP              | 串行线IP    |
| SONET | Synchronous Optical Network | 同步光网络    |
|       | Selective Repeat            | 选择重传(协议) |
|       | sliding window              | 滑动窗口     |
|       | Stop-and-wait               | 停止等待(协议) |
|       | store-and-forward switching | 存储转发交换   |
|       | subnet                      | 子网       |
|       | switch                      | 交换机      |
|       | switched Ethernet           | 交换式以太网   |
| TDM   | Time Division Multiplexing  | 时分复用     |
|       | Timeout Timer               | 超时定时器    |
|       | transparent transmission    | 透明传输     |
|       | transmission delay          | 发送时延     |
|       | transport layer             | 传输层      |
| WAN   | Wide Area Network           | 广域网      |
| WLAN  | Wireless Local Area Network | 无线局域网    |

# Copyright Information

- Some figures used in this presentation have been either directly copied or adapted from following materials
  - ◆ Lecture notes of book “Computer Networks, a Top-down Approach”, J.Kurose and W.Ross, 3<sup>rd</sup> Edition, which are marked with *[Kurose]*
  - ◆ “Data communications and networking”, B.A.Forouzan, which are marked with *[Forouzan]*
  - ◆ “Data and Computer Communications”, William Stallings, which are marked with *[Stallings]*
  - ◆ “计算机网络”, 谢希仁, 第五版, 电子工业出版社, 2008年, which are marked with *[谢]*

# The Calculation of CRC (Software)

## ■ Software (from RFC1662)

### ◆ FCS: Frame Check Sequence, 16bits

```
static u16 fcstab[256] = {
    0x0000, 0x1189, 0x2312, 0x329b, 0x4624, 0x57ad, 0x6536,
    0x74bf, 0x8c48, 0x9dc1, 0xaf5a, 0xbed3, 0xca6c, 0xdbe5,
    .....,
    0xd49d, 0xc514, 0xb1ab, 0xa022, 0x92b9, 0x8330, 0x7bc7,
    0x6a4e, 0x58d5, 0x495c, 0x3de3, 0x2c6a, 0x1ef1, 0x0f78
};

u16 fcs16(u16 fcs, unsigned char *p, int len)
{
    while (len--)
        fcs = (fcs >> 8) ^ fcstab[(fcs ^ *p++) & 0xff];
    return (fcs);
}
```